

Java

Frameworks e Aplicações Corporativas

Razer Anthom

A RNP – Rede Nacional de Ensino e Pesquisa – é qualificada como uma Organização Social (OS), sendo ligada ao Ministério da Ciência, Tecnologia e Inovação (MCTI) e responsável pelo Programa Interministerial RNP, que conta com a participação dos ministérios da Educação (MEC), da Saúde (MS) e da Cultura (MinC). Pioneira no acesso à Internet no Brasil, a RNP planeja e mantém a rede Ipê, a rede óptica nacional acadêmica de alto desempenho. Com Pontos de Presença nas 27 unidades da federação, a rede tem mais de 800 instituições conectadas. São aproximadamente 3,5 milhões de usuários usufruindo de uma infraestrutura de redes avançadas para comunicação, computação e experimentação, que contribui para a integração entre o sistema de Ciência e Tecnologia, Educação Superior, Saúde e Cultura.



RNP

MINISTÉRIO DA
DEFESA

MINISTÉRIO DA
CULTURA

MINISTÉRIO DA
SAÚDE

MINISTÉRIO DA
EDUCAÇÃO

MINISTÉRIO DA
**CIÊNCIA, TECNOLOGIA,
INOVAÇÕES E COMUNICAÇÕES**





JAVA

Frameworks e Aplicações Corporativas

Razer Anthom



JAVA

Frameworks e Aplicações Corporativas

Razer Anthom

Rio de Janeiro
Escola Superior de Redes
2016

Copyright © 2016 – Rede Nacional de Ensino e Pesquisa – RNP
Rua Lauro Müller, 116 sala 1103
22290-906 Rio de Janeiro, RJ

Diretor Geral
Nelson Simões

Diretor de Serviços e Soluções
José Luiz Ribeiro Filho

Escola Superior de Redes

Coordenação
Leandro Marcos de Oliveira Guimarães

Edição
Lincoln da Mata

Coordenador Acadêmico da Área de Desenvolvimento de Sistemas
John Lemos Forman

Equipe ESR (em ordem alfabética)
Adriana Pierro, Alynne Pereira, Celia Maciel, Derlinéa Miranda, Edson Kowask, Elimária Barbosa, Evellyn Feitosa, Felipe Nascimento, Lourdes Soncin, Luciana Batista, Luiz Carlos Lobato, Renato Duarte e Yve Marcial.

Capa, projeto visual e diagramação
Tecnodesign

Versão
1.0.0

Este material didático foi elaborado com fins educacionais. Solicitamos que qualquer erro encontrado ou dúvida com relação ao material ou seu uso seja enviado para a equipe de elaboração de conteúdo da Escola Superior de Redes, no e-mail info@esr.rnp.br. A Rede Nacional de Ensino e Pesquisa e os autores não assumem qualquer responsabilidade por eventuais danos ou perdas, a pessoas ou bens, originados do uso deste material.
As marcas registradas mencionadas neste material pertencem aos respectivos titulares.

Distribuição
Escola Superior de Redes
Rua Lauro Müller, 116 – sala 1103
22290-906 Rio de Janeiro, RJ
<http://esr.rnp.br>
info@esr.rnp.br

Dados Internacionais de Catalogação na Publicação (CIP)

C385m Anthom, Razer
 Java - Frameworks e Aplicações Corporativas / Razer Anthom. – Rio de Janeiro: RNP/ESR,
 2016
 182 p. : il. ; 28 cm.

ISBN 978-85-63630-50-6

1. Arquitetura Java EE e Servidores de Aplicação. 2. JSF (Java Server Faces). 3. Framework Hibernate. I. Título

CDD 000

Sumário

Escola Superior de Redes

A metodologia da ESR ix

Sobre o curso x

A quem se destina x

Convenções utilizadas neste livro xi

Permissões de uso xi

Sobre o autor xii

1. Introdução

Arquitetura Java EE 1

EJB 5

JavaServer Faces 6

Servidores de Aplicação 6

Padrões de Projeto Front Controller e MVC 8

Arquitetura JSF 9

Primeiro projeto 12

Comparação Servlets e JSF 13

Aplicação Servlets 13

Aplicação JSF 15

Comparação 16

2. Java Server Faces – Introdução

Injeção de Dependência e Inversão de Controle 17

XHTML e Managed Beans 21

Ações 23

Escopos 25

Processamento de uma Requisição 28

Process Events 29

Restore View 29

Apply Request Values 30

Process Validations 30

Update Model Values 30

Invoke Application 30

Render Response 31

Ciclo de vida simplificado 31

Navegação 31

3. JSF – Componentes visuais

Estrutura básica 33

Formulários 34

Binding e processamento 36

Caixas de texto, rótulos e campos ocultos 37

Caixas de texto 37

Caixas de texto de múltiplas linhas 38

Caixas de texto de senha 39

Exemplo de caixas de texto 40

Rótulos 40

Campos ocultos 41

Caixas de Seleção 42

Exercícios 47

4. JSF – Componentes visuais

Botões e links 59

Botão de Ação <h:commandButton> 60

Link de ação: <h:commandLink> 60

Botão <h:button> 61



Link <h:link>	62
Link Externo: <h:outputLink>	62
Exemplo com botões e links	63
Textos	64
Textos simples	64
Textos formatados	65
Imagens	65
Biblioteca de recursos	66
Versionamento de Recursos	67
JavaScript e CSS	68
Atributo Rendered	68
Componentes de organização	69
Tabelas	70
Mensagens	73
Repetição	75

5. JSF – Tratamento de dados e eventos

Páginas e templates	77
Inclusão de páginas	77
Templates	78
Conversores	80
Conversão de números	81
Conversão de datas ou horas	82
Armazenar um objeto em um MB em vez de uma string	84
Mensagem de erro de conversão	87
Validadores	88
Bean Validation	90
Validador Personalizado	91
Eventos	92
ValueChangeEvent	94
Atributo immediate	96

6. JSF – Internacionalização, AJAX e Primefaces

Internacionalização 99

Arquivos de mensagens 99

Managed Bean de internacionalização 100

XHTML com Internacionalização 101

AJAX 102

Eventos 102

Componentes 104

Atualização 104

Método Invocado 105

Resumo 105

Primefaces 106

Layout – <p:layout> 107

DataTable – <p:dataTable> 111

Calendar – <p:calendar> 112

InputMask – <p:inputMask> 114

Editor – <p:editor> 115

PickList – <p:pickList> 116

Google Maps – <p:gmap> 117

Accordion Panel – <p:accordionPanel> 119

Menus – <p:menu> 119

Growl – <p:growl> 124

7. Hibernate

Introdução 125

Exercício de Fixação 126

Classe persistente 128

Acesso simples ao banco de dados: inserção e consulta 129

Managed Bean e XHTMLs 130

Arquivo de Configuração: hibernate.cfg.xml 132

Conteúdo de uma aplicação 133

Exercício de Fixação 133

Conceitos e Ciclo de Vida 133

Anotações 134

Atributos transientes 139

Método de acesso aos atributos 139

Interfaces do Hibernate 139



8. Hibernate – Associações

Associações 143

Cascadeamento 143

9. Hibernate – HQL e Criteria

HQL 159

Joins 164

Cláusula SELECT 166

Ordenação com Order By 166

Cláusulas Group By e Having 167

Criteria 167

Ordenação 168

Restrições 168

Combinação de Restrições: AND e OR 170

Projeções 171

JSF e Hibernate 171

10. Java EE

Java EE 175

Objetos distribuídos 176

Transações 182

CMT 183

BMT 183

web Services 184

SOAP 185

REST 188

SOA – Arquitetura Orientada a Serviços 191



Escola Superior de Redes

A Escola Superior de Redes (ESR) é a unidade da Rede Nacional de Ensino e Pesquisa (RNP) responsável pela disseminação do conhecimento em Tecnologias da Informação e Comunicação (TIC). A ESR nasce com a proposta de ser a formadora e disseminadora de competências em TIC para o corpo técnico-administrativo das universidades federais, escolas técnicas e unidades federais de pesquisa. Sua missão fundamental é realizar a capacitação técnica do corpo funcional das organizações usuárias da RNP, para o exercício de competências aplicáveis ao uso eficaz e eficiente das TIC.

A ESR oferece dezenas de cursos distribuídos nas áreas temáticas: Administração e Projeto de Redes, Administração de Sistemas, Segurança, Mídias de Suporte à Colaboração Digital e Governança de TI.

A ESR também participa de diversos projetos de interesse público, como a elaboração e execução de planos de capacitação para formação de multiplicadores para projetos educacionais como: formação no uso da conferência web para a Universidade Aberta do Brasil (UAB), formação do suporte técnico de laboratórios do Proinfo e criação de um conjunto de cartilhas sobre redes sem fio para o programa Um Computador por Aluno (UCA).

A metodologia da ESR

A filosofia pedagógica e a metodologia que orientam os cursos da ESR são baseadas na aprendizagem como construção do conhecimento por meio da resolução de problemas típicos da realidade do profissional em formação. Os resultados obtidos nos cursos de natureza teórico-prática são otimizados, pois o instrutor, auxiliado pelo material didático, atua não apenas como expositor de conceitos e informações, mas principalmente como orientador do aluno na execução de atividades contextualizadas nas situações do cotidiano profissional.

A aprendizagem é entendida como a resposta do aluno ao desafio de situações-problema semelhantes às encontradas na prática profissional, que são superadas por meio de análise, síntese, julgamento, pensamento crítico e construção de hipóteses para a resolução do problema, em abordagem orientada ao desenvolvimento de competências.

Dessa forma, o instrutor tem participação ativa e dialógica como orientador do aluno para as atividades em laboratório. Até mesmo a apresentação da teoria no início da sessão de aprendizagem não é considerada uma simples exposição de conceitos e informações. O instrutor busca incentivar a participação dos alunos continuamente.

As sessões de aprendizagem onde se dão a apresentação dos conteúdos e a realização das atividades práticas têm formato presencial e essencialmente prático, utilizando técnicas de estudo dirigido individual, trabalho em equipe e práticas orientadas para o contexto de atuação do futuro especialista que se pretende formar.

As sessões de aprendizagem desenvolvem-se em três etapas, com predominância de tempo para as atividades práticas, conforme descrição a seguir:

Primeira etapa: : apresentação da teoria e esclarecimento de dúvidas (de 60 a 90 minutos).

O instrutor apresenta, de maneira sintética, os conceitos teóricos correspondentes ao tema da sessão de aprendizagem, com auxílio de slides em formato PowerPoint. O instrutor levanta questões sobre o conteúdo dos slides em vez de apenas apresentá-los, convidando a turma à reflexão e participação. Isso evita que as apresentações sejam monótonas e que o aluno se coloque em posição de passividade, o que reduziria a aprendizagem.

Segunda etapa: atividades práticas de aprendizagem (de 120 a 150 minutos).

Esta etapa é a essência dos cursos da ESR. A maioria das atividades dos cursos é assíncrona e realizada em duplas de alunos, que acompanham o ritmo do roteiro de atividades proposto no livro de apoio. Instrutor e monitor circulam entre as duplas para solucionar dúvidas e oferecer explicações complementares.

Terceira etapa: : discussão das atividades realizadas (30 minutos).

O instrutor comenta cada atividade, apresentando uma das soluções possíveis para resolvê-la, devendo ater-se àquelas que geram maior dificuldade e polêmica. Os alunos são convidados a comentar as soluções encontradas e o instrutor retoma tópicos que tenham gerado dúvidas, estimulando a participação dos alunos. O instrutor sempre estimula os alunos a encontrarem soluções alternativas às sugeridas por ele e pelos colegas e, caso existam, a comentá-las.

Sobre o curso

Este é um curso de nível avançado com foco no uso de frameworks e tecnologias para desenvolvimento de aplicações corporativas em Java. Este tipo de aplicação exige um grau de confiabilidade e performance mais elevado, fazendo uso de recursos específicos do Java Enterprise Edition - Java EE, tais como JSF (Java Server Faces), AJAX, Primefaces e Hibernate.

O curso se inicia com uma visão geral da Arquitetura do Java EE e características dos servidores de aplicação capazes de suportar as tecnologias/aplicações corporativas, para em seguida se aprofundar no JSF e Hibernate.

Cada sessão apresenta um conjunto de exemplos e atividades práticas que permitem a prática das habilidades apresentadas.

A quem se destina

Pessoas interessadas em desenvolver aplicações corporativas com maior grau de confiabilidade e performance. É um curso recomendado para quem já tem bons conhecimentos sobre a linguagem Java, desde seus fundamentos e o desenvolvimento de aplicações web até acesso a bancos de dados. Para os alunos que não tenham a prática recente de desenvolvimento de sistemas nesta linguagem recomenda-se fortemente que considerem se matricular previamente nos seguintes cursos: DES2 – Java – Interfaces Gráficas e Bancos de Dados; e DES3 – Java – Aplicações Web.

Convenções utilizadas neste livro

As seguintes convenções tipográficas são usadas neste livro:

Itálico

Indica nomes de arquivos e referências bibliográficas relacionadas ao longo do texto.

Largura constante

Indica comandos e suas opções, variáveis e atributos, conteúdo de arquivos e resultado da saída de comandos. Comandos que serão digitados pelo usuário são grifados em negrito e possuem o prefixo do ambiente em uso (no Linux é normalmente # ou \$, enquanto no Windows é C:\).

Conteúdo de slide

Indica o conteúdo dos slides referentes ao curso apresentados em sala de aula.

Símbolo

Indica referência complementar disponível em site ou página na internet.

Símbolo

Indica um documento como referência complementar.

Símbolo

Indica um vídeo como referência complementar.

Símbolo

Indica um arquivo de áudio como referência complementar.

Símbolo

Indica um aviso ou precaução a ser considerada.

Símbolo

Indica questionamentos que estimulam a reflexão ou apresenta conteúdo de apoio ao entendimento do tema em questão.

Símbolo

Indica notas e informações complementares como dicas, sugestões de leitura adicional ou mesmo uma observação.

Símbolo

Indica atividade a ser executada no Ambiente Virtual de Aprendizagem – AVA.

Permissões de uso

Todos os direitos reservados à RNP.

Agradecemos sempre citar esta fonte quando incluir parte deste livro em outra obra.

Exemplo de citação: TORRES, Pedro et al. *Administração de Sistemas Linux: Redes e Segurança*.

Rio de Janeiro: Escola Superior de Redes, RNP, 2013.

Comentários e perguntas

Para enviar comentários e perguntas sobre esta publicação:

Escola Superior de Redes RNP

Endereço: Av. Lauro Müller 116 sala 1103 – Botafogo

Rio de Janeiro – RJ – 22290-906

E-mail: info@esr.rnp.br

Sobre o autor

Razer Anthom Nizer Rojas Montañó é Doutorando em Informática (ênfase em Inteligência Artificial) pela UFPR, Mestre e Bacharel em Informática pela UFPR. Atualmente é professor da UFPR ministrando disciplinas de desenvolvimento em Java Web e de Aplicações Corporativas. Possui certificação SCJP, COBIT, ITIL. Acumula mais de 15 anos de experiência em docência e mais de 20 anos de experiência no mercado de desenvolvimento, análise e arquitetura de aplicações.

John Lemos Forman é Mestre em Informática (ênfase em Engenharia de Software) e Engenheiro de Computação pela PUC-Rio, com pós-graduação em Gestão de Empresas pela COPPEAD/UF RJ. É vice-presidente do Sindicato das Empresas de Informática do Rio de Janeiro – TIRIO, Presidente do Conselho Deliberativo da Riosoft e membro do Conselho Consultivo e de normas Éticas da Assespro-RJ. É sócio e Diretor da J.Forman Consultoria e coordenador acadêmico da área de desenvolvimento de sistemas da Escola Superior de Redes da RNP. Acumula mais de 30 anos de experiência na gestão de empresas e projetos inovadores de base tecnológica, com destaque para o uso das TIC na Educação, mídias digitais, Saúde e Internet das Coisas.

1

Introdução

objetivos

Conhecer o Java EE; Aprender sobre o histórico da evolução da especificação;
Conhecer os servidores de aplicação e um primeiro projeto utilizando essa arquitetura.

Arquitetura Java EE, JCP, EJB, JSF; Servidores de Aplicação; Container;
Front Controller e MVC.

conceitos

Arquitetura Java EE

Java EE (Java Platform, Enterprise Edition) é a plataforma padrão para desenvolvimento de aplicações corporativas em Java. Inclui diversas tecnologias que dão suporte a esse tipo de aplicação, como desenvolvimento Web, Web Services, Componentes distribuídos, Componentes de persistência etc.

O Java SE (Java Platform, Standard Edition) é a plataforma básica do Java, usada para desenvolver aplicativos baseados em console e aplicativos desktop (visuais). Para executar essas aplicações, basta o JVM (Java Virtual Machine), que é a máquina virtual do Java. Já para executar aplicações Java EE, faz-se necessário o uso da JVM e de um servidor de aplicação com suporte às tecnologias disponíveis na plataforma.

A figura a seguir mostra as tecnologias disponíveis. Entre as mais difundidas, temos:

- **Servlets/JSP (JavaServer Pages):** tecnologias básicas para desenvolvimento de aplicações web, componentes que respondem a requisições HTTP;
- **JSF (JavaServer Faces):** framework baseado em Servlets usado para simplificar o desenvolvimento de aplicações web;
- **JPA (Java Persistence API):** tecnologia de mapeamento objeto-relacional, usada para criar componentes de persistência;
- **EJB (Enterprise Java Beans):** componentes de negócio que podem ser executados de forma distribuída, portátil, baseados em transações etc.;
- **Web Services:** serviços disponibilizados na internet, com invocação padronizada, que podem ser acessados por qualquer componente de software.



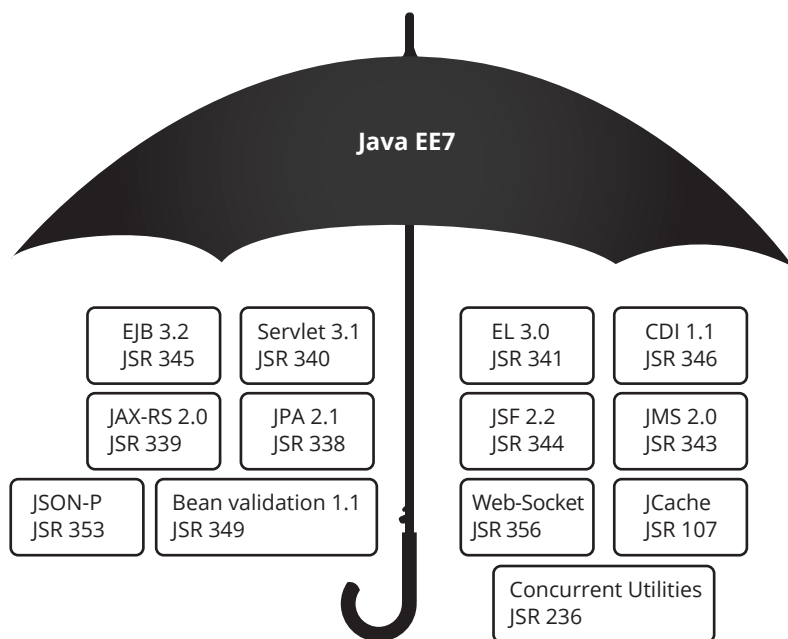


Figura 1.1
Tecnologias Java EE.

O Java EE é um conjunto de especificações, mantido pela JCP (Java Community Process – <http://www.jcp.org>), nas quais todas as tecnologias são descritas. O JCP é formado por especialistas, empresas e comunidade de desenvolvedores, e tem como objetivo padronizar a tecnologia Java, criando as JSRs (Java Specification Requests) para definir a evolução destas.



As tecnologias com foco em aplicações corporativas (enterprise) no início eram conhecidas como J2EE, mas com a alteração de nomenclatura a partir da versão 1.5 do Java, passaram a se chamar Java EE.

Versões do Java EE:

- J2EE 1.2 – 12/12/1999;
- J2EE 1.3 – 24/9/2001;
- J2EE 1.4 – 11/11/2003;
- Java EE 5 – 11/5/2006;
- Java EE 6 – 10/12/2009;
- Atualmente: Java EE 7 – 5/4/2013.

A tabela 1.1 apresenta as tecnologias presentes na especificação Java EE 7¹.

Web Application Technologies

Java API for WebSocket: protocolo que permite comunicação full-duplex entre dois pontos sobre TCP.

Java API for JSON Processing: API para intercâmbio de dados com base na JavaScript Object Notation: JSON

Java Servlet: Invocação de classes via HTTP

JavaServer Faces (JSF): Framework para desenvolvendo de aplicações web

Web Application Technologies
JavaServer Pages (JSP): tecnologia para inserir scriptlets Java em páginas HTML
Expression Language (EL): linguagem de expressões para manipulação de dados em JSP e JSF
Standard Tag Library for JavaServer Pages (JSTL): biblioteca de tags para uso com JSP
Enterprise Application Technologies
Batch Applications for the Java Platform: tarefas que podem ser executadas sem a interação do usuário
Concurrency Utilities for Java EE: API que provê funcionalidade assíncrona para componentes Java EE
Contexts and Dependency Injection for Java: serviços padrão de gestão de contexto e injeção de dependências
Dependency Injection for Java: conjunto de anotações para classes injetáveis
Bean Validation: modelo para validação de dados dentro de beans
Enterprise JavaBeans (EJB): componentes gerenciados e distribuídos
Interceptors: interceptação de chamadas a métodos de EJBs
Java EE Connector Architecture: API para integração de sistemas
Java Persistence API (JPA): solução padrão do Java para persistência
Common Annotations for the Java Platform: anotações para programação declarativa em Java
Java Message Service API: API de mensagens, para comunicação entre componentes
Java Transaction API (JTA): solução Java para controle e delimitação de transações
JavaMail API: API para envio de e-mails
Web Services Technologies
Java API for RESTful web Services (JAX-RS): API para web Services usando REST
Implementing Enterprise web Services: modelo programático e em tempo de execução para implantação e busca de web Services
Java API for XML-Based web Services (JAX-WS): suporte para web Services usando a API JAXB
Web Services Metadata for the Java Platform: anotações de metadados de web Services para fácil definição
Java API for XML-based RPC (JAX-RPC) (Opcional): API de construção de web Services usando RPC
Java APIs for XML Messaging (JAXM): API padrão para comunicação de XML na internet
Java API for XML Registries (JAXR): serviços de registros distribuídos para integração B2B
Management and Security Technologies
Java Authentication Service Provider Interface for Containers (JASPIC): API para criação de provedores de serviço para autenticação

Management and Security Technologies
Java Authorization Contract for Containers (JACC): define um contrato entre aplicação Java EE e servidor de políticas de autorização
Java EE Application Deployment (Opcional): gestão dos recursos de uma aplicação entre implantações e atualizações da aplicação
J2EE Management: mapeamento de padrões de gerenciamento para Java, como o SNMP
Debugging Support for Other Languages: ferramentas padronizadas para relacionamento do bytecode com outras linguagens que não Java
Java EE-related Specs in Java SE
Java Architecture for XML Binding (JAXB): arquitetura para ligar um XML a uma representação em programa Java
Java API for XML Processing (JAXP): processamento de arquivos XML
Java Database Connectivity: API para conexão e execução de SQL em servidores de banco de dados
Java Management Extensions (JMX): forma padrão de tratamento de recursos (aplicações, dispositivos e serviços)
JavaBeans Activation Framework (JAF): determinação automática do tipo de um pedaço de código para instanciação do bean correto
Streaming API for XML (StAX): API baseada em streaming para leitura e escrita de documentos XML

Tabela 1.1
Tecnologias Java EE.

Mais à frente são apresentados alguns modelos de programação em uso atualmente, bem como servidores de aplicações corporativas, indispensáveis para executar aplicações Java EE (p.e. Glassfish ou JBoss). É importante saber que existem servidores web, como o Tomcat, que também suportam aplicações web, mas não aplicações Java EE.

O Java EE demanda um servidor web para executar as aplicações.

- Aplicações somente web:
 - ▣ Servidor WEB;
 - ▣ Container Servlet.
 - ▣ Servlet, JSP, JSTL, JSF.
 - ▣ Exemplo: tomcat.
- Aplicações Corporativas:
 - ▣ Servidor JavaEE;
 - ▣ Exemplo: glassfish, Jboss.



A evolução do Java EE também trouxe avanços no modelo de programação usado. Antigamente era prática comum o uso de vários arquivos XML para configuração de componentes e descritores de aplicação. Hoje em dia usam-se anotações em vários pontos de uma aplicação. Por exemplo, na API Servlets 2 era necessário configurar o arquivo web.xml para descrever as Servlets. Já na versão 3 em diante basta anotar uma classe com @WebServlet que já se indica que a classe é uma Servlet, sem a necessidade do arquivo XML.



Antigamente – XML Hell:

- Muitos arquivos XML para serem configurados (deployment descriptors).

Hoje:

- XML é opcional na maioria dos casos;
- Usam-se anotações (@) para configurar os componentes, configurando-os em tempo de deploy e em tempo de execução.

Vantagem:

- A configuração está próxima do componente a que se refere.

Vejamos a seguir alguns componentes e frameworks utilizados em Java EE e que serão detalhados ao longo deste curso.

EJB

EJB (Enterprise Java Beans) são componentes de software da plataforma Java EE, usados no desenvolvimento de aplicações de grande porte/corporativas.

Entre as características dos EJB, temos:

- **Distribuídos:** que podem ser executados em containers ou servidores diferentes, fazendo parte do mesmo sistema;
- **Reusáveis:** podemos aproveitá-los em outros projetos ou subsistemas do mesmo projeto;
- **Produtividade:** seu desenvolvimento é fácil, rápido e componentizado, sendo que os servidores fazem toda a gestão de seu ciclo de vida.

Atualmente, a versão oficial do EJB é a 3.2, especificada pela JSR 345. Entre suas contribuições temos o maior uso de anotações para configurações de componentes e seus comportamentos e preparação para **cloud computing** (Java EE como consumidor de serviços na nuvem, como integrador de serviços – vide JBI – Java Business Integration – e como produtor de serviços na nuvem – PaaS). O servidor GlassFish 4, descrito em mais detalhes adiante, já está preparado para esta tecnologia.

Cloud computing

“Computação nas nuvens”, em inglês. É o armazenamento de arquivos ou aplicativos na internet.

Em EJB podem ser definidos, basicamente, quatro tipos de componentes:

- **Stateless Session Beans (@Stateless):** componente negocial que não guarda o estado de seus atributos entre uma requisição e outra;
- **Stateful Session Beans (@Stateful):** componente negocial que guarda o estado de seus atributos entre requisições;
- **Singleton Session Beans (@Singleton):** componente negocial que possui a restrição de somente haver uma instância na aplicação (padrão de projeto **singleton**);
- **Message-Driven Beans (@MessageDriven):** componente baseado em mensagens, onde o processamento é feito de forma assíncrona.

Singleton

Padrão de projeto do GOF (Gamma et al) onde, para uma determinada classe, somente será possível se obter uma instância, isto é, o sistema todo acessará sempre o mesmo objeto.



JavaServer Faces

JavaServer Faces (JSF) é uma especificação integrante do Java EE, que tem por objetivo ser um framework de desenvolvimento web. Atualmente está na versão 2.2 (JSR 344).

Em relação a outros frameworks, o JSF se mostra muito maduro, visto que teve sua adoção como padrão na especificação Java EE 6. Também é um framework extensível e que vem melhorando as tecnologias, como fácil integração com HTML5, AJAX etc. Também há muito envolvimento com a comunidade, o que torna o JSF proeminente e foco de testes e atualizações.

O JSF também opera através de requisição e resposta, mas possui um ciclo de vida bem definido. É baseado em páginas XHTML ligadas a componentes conhecidos como Managed Beans. O processamento de uma requisição possui um ciclo de vida bem definido, que permite a manutenção do estado dos componentes ao longo de requisições.

Java Server Faces – JSF.

- Funciona via requisição e resposta:
 - ▣ O processamento de uma requisição se dá através de um ciclo de vida bem definido;
 - ▣ Resolve o problema de não manutenção de estado de componentes.

Duas tecnologias são fundamentais para o JSF:

- **Facelets**
 - ▣ Tecnologia de templates para telas em JSF (), sem usar o tradicional JSP;
 - ▣ Integração com EL – Expression Language (`{bean.propriedade}`);
 - ▣ Arquivos XHTML.
- **Managed Beans**
 - ▣ classes Java (POJOs);
 - ▣ Beans gerenciados (ciclo de vida) pelo framework (CDI);
 - ▣ Componentes que armazenam informações e executam ações.

Outra grande vantagem em se usar JSF é a integração com bibliotecas de componentes ricos, como o PrimeFaces, que traz uma vasta coleção de componentes baseados em jQuery (<http://www.jquery.org>) e totalmente integrados com o JSF.

Requisitos JSF.

- Executa sobre um Container:
 - ▣ GlassFish 4.0 com JDK 7;
 - ▣ GlassFish 3.1+ com JDK 6;
 - ▣ Tomcat 7 (necessário adicionar CDI);

Pode integrar bibliotecas de componentes ricos.

- PrimeFaces – <http://www.primefaces.org>.

Servidores de Aplicação

Para que aplicativos Java EE sejam disponibilizados, faz-se necessário o uso de Servidores de Aplicações, que oferecem toda a infraestrutura para execução de aplicações com as tecnologias disponibilizadas na plataforma. A figura 1.2 mostra as diversas camadas de um servidor Java EE, apresentando seus contêineres e suas relações.

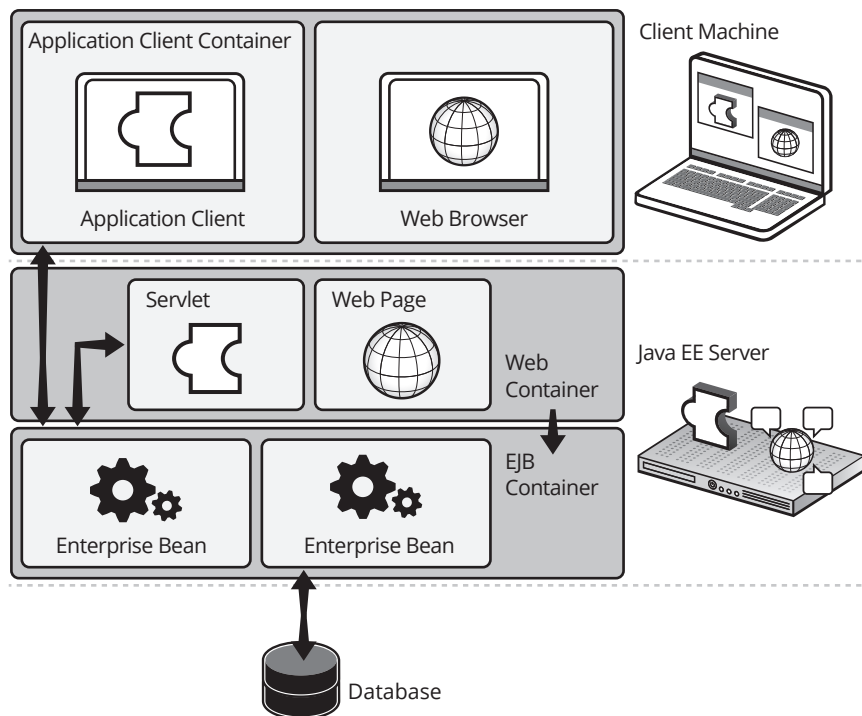


Figura 1.2
Servidor Java EE
e Containers.

Um Container é uma interface entre um componente e uma funcionalidade de baixo nível da plataforma, que o suporta. Por exemplo, Servlets e seu tratamento na plataforma Java.

Existem basicamente dois tipos de Containers:

- **Servlet Container:**
 - ▣ Container web;
 - ▣ Usado para execução de aplicações web;
 - ▣ Implementam somente as tecnologias web (Servlets, JSP, JSF, EL e JSTL).
- **EJB Container:**
 - ▣ Containers usados para execução de componentes EJB.

O servidor usado nesse material é o Oracle GlassFish 4, que é um Servidor Java EE completo, com Servlet Container e EJB Container. Ele é empacotado na instalação do Netbeans e já é configurado de forma automática. O GlassFish é a implementação de referência do Java EE.

Oracle GlassFish 4

- Implementação de Referência: <https://glassfish.java.net/>
- Servidor de aplicação: implementa toda a especificação Java EE.
- Para iniciá-lo em linha de comando:
 - ▣ Ir até: <Diretório do GLASSFISH>/bin
 - ▣ Executar: ./asadmin start-domain --verbose domain1
- Estará disponível em: http://localhost:8080

A figura 1.3 apresenta o painel administrativo do GlassFish, que possui diversas configurações, visto ser uma implementação completa da especificação Java EE.



Figura 1.3
Painel Administrativo
do GlassFish.

Para iniciar o GlassFish em linha de comando, com o prompt ou shell, demos ir até o diretório de instalação, subdiretório bin, e executar o seguinte comando:

```
./asadmin start-domain --verbose domain1
```

O servidor estará disponível no navegador no endereço: <http://localhost:8080>. Ao ser requisitada, uma tela inicial será apresentada com um link para o Administration Console (o painel administrativo do GlassFish, como mostrado na figura 1.3. No menu “Aplicações”, podem ser visualizadas todas as aplicações instaladas ou implantadas que estão sendo servidas pelo domínio. Um botão “Implantar” é disponibilizado e consegue-se fazer a implantação (deploy) de uma nova aplicação por essa opção, mesmo não tendo acesso ao diretório de instalação do servidor.

Para implantação de uma nova aplicação, é necessário seu arquivo de empacotamento com extensão .war (de web ARchive). Para acessar a aplicação recém-instalada, usamos o endereço <http://localhost:8080/HelloWorld>, onde HelloWorld é o nome do arquivo .war instalado.

Outros servidores compatíveis com Java EE são:

- Apache Tomcat – Servlet Container
 - ▣ 8.0.x (Java 7) – Servlet 3.1 / JSP 2.3 / EL 3.0;
 - ▣ 7.0.x (Java 6) – Servlet 3.0 / JSP 2.2 / EL 2.2.
- Oracle Glassfish – Application Server (web e EJB)
 - ▣ 3.1.x – Java EE 6.
- RedHat Jboss
 - ▣ Enterprise 6.2 – Java EE 6.



Padrões de Projeto Front Controller e MVC

Front Controller é um padrão de projeto usado em aplicações web, usado para fornecer um ponto central de entrada para todas as requisições que são feitas para a aplicação.

Uma grande vantagem no seu uso é o controle centralizado de navegação, acesso, permissões, log etc., fazendo com que configurações externas ou mesmo a programação não fique diluída ao longo de muitas páginas, tornando a manutenção do software mais produtiva.

Para que o Front Controller seja realmente eficiente, é necessário que ele seja desenvolvido de forma enxuta, abstraindo somente as operações genéricas a todos (ou a maioria) dos elementos. Por exemplo: verificação se um usuário está logado, para acessar determinada página.

Muitos frameworks de mercado usam o padrão Front Controller em seu desenvolvimento e já disponibilizam esse componente de forma automática, bastando prover sua configuração. Por exemplo, o Struts provê o `ActionServlet`, e o JSF provê o `FacesServlet`.

O padrão de projeto MVC (Model-View-Controller) é usado para separar o desenvolvimento da aplicação em partes bem definidas, facilitando o desenvolvimento, a manutenção e extensão.

Modelo (ou model) se refere aos componentes do software que lidam com a representação de dados (ligação com banco de dados) e lógica de negócio. A Visão (ou view) faz a interface com o usuário. A Controladora (ou controller) é responsável por transformar as requisições do usuário, geradas na visão, para ações de lógica de negócio ou manipulação de dados, e também controlar a navegação entre visões, conforme o fluxo da aplicação.

Percebe-se que os padrões Front Controller e MVC se complementam no desenvolvimento de uma aplicação web, sendo que a camada de controle do modelo MVC pode ser implementada de várias formas, inclusive usando-se Front Controller.

Arquitetura JSF

O JSF encontra-se em permanente evolução. Entre os avanços mais recentes, podemos destacar:

- Integração com HTML5: nas versões anteriores, não se podia inserir atributos diferentes dos do HTML4, pois eram ignorados. Agora temos várias maneiras de propagar esses atributos para que eles sejam renderizados em HTML;
- FacesFlow: onde se pode agrupar várias páginas (views) em um fluxo (flow), com um determinado ponto de entrada e de saída;
- Componente de Upload;
- Várias correções de erros.

Para se usar o JSF é necessário ter uma implementação deste. Vários fabricantes têm suas próprias bibliotecas de JSF implementadas conforme a especificação.

Entre as mais importantes, temos:

- Sun/Oracle Mojarra 2.2.6;
- Apache MyFaces 2.2.2;
- Oracle – ADF Faces.

As ferramentas IDE mais comumente utilizadas (Netbeans e Eclipse) já possuem suporte ao JSF através de plug-ins. O Netbeans já traz nativamente essa integração, enquanto que no Eclipse deve-se instalá-lo. A documentação do Java EE e do JSF é muito ampla, mas está toda disponível na internet.

A figura 1.4 mostra a arquitetura de uma aplicação web usando Servlets/JSP. Nesse modelo, todas as requisições efetuadas pelo cliente são atendidas por componentes web, Servlets ou JSP, dependendo de como a aplicação foi desenvolvida. Esses componentes podem fazer uso de outros, como JavaBeans para representar dados, lógica de negócio etc.



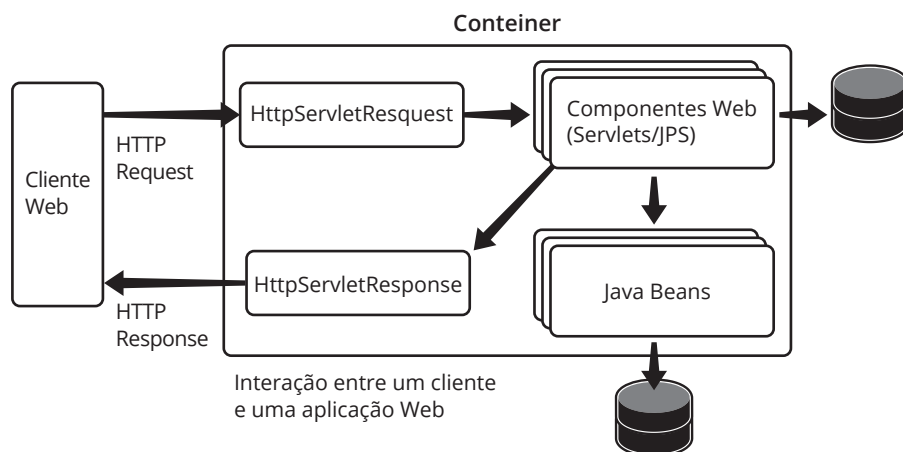


Figura 1.4
Arquitetura
Aplicação web:
jSP/Servlets.

Nesse modelo, todas as tarefas devem ser especificadas pelo programador dentro dos componentes. Por exemplo, os dados enviados por um formulário devem ser, um a um, obtidos do objeto de requisição (HttpServletRequest) e preenchidos em variáveis do componente ou atributos de Beans. A navegação entre telas também deve ser feita de forma manual, através de redirecionamentos (forward ou redirect).

A figura 1.5 apresenta a arquitetura de uma aplicação usando JSF. Nesse modelo o próprio framework já provê várias tarefas de forma automática, visto que vários componentes são automaticamente instalados. Um exemplo é o Faces Servlet, parte integrante do JSF e que faz o papel de controlador na arquitetura MVC.

Esse Servlet está programado para, de forma automática, receber o resultado de métodos do modelo como uma String e usar seu valor como nome de tela para aonde o sistema deve ser redirecionado. Isso é chamado de navegação implícita.

Outra tarefa automaticamente feita é o preenchimento dos dados provenientes de formulários diretamente em atributos de componentes (chamados Managed Beans). Esses dados são também, automaticamente, convertidos (exemplo, string para inteiro) e validados.

A figura 1.6 mostra a arquitetura JSF com mais detalhes, apresentando os componentes que interagem para a execução de uma aplicação.

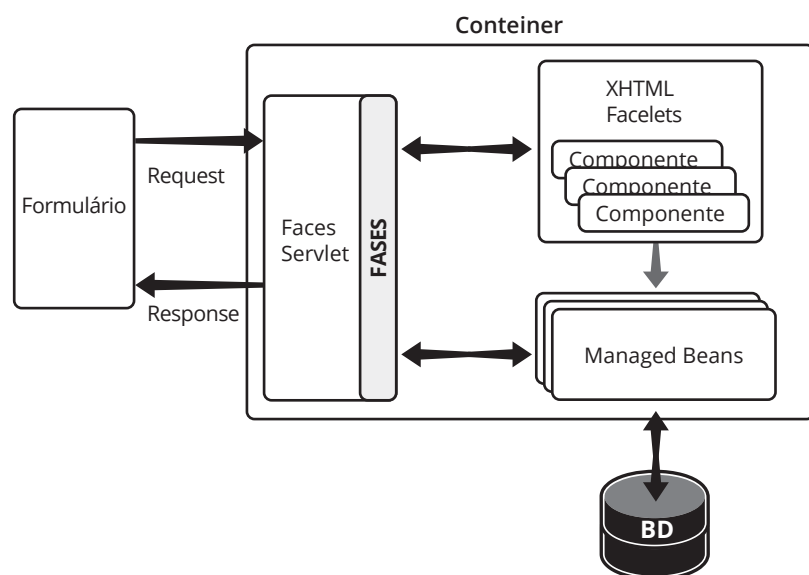


Figura 1.5
Arquitetura JSF
simplificada.

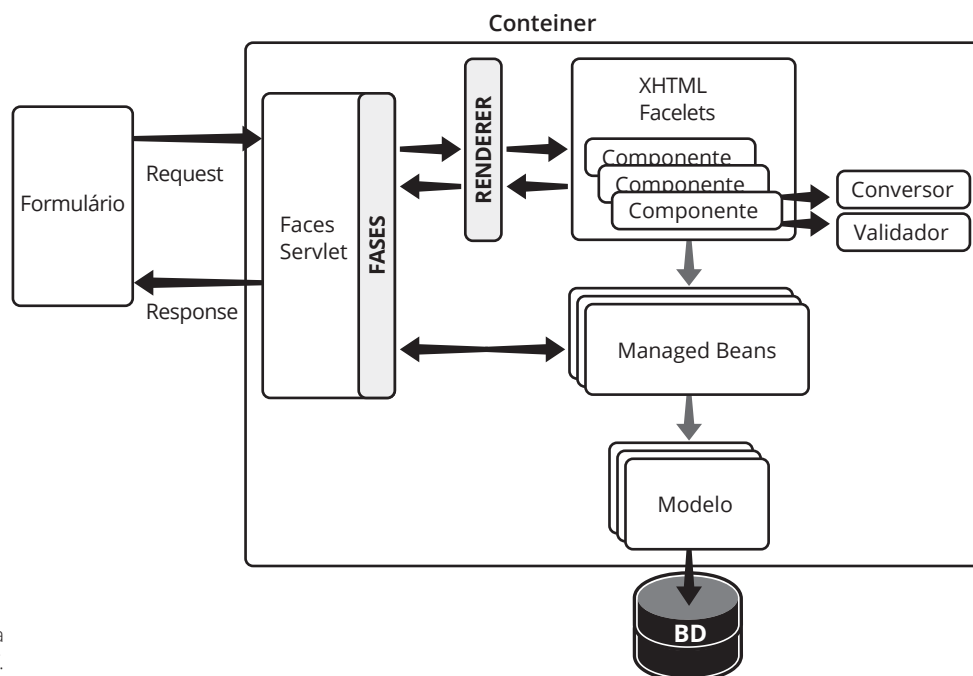


Figura 1.6
Arquitetura
Aplicação web – JSF.

Em relação às aplicações tradicionais usando Servlets/JSP, percebe-se que o JSP possui muito mais funcionalidades embutidas, que tornam o desenvolvimento mais eficiente, como por exemplo:

- **Recuperação de dados da requisição:** enquanto em uma aplicação tradicional deve-se efetuar várias chamadas a `request.getParameter()`, em JSF os dados são automaticamente atribuídos a atributos de um Managed Bean;
- **Conversão automática de dados da requisição:** a chamada a `request.getParameter()` sempre retorna uma String, que deve ser convertida para o tipo correto dentro da aplicação. Em JSF essa conversão é feita de forma automática;
- **Navegação implícita:** em uma aplicação tradicional deve-se efetuar forward ou redirect para se navegar entre JSPs e Servlets. Em JSF, usando-se navegação implícita. O retorno de um método do Managed Bean já indica para onde a aplicação deve ser redirecionada, sem que nenhum código seja necessário.

Aplicação Tradicional x JSF

- JSF possui muitas funcionalidades implícitas, por exemplo:
 - ▣ Recuperação de dados da requisição;
 - ▣ Conversão automática de dados da requisição;
 - ▣ Navegação implícita.

Os componentes arquiteturais de uma aplicação web usando JSF são:

- **FacesServlet:** servlet principal, que recebe todas as requisições do JSF. Padrão FrontController;
- **Renderer:** componente responsável por traduzir uma entrada de componente em representação interna e também por exibir um componente;
- **XHTML:** arquivo que descreve como as páginas serão renderizadas, usando Facelets;
- **Conversor:** componentes usados para converter elementos de String para Objeto (na requisição) e de Objeto para String (na resposta);

- **Validador:** componentes usados para validar se um componente possui um valor válido;
- **Managed Bean:** bean que executa a lógica do negócio e controla a navegação entre páginas;
- **Modelo:** beans de representação de dados. Exemplo: entidades mapeadas usando ferramentas para persistência de dados.

A figura 1.6 apresenta a estrutura de diretórios de uma aplicação JSF (encapsulada dentro de um arquivo .war).

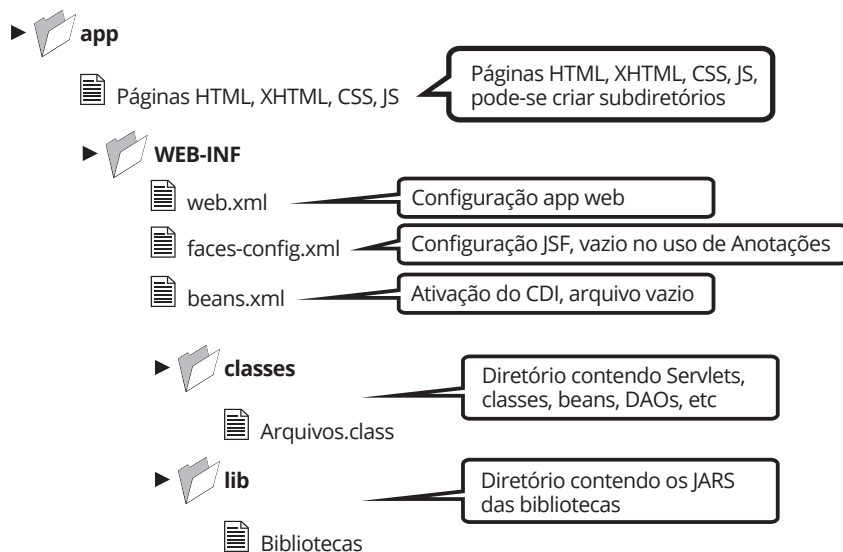


Figura 1.7
Estrutura de uma
Aplicação JSF.

Primeiro projeto

Para se iniciar um projeto usando o JSF, basta selecionar, no final da criação do projeto, o framework JavaServer Faces. Nesta sessão será criado um projeto simples, somente com a página default criada pelo Netbeans. Os passos são:

1. Iniciar o Netbeans;
2. Escolher Menu Arquivo | Novo Projeto;
3. Escolher Categoria "Java Web" e Projeto "Aplicação Web";
4. Pressionar Próximo;
5. Alterar o nome do projeto para "TesteJSF";
6. Pressionar Próximo;
7. Escolher o Servidor "GlassFish Server";
8. Pressionar Próximo;
9. Escolher o Framework "JavaServer Faces";
10. Pressionar Finalizar;
11. Executar o Projeto.

Comparação Servlets e JSF

A seguir, serão apresentadas duas implementações de um sistema simples, que recebe um número e um texto entrados pelo usuário para em seguida apresentar o quadrado do número e o texto em caixa-alta (letras maiúsculas). O objetivo é comparar as duas principais tecnologias usadas no desenvolvimento de sistemas: servlets/JSP e JSF.

Aplicação Servlets

Uma aplicação usando Servlets é formada por páginas em JSP (usando as bibliotecas de tags JSTL/EL), que interagem com o usuário, e Servlets para efetuar o processamento dessas páginas. Apesar de se poder fazer todo o processamento em páginas JSP, isso não é recomendado, pois com o passar do tempo o sistema pode crescer e a manutenção das páginas acaba se tornando inviável. Se o programador quiser usar MVC e Front Controller, ele deve implementar manualmente.

A seguir, o código index.jsp de um sistema simples.

```
<form action="Processar" method="post">
  <table>
    <tr>
      <td>Texto Processado:</td>
      <td><c:out value="${texto}" /></td>
    </tr>
    <tr>
      <td>Número Processado:</td>
      <td><c:out value="${numero}" /></td>
    </tr>
    <tr>
      <td>Texto:</td>
      <td><input type="text" name="texto" value="${texto}" />
        <c:if test="${not empty textoRequerido}">
          <span style="color: red">
            <c:out value="${textoRequerido}" />
          </span>
        </c:if>
      </td>
    </tr>
    <tr>
      <td>Número:</td>
      <td><input type="text" name="numero" value="${numero}" />
        <c:if test="${not empty numeroRequerido}">
          <span style="color: red">
            <c:out value="${numeroRequerido}" />
          </span>
        </c:if>
      </td>
    </tr>
  </table>
</form>
```

```

        <td><input type="submit" value="Processar" /></td>
    </td></td>
</tr>
</table>
</form>

```

A seguir o código Servlet Processar.java do sistema.

```

@WebServlet(name = "Processar", urlPatterns = {"/Processar"})
public class Processar extends HttpServlet {
    protected void processRequest(HttpServletRequest request,
                                   HttpServletResponse response)
        throws ServletException, IOException {
        String texto = request.getParameter("texto");
        String numero = request.getParameter("numero");

        if (texto == null || "".equals(texto.trim())) {
            request.setAttribute("textoRequerido",
                                "Texto é obrigatório");
            RequestDispatcher rd = getServletContext().
                getRequestDispatcher("/index.jsp");
            rd.forward(request, response);
        }
        if (numero == null || "".equals(numero.trim())) {
            request.setAttribute("numeroRequerido",
                                "Número é obrigatório");
            RequestDispatcher rd = getServletContext().
                getRequestDispatcher("/index.jsp");
            rd.forward(request, response);
        }
        int nr = 0;
        try {
            nr = Integer.parseInt(numero);
        }
        catch (NumberFormatException e) {
            request.setAttribute("numeroRequerido", "Número inválido");
            RequestDispatcher rd = getServletContext().
                getRequestDispatcher("/index.jsp");
            rd.forward(request, response);
        }
        ///// Efetivo processamento dos dados
        nr = nr * nr;
        texto = texto.toUpperCase();
        /////
        request.setAttribute("texto", texto);
        request.setAttribute("numero", String.valueOf(nr));
        RequestDispatcher rd = getServletContext().
            getRequestDispatcher("/index.jsp");
        rd.forward(request, response);
    }
}

```

```

@Override
protected void doGet(HttpServletRequest request,
                      HttpServletResponse response)
                      throws ServletException, IOException {
    processRequest(request, response);
}

@Override
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
                      throws ServletException, IOException {
    processRequest(request, response);
}

@Override
public String getServletInfo() {
    return "Short description";
}
}

```

Aplicação JSF

Uma aplicação usando JSF é formada por páginas XHTML que interagem com o usuário e Managed Beans para efetuar o processamento dessas páginas. Não é possível efetuar o processamento todo do sistema nas páginas XHTML, o que força o programador a usar MVC no código. MVC e Front Controller já estão implementados e são sempre usados, de forma transparente e sem aumentar o custo de codificação. A seguir, o código index.xhtml de um sistema simples.

```

<h:form>
    <h:panelGrid columns="2">
        <h:outputText value="Texto Processado:" />
        <h:outputText value="#{exemploMB.texto}" />
        <h:outputText value="Número Processado:" />
        <h:outputText value="#{exemploMB.numero}" />
        <h:outputLabel for="txtTexto" value="Texto:"/>
        <h:inputText id="txtTexto" value="#{exemploMB.texto}"
                    required="true" />

        <h:outputLabel for="txtNumero" value="Número:"/>
        <h:inputText id="txtNumero" value="#{exemploMB.numero}"
                    required="true" />

        <h:commandButton action="#{exemploMB.processar}"
                        value="Processar" />
    </h:panelGrid>
</h:form>

```

A seguir, o código ExemploMB.java, que é o Managed Bean usado para processar o XHTML.

```
@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
    private String texto;
    private int numero;
    public ExemploMB() {
    }
    public String getTexto() {
        return texto;
    }
    public void setTexto(String texto) {
        this.texto = texto;
    }
    public int getNumero() {
        return numero;
    }
    public void setNumero(int numero) {
        this.numero = numero;
    }
    public void processar() {
        this.texto = this.texto.toUpperCase();
        this.numero = this.numero * this.numero;
    }
}
```

Comparação

Fora a questão de quantidade de linhas de código, que claramente em JSF é menor, a complexidade do código desenvolvido pelo programador deve ser levada mais em conta.

A grande vantagem do código desenvolvido em JSF é que validações (se o número foi mesmo entrado), conversões (de String para int), mensagens de erro, atribuição do dado proveniente na tela para variáveis etc., são feitas de forma automática pelo framework, o que foca o esforço de desenvolvimento no que realmente é importante, que é o desenvolvimento do negócio.

Apesar de Servlets/JSP dar mais flexibilidade no desenvolvimento, o tempo de programação, risco de propagação de erros em tarefas básicas (como conversão e validação) e esforço de manutenção do sistema pode não valer a pena. Obviamente, deve-se conhecer o JSF para se obter benefícios no seu uso, pois mesmo guiando o programador, algumas boas práticas devem ser seguidas para que o resultado seja satisfatório.

2

Java Server Faces – Introdução

objetivos

Conhecer os elementos da arquitetura do JSF e o ciclo de vida de seus componentes.

Injeção de Dependência; Inversão de Controle; CDI; Managed Beans; POJO; Ações; Escopos; Requisições; Componentes; Navegação.

conceitos

Conforme visto na sessão anterior, JSF (Java Server Faces) é um dos principais frameworks disponíveis no mercado para desenvolvimento em Java para web. Atualmente, é parte integrante da especificação Java EE (a partir da versão 6) e, portanto, todos os servidores Java EE certificados o suportam.

JSF traz para o desenvolvedor as facilidades do desenvolvimento usando os padrões de projeto Front Controller e MVC (Model-View-Controller). Esses dois padrões são responsáveis por toda a facilidade e agilidade no desenvolvimento em JSF. Além disso, o suporte a várias operações, extensões e customizações tornam o JSF uma opção altamente recomendável para projetos web em Java.

O desenvolvimento baseado em componentes é outro benefício que se obtém do uso de JSF. Desenvolver usando um framework que suporta componentização e eventos favorece a manutenibilidade e extensibilidade do sistema. Em aplicações corporativas, onde o volume de funcionalidades em geral é grande e onde se faz necessário um ambiente que suporte agregação de outros requisitos, os frameworks têm um papel central na construção de sistemas flexíveis.

Outro ponto relevante é o fato de que muitas funcionalidades necessárias a sistemas corporativos, tais como persistência, transações e validações, já estão prontas, podendo ser usadas e personalizadas conforme a necessidade do cliente.

Injeção de Dependência e Inversão de Controle

O modelo de programação usado em aplicações Java EE evoluiu muito desde seu lançamento. Hoje em dia se faz muito pouco uso de arquivos XML para configuração de aplicações e descritores de implantação. O advento das anotações (como padrão no Java 5) trouxe uma grande flexibilidade no desenvolvimento, bem como uma clareza maior na configuração de elementos da aplicação.

Outro modelo de programação que trouxe grande agilidade no desenvolvimento foi a Injeção de Dependências, que pode ser descrito como: seja um componente, chamado A, que depende (possui como atributos, por exemplo) de B e C. Para que A funcione, é necessário ter uma instância de B e uma instância de C, conforme ilustra a figura 2.1.

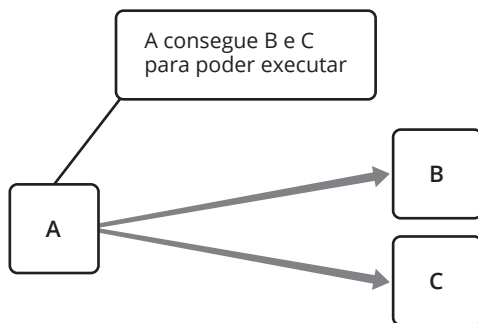


Figura 2.1
Dependências.

Um código possível para descrever esse problema pode ser visto a seguir:

```
public class ComponenteA {  
    private ComponenteB b;  
    private ComponenteC c;  
    ...  
    public void operacao() {  
        b = new ComponenteB();  
        c = new ComponenteC();  
        // efetua operação que depende de B e C  
        b.algumaCoisa();  
        c.outraCoisa();  
    }  
}
```

Percebe-se que a criação dos componentes B e C está sendo feita através da instanciação (new), mas não se tem garantia de que esta é a forma correta de obtê-los, nem se é uma forma suficiente. Muitos componentes, para serem criados, devem ser geridos pelo servidor ou devem vir previamente configurados, o que faz com que a instanciação não seja suficiente. É o caso de fontes de dados, gerenciadas pelos servidores de aplicação.

Além disso, além da dependência em si (componente A precisar de B e C para funcionar), fere-se um grave princípio que é o de baixo acoplamento, fazendo com que A saiba exatamente como criar (ou obter) B e C, com todas as suas particularidades.

Uma solução para esse problema é inverter o controle (IoC – Inversion of Control). Isto é, em vez da responsabilidade em encontrar os componentes B e C ser do componente que depende deles (no caso A), faz-se com que essas dependências sejam preenchidas de forma automática. Esse é o princípio de Hollywood: “Não nos ligue, nós ligamos para você!”. Essa resolução de dependências deve acontecer em tempo de execução e os componentes, agora, estão desacoplados.



Inversão de Controle – IoC:

- Princípio de Hollywood: “Não nos ligue, nós ligaremos.”;
- Envolve dependência de componentes;
- Ocorre em tempo de execução;
- Em vez de o objeto dependente conseguir as referências, encontrar um meio de preenchê-las de forma desacoplada;
- Componente não precisa saber detalhes de como criar as dependências.

A inversão de controle pode acontecer de várias formas, conforme veremos a seguir.

A Procura por Dependência, onde o componente implementa uma maneira genérica de se obter as dependências, pode ser de dois tipos: Dependency Pull (DP) e Contextualized Dependency Lookup (CDL).

Dependency Pull (DP)

Onde as dependências são obtidas através de um registro central, como um repositório no servidor, por exemplo:

```
public class Teste {  
    ...  
    public void testar() {  
        Registry registro = getRegistry();  
        InterfaceDependencia I =  
            registro.getInstance("dependencia");  
    }  
}
```

Contextualized Dependency Lookup (CDL)

O componente implementa uma interface específica, indicando ao contêiner que uma dependência deve ser obtida através da chamada de um método específico (da interface) por exemplo:

```
public interface ManagedComponent {  
    public void performLookup(Container container);  
}  
public class Teste implements ManagedComponent {  
    private Dependencia dep;  
    public void performLookup(Container container) {  
        this.dep = (Dependencia)container.  
            getDependency("minhaDependencia");  
    }  
}
```

Já a Injeção de Dependências, onde o componente não se preocupa com as dependências, essas precisam ser injetadas de alguma forma. Se feita de forma correta, conseguem-se módulos plugáveis e extensíveis na aplicação. No exemplo a seguir os componentes “b” e “c” são usados sem se preocupar com sua criação ou obtenção.



```

public class ComponenteA {
    private ComponenteB b;
    private ComponenteC c;
    ...
    public void operacao() {
        // efetua operação que depende de B e C
        // B e C já existem
        b.algumaCoisa();
        c.outraCoisa();
    }
}

```

Três tipos de Injeção de Dependência (Dependency Injection – DI):

- **Via Construtor:** passa-se os recursos como parâmetro no construtor da classe que necessita deles (são injetadas pelo contêiner);
- **Via atributo:** através do método set de algum atributo da classe, assim o container pode injetar as dependências;
- **Via Interface:** o componente implementa uma interface indicando que o contêiner deve injetar uma dependência.

Até a versão Java EE 5 era possível, de forma automática, se fazer injeção de recursos em uma aplicação: eJBs, DataSources, Persistence Units etc. A partir da versão Java EE 6, foi introduzido o CDI (Contexts and Dependency Injection), que é uma especificação para controle de injeção de dependências de forma geral (não só de recursos), de forma padronizada. O CDI é definido pela JSR 299.

CDI – Injeção de Dependência e Contextos:

- **Contextos:** a capacidade de ligar um componente a um ciclo de vida, bem como a comportamentos de estado nesse ciclo de vida;
- **Injeção de Dependências:** capacidade de injetar instâncias de componentes, tipadas, de forma padronizada.

O CDI trata injeção de dependências de forma geral, onde qualquer componente pode ser injetado. O uso da anotação @Inject facilita a legibilidade do código e a sua flexibilidade, deixando decisões de o que deve ser injetado para o contêiner.

Para que o CDI seja habilitado em seu projeto, deve-se criar o arquivo vazio beans.xml. Só a presença desse arquivo já indicia ao contêiner que o CDI foi ativado e todas as suas facilidades estarão disponíveis.

Java EE 5:

- Anotações que injetavam Recursos;
- Recursos: EJBs, Datasources, JMSs;
- @EJB, @Resource, @PersistenceUnit;

Java EE 6+;

- Com CDI;
- @Inject consegue injetar qualquer componente.

XHTML e Managed Beans

Os Beans Gerenciados (do inglês: Managed Beans: MB) são componentes fundamentais dentro da arquitetura do JSF. São eles que fornecem dados que serão exibidos nas páginas, recebem os dados enviados em requisições e executam tarefas referentes a ações do usuário.

Os MBs são classes Java que não herdam nem implementam nenhum tipo de biblioteca, portanto conhecidas como POJO (Plain-Old Java Objects). Para definir uma classe como um MB, deve-se anotá-la com `@Named` (pacote `javax.inject.Named`), indicando que estará disponível no XHTML através de expressões EL (Expression Language).

O ciclo de vida desses objetos é controlado pelo Servidor/Framework e esse é mais um aspectos que torna atrativo o uso do JSF, pois não deixa a responsabilidade de criação e remoção dos objetos para o programador.

XHTML
eXtensible Hypertext
Markup Language, é
uma reformulação
da linguagem de
marcação HTML,
baseada em XML.
Combina as tags de
marcação HTML com
regras da XML.....

```
import javax.inject.Named;
@Named
@RequestScoped
public class ExemploBean {
    private String texto;
    public String getTexto() {
        return texto;
    }
    public void setTexto(String texto) {
        this.texto = texto;
    }
}
```

Para disponibilizar os dados de um MB na tela e também para interagir com o usuário, escreve-se um arquivo **XHTML**. Esse arquivo possui tags HTML e também tags do JSF. Tudo que estiver ligado ao JSF será processado dentro do ciclo de vida da requisição.

Esse processo de padronização do XHTML provê a exibição de uma página web nesse formato por diversos dispositivos (televisão, palm, celular etc.), além da melhoria da acessibilidade do conteúdo. A principal diferença entre XHTML e HTML é que o primeiro é XML válido, enquanto o segundo possui sintaxe própria. Ambos possuem sentido semântico.

A seguir temos o XHTML usado para apresentar os dados do MB anterior.

```
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://java.sun.com/jsf/html">
  <h:head><title>Exemplo</title></h:head>
  <h:body>
    Texto: #{exemploBean.texto}
    <h:form>
      <h:inputText value="#{exemploBean.texto}" />
      <h:commandButton value="Alterar" />
    </h:form>
  </h:body>
</html>
```

O ciclo de vida do processamento de uma requisição nesse exemplo é mostrado na figura 2.2. Nesse caso, depois que o formulário é mostrado para o usuário, ele pode preencher os dados e pressionar o botão “Alterar”. Quando essa requisição chega ao servidor, uma representação interna da página XHTML (árvore de componentes) é obtida. Uma instância do MB também é obtida (criada ou recuperada, dependendo das especificações de contextos). Os dados que são enviados na requisição (campos do formulário) são obtidos e preenchidos no MB. Em seguida, como está sendo mostrada a mesma página, os campos na representação são preenchidos e enviados para o cliente (navegador).

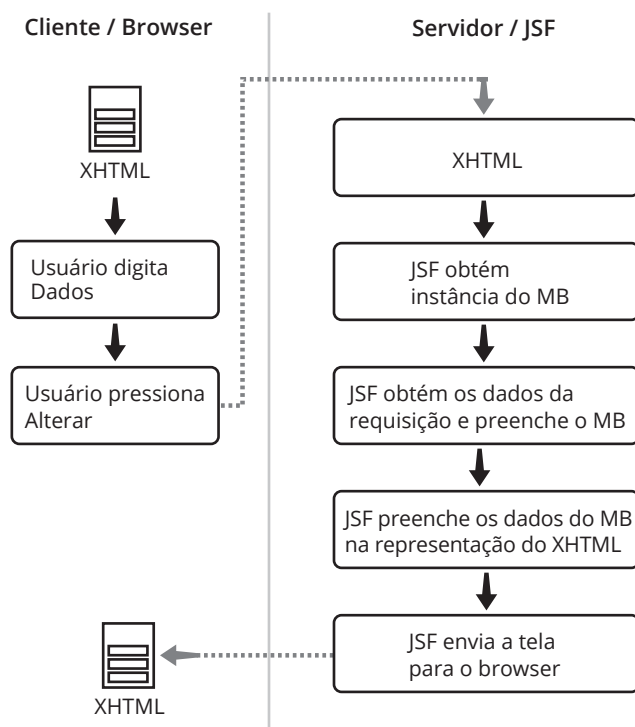


Figura 2.2
Ciclo de Vida Simplificado.

Como exercício, podemos criar uma nova aplicação de nome ExemploJSF1 (não esquecer de adicionar o Framework JavaServer Faces no projeto). Os passos para a criação do projeto são:

- Abrir o Netbeans;
- Menu ARQUIVO | NOVO PROJETO;
- Escolher Categoria JAVA WEB;
- Escolher Projetos APLICAÇÃO WEB;
- Clicar PRÓXIMO;
- Alterar o NOME DO PROJETO;
- Clicar PRÓXIMO;
- Escolher o SERVIDOR;
- Clicar PRÓXIMO;
- Escolher o Framework JAVASERVER FACES;
- Clicar em “FINALIZAR”.

Para criar um Managed Bean, deve-se:

- Clicar com o botão direito do mouse sobre o Projeto;
- Escolher Menu NOVO | BEAN GERENCIADOJSF;
- Escolher o NOME DA CLASSE;
- Escolher o NOME DO PACOTE;
- Clicar em “FINALIZAR”.

Neste exercício, deve-se criar um Managed Bean (Bean Gerenciado JSF) de nome ExemploBean, com o código a seguir.

```
import javax.inject.Named;
@Named
@RequestScoped
public class ExemploBean {
    private String texto;
    public String getTexto() {
        return texto;
    }
    public void setTexto(String texto) {
        this.texto = texto;
    }
}
```

A seguir, tem-se o XHTML usado para apresentar os dados do MB anterior e esse código deve ser digitado no arquivo index.xhtml.

```
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://java.sun.com/jsf/html">
  <h:head><title>Exemplo</title></h:head>
  <h:body>
    Texto: #{exemploBean.texto}
    <h:form>
      <h:inputText value="#{exemploBean.texto}" />
      <h:commandButton value="Alterar" />
    </h:form>
  </h:body>
</html>
```

Após, clicamos em “EXECUTAR” e testamos a aplicação.

Ações

No exemplo anterior, nenhum tipo de ação era feita quando o cliente pressionava o botão “Alterar”. Portanto, os dados eram submetidos, preenchidos no MB e mostrados na tela novamente, sem qualquer tipo de processamento.

Para que um método do MB seja invocado quando os dados são submetidos, usamos o atributo action da tag <h:commandButton />. Os métodos invocados podem retornar void ou String, que definem:

- **Retorno void:** indica que a próxima página a ser renderizada é a mesma que originou a requisição;

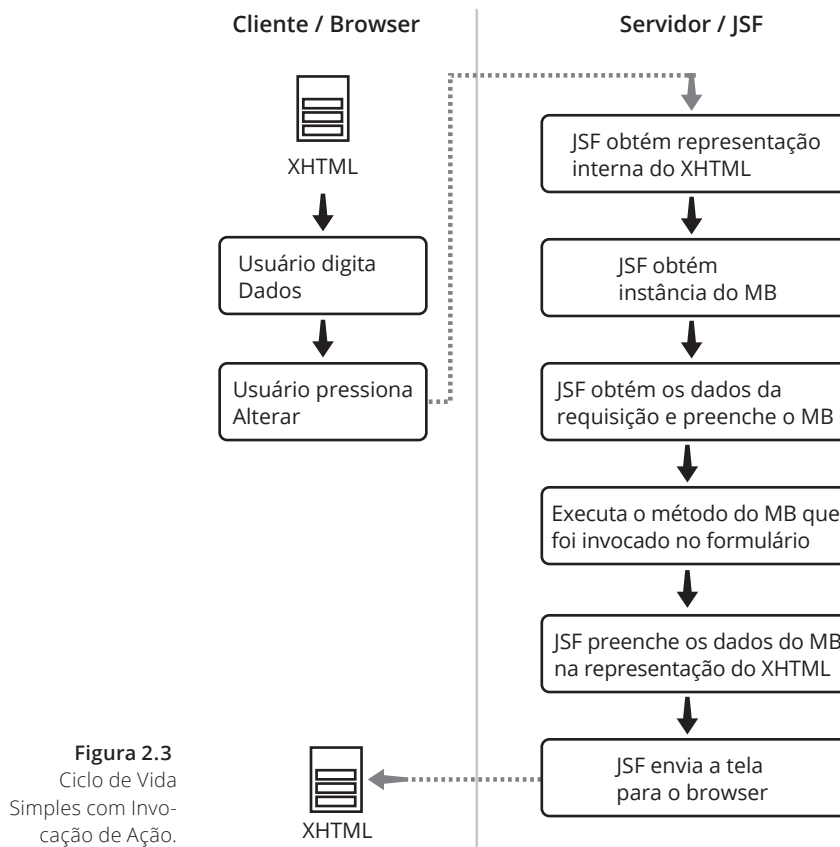
- **Retorno String:** o valor da String (concatenado com.xhtml) é o nome da próxima página a ser renderizada.

No exemplo a seguir, o MB possui um método chamado caixaAlta(), que retorna void, indicando que a próxima página a ser mostrada é a mesma que originou a requisição.

```
import javax.inject.Named;
@Named
@RequestScoped
public class ExemploBean {
    private String texto;
    public String getTexto() {
        return texto;
    }
    public void setTexto(String texto) {
        this.texto = texto;
    }
    // setter/getter de texto
    public void caixaAlta() {
        this.texto = this.texto.toUpperCase();
    }
}
```

O XHTML necessário para invocar o método caixaAlta() do MB está mostrado no código a seguir. Perceba que o atributo action contém o nome do método a ser invocado. Esse novo ciclo de vida pode ser visto na figura 2.3. Antes de a ação ser chamada, todos os dados do MB provenientes da requisição (digitados na tela pelo cliente) são preenchidos e a próxima tela só é processada após a execução da ação.

```
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://java.sun.com/jsf/html">
  <h:head><title>Exemplo</title></h:head>
  <h:body>
    Texto: #{exemploBean.texto}
    <h:form>
      <h:inputText value="#{exemploBean.texto}" />
      <h:commandButton value="Alterar"
        action="#{exemploBean.caixaAlta}" />
    </h:form>
  </h:body>
</html>
```



Escopos

Escopos têm a ver com o ciclo de vida dos Managed Beans, isto é, quando o objeto é criado e quando é descartado. Isso implica na durabilidade dos dados que ele mantém.

Escopos são usados para indicar por quanto tempo um Managed Bean se manterá criado e, portanto, por quanto tempo manterá seus dados ativos. O JSF e o CDI possuem mecanismos diferentes para manter o ciclo de vida dos MBs, e o programador só precisa indicar qual é esse tempo usando uma anotação específica. Não há necessidade de criação nem mesmo remoção dos MBs.

Os escopos do JSF são usados somente se o programador estiver em um ambiente puramente JSF sem o suporte a CDI. Mas como comumente se usam Servidores de Aplicação Java EE 6 ou superior, que já possuem suporte nativo a JSF e CDI, deve-se usar os escopos do próprio CDI.

No CDI, a partir da versão 2.2, também está disponível um escopo do JSF muito útil para confecção de Managed Beans que precisam estar disponíveis enquanto o usuário se mantiver em uma determinada página.

Escopos do CDI – importados de `javax.enterprise.context`:

- **Dependent:** `@DependentScoped`;
- **Request:** `@RequestScoped`;
- **Session:** `@SessionScoped`;
- **Application:** `@ApplicationScoped`;
- **Conversacional:** `@ConversacionalScoped`;



Escopo do JSF: importados de `javax.faces.view.ViewScoped`;

■ **View:** `@ViewScoped`.

A seguir, vamos detalhar cada um desses escopos.

@DependentScoped

É o escopo default do CDI. Quando um MB é marcado com esse escopo, ele assume o escopo do objeto onde ele está sendo injetado. Por exemplo, se foi injetado em um Componente que possui escopo de sessão, então o MB também possuirá escopo da sessão.

Para aplicações puramente JSF/CDI, esse escopo é pouquíssimo usado e, em geral, será necessário trocar o escopo do MB para o de requisição. O código a seguir mostra um trecho de código de um MB com escopo dependente.

```
import javax.enterprise.context.DependentScoped;
@Named
@DependentScoped
public class TesteMB {
}
```

@RequestScoped

Quando o MB é marcado com esse escopo, ele é criado a cada nova requisição, e fica ativo até que a requisição termine. Quando a requisição termina, todos os dados são descartados, isto é, nada é mantido para a requisição seguinte.

O código a seguir mostra um trecho de código de um MB com escopo de requisição.

```
import javax.enterprise.context.RequestScoped;
@Named
@RequestScoped
public class TesteMB {
}
```

@SessionScoped

O MB que possui escopo da sessão é mantido na sessão do usuário e, portanto, só é descartado quando o usuário faz logout, deixa de fazer requisições por determinado tempo ou o servidor/aplicação são parados. O MB deve implementar a interface `Serializable`, pois servidores podem armazenar dados de sessão em arquivos. O código a seguir mostra um exemplo de MB com escopo da sessão.

```
import javax.enterprise.context.SessionScoped;
@Named
@SessionScoped
public class TesteMB implements Serializable {
}
```

Um ponto importante sobre o escopo da sessão é que na maioria das vezes não se deve deixar MBs na sessão. A sessão possui um tamanho limitado e seu uso indiscriminado pode prejudicar a escalabilidade e desempenho da aplicação.



@ApplicationScoped

Quando um MB é assinalado como possuidor do escopo da aplicação, uma, e somente uma instância é criada para a aplicação toda, e é criada assim que o MB for usado pela primeira vez (instanciação conhecida como lazy).

Essa instância é mantida até a aplicação ser finalizada e é compartilhada por todos os usuários da aplicação, isto é, sempre que requisitado o mesmo bean é retornado. Deve implementar Serializable. O quadro mostra um exemplo de MB com escopo da aplicação.

```
import javax.enterprise.context.ApplicationScoped;
@Named
@ApplicationScoped
public class TesteMB implements Serializable {
}
```

@ViewScoped

O escopo de visão existe como um escopo do JSF, adicionado no JSF 2, mas não existe como escopo do CDI. No JSF 2.2, foi introduzida uma maneira de se usá-lo no CDI para que as aplicações possam ser desenvolvidas com esta funcionalidade.

Para usá-lo, o Managed Bean deve importar o ViewScoped do JSF, que é a seguinte classe:

```
import javax.faces.view.ViewScoped;
```

Nesse escopo o MB existe enquanto o usuário não sair da página, lembrando que métodos de ação do MB que retornam void permanecem na mesma página ou visão. Quando o usuário muda de tela, o MB é descartado.

Assim, MBs anotados com esse escopo não compartilham dados entre telas ou abas diferentes na mesma sessão; cada visão é uma instância diferente do MB. O MB deve implementar Serializable. O código a seguir mostra um exemplo de MB com escopo de visão.

```
import javax.faces.view.ViewScoped;
@Named
@ViewScoped
public class TesteMB implements Serializable {
}
```

@ConversationScoped

O escopo de conversação é maior que uma requisição e menor que uma sessão. O MB existe com um início e fim determinados programaticamente, através da chamada de métodos específicos determinados pelo programador.

Podemos fazer várias chamadas ao bean (exemplo: aAJAX) e os dados serão os mesmos, e o programador deve fechar programaticamente a conversação.

Para funcionar, deve-se injetar um objeto do tipo Conversation. O início do escopo se dá pela chamada do método begin(), e o término do escopo se dá pela chamada do método end(). O MB deve implementar Serializable. O código a seguir mostra um exemplo de escopo de conversação.

```
import javax.enterprise.context.ConversationScoped;
@Named
@ConversationScoped
public class TesteMB implements Serializable {
    @Inject
    private Conversation conversation;
    public void initConversation() {
        conversation.begin();
    }
    public String endConversation() {
        conversation.end();
    }
    public Conversation getConversation() {
        return this.conversation;
    }
}
```

@TransactionScoped

O escopo de transação foi criado no Java EE 7 e é usado quando um MB deve ter a mesma duração que uma transação do JTA (Java Transaction API).

@FlowScoped

O escopo de fluxo foi criado no Java EE 7 e é um escopo maior que uma requisição e menor que uma sessão. Dentro de uma aplicação, podemos definir vários fluxos com as seguintes características:

- Cada fluxo possui vários nodos, entre visões (XHTML), chamadas de função etc.;
- Um fluxo pode chamar outros fluxos;
- Possui um ponto de entrada e vários pontos de saída;
- Um MB nesse escopo existe enquanto o fluxo estiver ativo.



Esse escopo é ideal para representar wizards (conjunto de telas que ajudam o usuário a executar alguma tarefa, um assistente) e fluxos de operação. Um detalhe importante é que o MB deve ser serializável (implementar Serializable).

Processamento de uma Requisição

Quando uma requisição é feita, é recebida pelo FacesServlet. Esse é responsável pelo processamento completo dessa requisição, desde seu recebimento até que a nova tela possa ser exibida. Esse processamento passa por diversas fases e cada uma dessas fases tem um objetivo específico. Conhecer esses passos é essencial para o desenvolvedor que usa JSF.

A figura 2.4 mostra um diagrama das fases e seu sequenciamento, executadas pelo FacesServlet. A seguir, serão detalhadas todas as fases e o processamento realizado em cada uma delas.

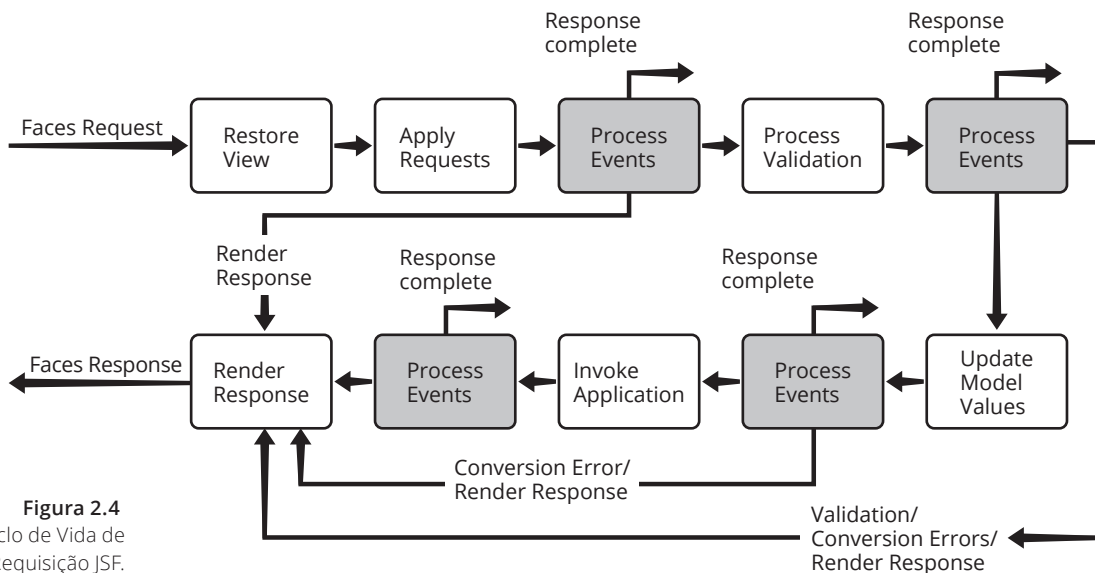


Figura 2.4
Ciclo de Vida de
uma Requisição JSF.

Process Events

Os vários passos chamados Process Events na figura são pontos em que o JSF pode desviar o fluxo da aplicação, para não seguir o processamento normal. Isso se dá por conta de erros (exemplo: validação de dados), tipo de requisição (exemplo: primeira requisição em vez de postback) e por conta de chamada voluntária do programador. Nesse caso, em uma chamada a `FacesContext.responseComplete()`.

Quando a aplicação precisa redirecionar para uma página externa ou uma página que não esteja sendo tratada pelo JSF, precisa-se indicar que o JSF não deve renderizar a saída conforme fluxo normal da requisição. Isso é feito através da chamada a `responseComplete()`.

Restore View

Essa é a primeira fase do processamento de uma requisição em JSF. Duas situações podem ocorrer: ser a primeira requisição à página ou ser outra requisição (chamado de postback).

No caso de uma primeira requisição, a árvore de componentes que representa a tela é criada e armazenada no `FacesContext`, com dados vazios, e o fluxo é redirecionado para a fase de `Render Response`, responsável por enviá-la para o cliente.

No caso de postback, uma representação da árvore de componentes já existe, e é somente recuperada e reconstruída. Conversores, Validadores ou Renderizadores customizados, quando anexados a componentes, são também restaurados. Após isso, a próxima fase é invocada.

Em qualquer um dos casos, a árvore de componentes criada precisa ser armazenada para posterior recuperação. Existem duas técnicas, que pode ser configurada no arquivo `web.xml`:

- **Cliente:** o estado da tela (`ViewState`) é armazenado em um campo hidden na tela (HTML);
- **Servidor:** o estado da tela é armazenado na sessão do usuário.

Em qualquer caso, faz-se necessário armazenar esse estado para se conseguir o comportamento stateful do JSF, visto que HTML, originalmente, não armazena estado de nenhum componente (stateless). Outro ponto importante é a robustez do JSF. Se o estado da página não é armazenado, não há mecanismo de verificação indicando se alguma requisição HTTP foi interceptada e seus atributos alterados (como por exemplo, campos desabilitados, somente de leitura etc.).

O código a seguir apresenta um trecho exemplo de página XHTML, e a figura 2.5, sua representação como uma árvore de componentes.

```
<h:form>
  <h:inputText value="#{mb.valor}" id="valor" />
  <h:inputText value="#{mb.texto}" id="texto" />
  <h:commandButton value="Enviar" action="#{mb.acao}" />
</h:form>
```

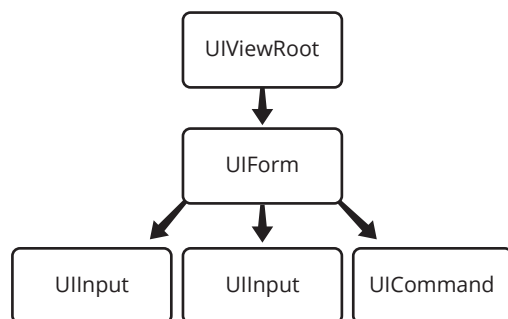


Figura 2.5
Árvore de
Componentes.

Apply Request Values

Nesta fase, com a árvore de componentes recuperada (ou criada), todos os componentes são varridos e seus dados decodificados (obtidos da requisição). Os valores obtidos são armazenados localmente, no próprio componente.

Se foram enviadas ações, como cliques de links ou botões, essas são identificadas e enfileiradas para serem tratadas na fase Invoke Application.

Process Validations

Nesta fase, os dados já decodificados na fase anterior são convertidos (conversões padrão ou conversões registradas) e validados (validadores registrados). Se algum erro ocorrer, as mensagens pertinentes são colocadas no contexto da requisição (FacesContext) e o fluxo é redirecionado para Render Response, para que a mesma página (com os erros) seja enviada e mostrada ao cliente.

Caso nenhum erro seja encontrado, o processo continua para a próxima fase.

Update Model Values

Nesta fase, todos os dados decodificados, convertidos e validados são armazenados em propriedades do MB. Essa fase só ocorre depois da validação, o que garante que seus dados estão validados, em nível de tela.

Invoke Application

Nesta fase, as ações correspondentes ao componente que disparou a requisição (clique de um botão ou link) são executadas. Podemos ter várias ações para serem executadas, e elas são executadas na seguinte ordem:

1. Método associado com o atributo `actionListener`;
2. Métodos associados com as tags `<f:actionListener>`, na ordem em que aparecem no XHTML;
3. Método associado com o atributo `action`.

O retorno do método associado ao atributo `action` determina qual a próxima página a ser renderizada. Se o retorno do método for `void` ou `null`, então permanece na mesma página. Se for uma `String`, esta é o nome da página (acrescida de `.xhtml`) que deverá ser renderizada.

Render Response

Fase na qual a próxima tela é renderizada e enviada ao cliente. Se a aplicação se mantiver na mesma página, sua representação já é conhecida, seu novo estado é armazenado e cada componente é convertido para HTML, para que possa ser enviado ao cliente.

Se uma nova página precisar ser gerada, uma nova árvore de componentes é criada e esta representação é armazenada, para que na próxima requisição possa ser recuperada.

Se houver alguma mensagem de erro no contexto da requisição e se houver tags `<h:message>` ou `<h:messages>`, essas mensagens também são renderizadas no HTML resultante para o cliente.

Ciclo de vida simplificado

Os ciclos de vida simplificados apresentados anteriormente são uma maneira de entender, de forma construtiva, o ciclo de vida de uma aplicação JSF. Mesmo que o formulário (`<h:commandButton>`, por exemplo) não contenha a indicação de execução de um método do MB, a fase `Invoke Application` ainda será chamada, visto que é nessa fase que o JSF descobre qual será a próxima tela a ser renderizada para o usuário.

Navegação

A navegação entre telas de uma aplicação JSF é um mecanismo sofisticado, mas simples de implementar e entender. Tudo é baseado em um sinal enviado para o JSF, chamado `outcome`. Por exemplo, se uma página possui um link para uma segunda página, envia-se o `outcome` “segunda” para o JSF e o usuário será redirecionado para a página `segunda.xhtml`. Assim, o `outcome` nada mais é do que o nome da página sem a extensão `.xhtml`.

Os componentes visuais que fazem transição entre páginas são os componentes de ação `<h:commandButton>` e `<h:commandLink>`, e os componentes de redirecionamento `<h:button>` e `<h:link>`. Para se definir o `outcome` nos primeiros (`<h:commandButton>` e `<h:commandLink>`), usa-se o atributo `action`. Nos segundos (`<h:button>` e `<h:link>`), o atributo é `outcome`.

Componentes visuais:

- De ação: `<h:commandButton>`, `<h:commandLink>`
 - ▣ `Outcome` definido no atributo `action`.
- De redirecionamento: `<h:button>`, `<h:link>`
 - ▣ `Outcome` definido no atributo `outcome`.
- Por exemplo:

```
<h:form>
  <h:commandButton value="Página Teste"
    action="teste"/>
</h:form>
```



Ao ser clicado o componente <h:commandButton>, o usuário é levado para a página teste.xhtml.

Se um outcome inicia com "/", então o caminho da página destino inicia na raiz da aplicação, caso contrário será relativo ao diretório da página atual.

O retorno de métodos de um Managed Bean, como mostrado anteriormente, também usa o mesmo princípio, retornando um outcome. As regras de navegação podem ser definidas no arquivo de configuração faces-config.xml, mas geralmente usa-se a navegação implícita.

A navegação implícita nada mais é do que o retorno do método do MB ser considerado o nome da página, sem o.xhtml. Se iniciar com "/", será usado o caminho absoluto da raiz da aplicação, se não, será o caminho relativo à página atual. Se o método retornar um outcome contendo "faces-redirect=true", será feito um redirect em vez de um forward.

Redirect x Forward:

- Por default, o JSF efetua forward para redirecionamento;
- Para garantir o uso de redirect, deve-se indicar no outcome com:
"faces-redirect=true".
- Por exemplo:

```
import javax.inject.Named;
@Named
@RequestScoped
public class ExemploBean {
    public String testar() {
        //...
        return "segundaPagina?faces-redirect=true"
    }
}
```



3

JSF – Componentes visuais

objetivos

Conhecer os componentes visuais para confecção de telas em JSF, usando formulários e manipulação de dados para apresentação.

Componentes visuais; Propriedade xmlns; Namespaces; Binding; @PostConstruct; @PreDestroy; Opções Estáticas x Dinâmicas e seleção única x múltipla.

conceitos

Estrutura básica

As telas de uma aplicação JSF são definidas em arquivos XML. Tags bem definidas nesses arquivos fazem com que se possa descrever telas usando HTML e tags do próprio JSF.

O JSF possui um tratador de telas (visões) chamado de VDL (View Declaration Language), que até antes da versão JSF 2.0 era implementado com o JSP. Após a versão 2.0, foi introduzido o Facelets, que é um tratador otimizado de visões, sem os problemas de incompatibilidade que eram comuns ao JSP.

Para usar os componentes do JSF, devemos incluir as bibliotecas de tags nos arquivos XHTML.

As mais importantes são:

- **Core:** representado pela URI <http://xmlns.jcp.org/jsf/core>
- **HTML:** representado pela URI <http://xmlns.jcp.org/jsf/html>
- **Facelets:** representado pela URI <http://xmlns.jcp.org/jsf/facelets>

Estas tags são incluídas como atributos de namespace (espaço de nomes, para que não haja ambiguidade) na tag HTML, conforme o exemplo a seguir:

```
<?xml version='1.0' encoding='UTF-8' ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd" >
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:ui="http://java.sun.com/jsf/facelets"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">
```



```
<h:head>
  <title>JSF</title>
</h:head>
<h:body>
  <h:outputText value="Estrutura básica de uma tela JSF" />
</h:body>
</html>
```

Nesse exemplo, os atributos xmlns: representam:

- **xmlns:ui="http://java.sun.com/jsf/facelets"**: indica que todas as tags com prefixo "ui" serão tags do facelets;
- **xmlns:h="http://java.sun.com/jsf/html"**: indica que todas as tags com prefixo "h" serão tags html;
- **xmlns:f="http://java.sun.com/jsf/core"**: indica que todas as tags com prefixo "f" serão tags do núcleo jsf (core).

As tags são prefixadas, como no exemplo:

```
<h:outputText value="Estrutura ..." />
```

Onde <h:outputText indica que a tag outputText é da biblioteca h (html), representada pela URI "http://java.sun.com/jsf/html".

O exemplo anterior também apresenta a estrutura básica de uma visão (tela) XHTML para o JSF. Os seguintes elementos são obrigatórios:

- DOCTYPE;
- HTML.

A página é delimitada pela tag <html>.

- As bibliotecas são importadas pela propriedade xmlns;
- xmlns define os namespaces;
- Namespaces são usados para evitar conflitos de nomes nos elementos;
- Define prefixos para serem usados nos elementos.

Dentro de <html> temos:

- H:HEAD
- H:BODY

O uso das tags h:head e h:body é recomendado para que o JSF possa renderizar scripts ou recursos necessários na fase Render Response, para que a página funcione adequadamente.

Formulários

Formulários são definidos pela tag h:form. Os formulários em JSF são renderizados como formulários HTML que enviam dados via POST, independente se for definido um botão ou um link de submissão. Scripts inseridos na página pelo próprio JSF se encarregam de ajustar o comportamento.

Dentro do formulário, devem ser colocados componentes de entrada de dados, cujos valores inseridos serão enviados para o JSF ao ser efetuado um submit.





Entre os elementos possíveis destacamos:

- Caixas de Texto e Rótulos;
- Campos Ocultos;
- Caixas de Seleção;
- Botões e Links;
- Textos e Imagens;
- Componentes de Organização;
- Tabelas;
- Mensagens;
- JavaScript e CSS;
- Repetição.

A seguir, é mostrado um código XHTML para apresentar um formulário, esse mesmo formulário em HTML e, finalmente, como o formulário é apresentado no navegador (figura 3.1).

Código em JSF

```
<?xml version='1.0' encoding='UTF-8' ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/
TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://xmlns.jcp.org/jsf/html"
      xmlns:ui="http://xmlns.jcp.org/jsf/facelets">
  <h:head>
    <title>Exemplo</title>
  </h:head>
  <h:body>
    <h:form>
      <h:outputLabel value="Nome: " for="nome" />
      <h:inputText value="#{exemploMB.nome}" id="nome" />
      <h:commandButton value="Enviar" />
    </h:form>
  </h:body>
</html>
```



Renderizado em HTML

```
<?xml version='1.0' encoding='UTF-8' ?>
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"> <head id="j_idt2"><title>Exemplo</
title></head> <body>
<form id="j_idt5" name="j_idt5" method="post" action="/CaixasTexto/faces/index.
xhtml" enctype="application/x-www-form-urlencoded">
<input type="hidden" name="j_idt5" value="j_idt5" />
<label for="j_idt5:nome">Nome: </label>
<input id="j_idt5:nome" type="text" name="j_idt5:nome" />
<input type="submit" name="j_idt5:j_idt7" value="Enviar" />
<input type="hidden" name="javax.faces.ViewState" id="j_id1:javax.faces.
ViewState:0" value="-8689468098154464721:-6542804232569543406" autocomplete="off" />
</form> </body></html>
```



Figura 3.1
Formulário em JSF.

Binding e processamento

Quando se escreve um componente visual em uma página XHTML do JSF, em geral é necessário ligá-lo a um componente ou atributo no MB. Esse comportamento é chamado de binding ou ligação.

Podemos fazer dois tipos de ligação:

- **Valor:** faz-se com que o componente seja preenchido com um atributo do MB e, quando o formulário for submetido, faz com que o valor digitado seja escrito nesse mesmo atributo. Usa-se o atributo value:
`<h:inputText value="#{pessoaMB.nome}" />`
nesse caso, o componente é preenchido pela propriedade nome (String) de pessoaMB (executa getNome()) e, quando o formulário é submetido, seu conteúdo é inserido em nome (executa setNome());
- **Componente:** faz-se com que o elemento visual tenha um componente correspondente (da API Facelets) com todos os atributos e métodos, já que faz parte da árvore de componentes da visão construída. Usa-se o atributo binding:
`<h:inputText binding="#{pessoaMB.inputNome}" />`
nesse caso, o MB precisa de uma propriedade inputNome do tipo HtmlInputText (que é um componente da API do JSF).

Processamento:

- Quando mostra a tela, chama getNome() para preencher o inputText na fase de Render Response;
- Quando efetua a submissão, obtém o dado do inputText e executa o setNome() na fase de Update Model Values.



Caixas de texto, rótulos e campos ocultos

As caixas de texto são usadas para entrada de dados digitados pelo usuário.

São de três tipos:

- **<h:inputText />**: usada para entrar com textos simples. Renderiza um:
`<input type="text" />`
- **<h:inputArea />**: usada para entrar com textos em várias linhas. Renderiza um:
`<textarea>`
- **<h:inputSecret />**: usada para entrar com textos sem eco, por exemplo, senhas.
Renderiza um: `<input type="password" />`



Caixas de texto

A tabela 3.1 mostra alguns atributos do `<h:inputText />`.

Atributo	Descrição
id	Identificador do componente
value	Valor do componente, pode ligar a um MB com EL (<code>#{...}</code>)
readonly	true/false – somente leitura
disabled	true/false – elemento desabilitado (não recebe foco)
required	true/false – se o elemento é requerido
requiredMessage	Texto – mensagem se não preenchido
maxlength	Número – quantidade máxima de caracteres
size	Tamanho em caracteres do componente
title	tool tip text
style	Estilos CSS aplicados ao componente
styleClass	Lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

Tabela 3.1
Atributos de
`<h:inputText />`.

A seguir, temos um exemplo que mostra como um `<h:inputText />` é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Código em JSF

```
<h:outputLabel value="Nome: " for="nome" />
<h:inputText value="#{exemploMB.nome}" id="nome" />
```



Renderizado em HTML

```
<label for="j_idt5:nome">Nome: </label>
<input id="j_idt5:nome" type="text" name="j_idt5:nome" />
```

Nome:

Caixas de texto de múltiplas linhas

A tabela 3.2 mostra alguns atributos do `<h:inputTextarea />`.

Atributo	Descrição
id	Identificador do componente
cols	Número de linhas
rows	Número de colunas
value	Valor do componente, pode ligar a um MB com EL (<code>#{...}</code>)
readonly	true/false – somente leitura
disabled	true/false – elemento desabilitado (não recebe foco)
required	true/false – se o elemento é requerido
requiredMessage	texto – mensagem se não preenchido
maxlength	Número – quantidade máxima de caracteres
title	tool tip text
style	Estilos CSS aplicados ao componente
styleClass	Lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

Figura 3.2

Apresentação do `<h:inputText />` em JSF, renderizado em HTML e no navegador.

Tabela 3.2

Atributos de `<h:inputTextarea />`.

A figura a seguir mostra como um `<h:inputTextarea />` é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Código em JSF

```
<h:outputLabel value="Descrição: " for="descricao" />
<h:inputTextarea value="#{exemploMB.descricao}" id="descricao" />
```

Renderizado em HTML

```
<label for="j_idt5:descricao">Descrição: </label>
<textarea id="j_idt5:descricao" name="j_idt5:descricao">
</textarea>
```

Descrição:

Figura 3.3

Apresentação do `<h:inputTextarea />` em JSF, renderizado em HTML e no navegador.

Caixas de texto de senha

A tabela 3.3 mostra alguns atributos do `<h:inputSecret />`.

Atributo	Descrição
id	Identificador do componente
redisplay	(true/false) por default não é recarregado o valor do bean. Setando para true, o valor é trazido
value	Valor do componente, pode ligar a um MB com EL (<code>{...}</code>)
readonly	true/false – somente leitura
disabled	true/false – elemento desabilitado (não recebe foco)
required	true/false – se o elemento é requerido
requiredMessage	Texto – mensagem se não preenchido
maxlength	Número – quantidade máxima de caracteres
size	Tamanho em caracteres do componente
title	tool tip text
style	Estilos CSS aplicados ao componente
styleClass	Lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

Tabela 3.3
Atributos de
`<h:inputSecret />`.

Podemos ver adiante como um `<h:inputSecret />` é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

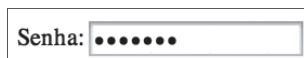
Código em JSF

```
<h:outputLabel value="Senha: " for="senha" />
<h:inputSecret value="#{exemploMB.senha}" id="senha" />
```

Renderizado em HTML

```
<label for="j_idt5:senha">Senha: </label>
<input id="j_idt5:senha" type="password" name="j_idt5:senha" value="" />
```

Figura 3.4
Apresentação do
`<h:inputSecret />`
em JSF, renderizado
em HTML e no
navegador.



Exemplo de caixas de texto

Os trechos de código a seguir mostram um exemplo de projeto contendo os três tipos de caixa de entrada.

```
<h:form>
  <h:panelGrid columns="2">
    <h:outputLabel value="Nome: " for="nome" />
    <h:inputText value="#{exemploMB.nome}" id="nome" />

    <h:outputLabel value="Senha: " for="senha" />
    <h:inputSecret value="#{exemploMB.senha}"
      id="senha" />
    <h:outputLabel value="Descrição: "
      for="descricao" />
    <h:inputTextarea value="#{exemploMB.descricao}"
      id="descricao" />

    <h:commandButton value="Enviar" />

  </h:panelGrid>
</h:form>
```

```
@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
    private String nome;
    private String senha;
    private String descricao;
    // setters/getters
}
```

A seguir, o formulário renderizado.

Figura 3.5

Os três tipos de caixa de entrada em um único formulário.

Rótulos

Para renderizar um rótulo de um componente no formulário, usa-se `<h:outputLabel />`. Esse componente aumenta a área clicável de elementos, como botões de rádio, e pode ser usado por leitores de tela em casos de aplicativos de acessibilidade. A tabela 3.4 mostra alguns atributos de `<h:outputLabel />`.

Atributo	Descrição
id	Identificador do componente
value	Texto do rótulo
for	Associa o rótulo a um componente. Deve conter o ID do componente para o qual ele é rótulo
style	Estilos CSS aplicados ao componente
styleClass	Lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

Tabela 3.4
Atributos de
<h:outputLabel>.

O código a seguir mostra como um <h:outputLabel /> é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Código em JSF

```
<h:outputLabel value="Nome: " for="nome" />
<h:inputText value="#{exemploMB.nome}" id="nome" />
```

Renderizado em HTML

```
<label for="j_idt5:nome">Nome: </label>
<input id="j_idt5:nome" type="text" name="j_idt5:nome" />
```

Figura 3.6
Apresentação do
<h:outputLabel />
em JSF, renderizado
em HTML e no
navegador.

Campos ocultos

A tag <h:inputHidden> renderiza um campo oculto (<input type="hidden" />). É usada para enviar informações em um formulário, sem que esse dado apareça para o usuário. A Tabela 3.5 apresenta seus principais atributos.

Atributo	Descrição
value	Valor do campo, pode ser ligado a uma propriedade de um bean através de EL (#{...})
id	Associa o rótulo a um componente, deve conter o ID do componente para o qual ele é rótulo

Tabela 3.5
Atributos de
<h:inputHidden>.

O código a seguir mostra como um <h:inputHidden /> é escrito em JSF e como é renderizado em HTML.

Código em JSF

```
<h:inputHidden value="#{exemploMB.nome}" />
```

Renderizado em HTML

```
<input type="hidden" name="j_idt5:j_idt10" />
```

Caixas de Seleção

O JSF fornece sete tipos de caixas de seleção, a saber:

- **<h:selectBooleanCheckbox>**: cria um CHECKBOX para seleção do tipo sim/não, vinculada a uma propriedade booleana;
- **<h:selectOneMenu>**: cria um COMBOBOX, onde somente uma opção é mostrada por vez e somente uma pode ser selecionada;
- **<h:selectManyListBox>**: cria um LISTBOX, onde várias opções são mostradas e várias podem ser selecionadas;
- **<h:selectManyCheckbox>**: cria vários campos CHECKBOX, para seleção de múltiplas escolhas;
- **<h:selectOneRadio>**: cria vários campos do tipo RADIO, mutuamente exclusivos (somente um pode estar selecionado);
- **<h:selectOneListBox>**: cria um LISTBOX, onde várias opções são mostradas, mas somente uma opção pode ser selecionada;
- **<h:selectManyMenu>**: cria um COMBOBOX, onde somente uma opção é mostrada por vez, mas várias podem ser selecionadas.

A figura 3.7 mostra um exemplo com todas as caixas de seleção renderizadas.

Figura 3.7
Todas as Caixas
de Seleção.

Caixa de Seleção: checkbox Único

O componente `<h:selectBooleanCheckbox>` renderiza um checkbox para seleção do tipo SIM/NÃO, que pode ser vinculada a uma propriedade booleana de um Managed Bean. Ele renderiza um componente em HTML `<input type="checkbox" />`. A tabela 3.6 apresenta seus principais atributos.

Atributo	Descrição
id	identificador do componente
value	(true/false) valor do componente, pode ligar a uma propriedade booleana de um MB
readonly	true/false – somente leitura
Disabled	true/false – elemento desabilitado (não recebe foco)
title	tool tip text
style	estilos CSS aplicados ao componente
styleClass	lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

Tabela 3.6
Atributos de
<h:selectBoolean
Checkbox>.

O código a seguir mostra como um <h:selectBooleanCheckbox> é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Código em JSF

```
<h:selectBooleanCheckbox id="teste" value="true" />
<h:outputLabel value="Teste" for="teste" />
```

Renderizado em HTML

```
<label for="j_idt5:teste">Teste</label>
<input id="j_idt5:teste" type="checkbox" name="j_idt5:teste" checked="checked" />
```

Figura 3.8
<h:selectBoolean
Checkbox> em
JSF, renderizado
em HTML e no
navegador.



Os códigos a seguir mostram o XHTML exemplo de uso e o Managed Bean usado.

```
<h:form>
  <h:panelGrid columns="2">
    <h:outputLabel value="Fumante" for="fumante" />
    <h:selectBooleanCheckbox id="fumante"
      value="#{exemploMB.fumante}" />
    <h:outputLabel value="Cardiaco" for="cardiaco" />
    <h:selectBooleanCheckbox id="cardiaco"
      value="#{exemploMB.cardiaco}" />
    <h:outputLabel value="Teste" for="teste" />
    <h:selectBooleanCheckbox id="teste" value="true" />
    <h:commandButton value="Enviar" />
  </h:panelGrid>
</h:form>

@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
  private boolean fumante;
  private boolean cardiaco;
  // setters/getters
}
```

Caixa de Seleção: comboBox de Seleção Única

O componente `<h:selectOneMenu>` renderiza um ComboBox contendo uma lista de opções para seleção, que pode ser vinculada a uma propriedade de um Managed Bean. Ele renderiza um componente `<select>` do HTML com atributo `size` com valor 1 e sem o atributo `multiple`. A tabela 3.7 apresenta seus principais atributos.

Tabela 3.7
Atributos de
`<h:selectOneMenu>`.

Atributo	Descrição
id	identificador do componente
value	valor do componente, pode ligar a uma propriedade de um MB
readonly	true/false – somente leitura
disabled	true/false – elemento desabilitado (não recebe foco)
title	tool tip text
style	estilos CSS aplicados ao componente
styleClass	lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

Para preenchimento dos itens usa-se a tag `<f:selectItem>` da biblioteca core do JSF. A tabela 3.8 apresenta seus principais atributos.

Atributo	Descrição
itemLabel	Texto apresentado para esta opção
itemValue	Valor a ser submetido quando esta opção estiver selecionada
itemDisabled	true/false, flag indicando se o item está desabilitado (default false)
noSelectionOption	true/false, flag indicando se o item é “não selecionável” e, em caso de campo requerido, gera erro se estiver selecionado (default false)

O código a seguir mostra como um `<h:selectOneMenu>` é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Tabela 3.8
Atributos de
`<f:selectItem>`.

Código em JSF

```
<h:outputLabel value="Sexo: " for="sexo" />
<h:selectOneMenu id="sexo" >
    <f:selectItem id="M" itemLabel="Masculino" itemValue="M" />
    <f:selectItem id="F" itemLabel="Feminino" itemValue="F" />
</h:selectOneMenu>
```

Renderizado em HTML

```
<label for="j_idt5:sexo">Sexo: </label>
<select id="j_idt5:sexo" name="j_idt5:sexo" size="1">
    <option value="M">Masculino</option>
    <option value="F">Feminino</option>
</select>
```



Figura 3.9
`<h:selectOneMenu>`
em JSF, renderizado
em HTML e no
navegador.

O código a seguir mostra o XHTML e o Managed Bean usados no exemplo.

```
<h:form>
  <h:panelGrid columns="2">
    <h:outputLabel value="Sexo: " for="sexostr" />
    <h:outputText value="#{exemploMB.sexo}"
      id="sexostr" />

    <h:outputLabel value="Sexo: " for="sexo" />
    <h:selectOneMenu id="sexo"
      value="#{exemploMB.sexo}" >
      <f:selectItem id="M" itemLabel="Masculino"
        itemValue="M" />
      <f:selectItem id="F" itemLabel="Feminino"
        itemValue="F" />
    </h:selectOneMenu>

    <h:commandButton value="Enviar" />
  </h:panelGrid>
</h:form>
```

```
@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
  private String sexo;
  // setters/getters
}
```

Caixa de Seleção: listBox de Seleção Múltipla

O componente `<h:selectManyListbox>` renderiza um `Listbox` contendo uma lista de opções para seleção múltipla (pressionando-se a tecla CTRL ou CMD), que pode ser vinculada a uma propriedade de um Managed Bean. Ele renderiza um componente `<select>` do HTML com atributo `size` igual ao número de opções e com o atributo `multiple`. A tabela 3.9 apresenta seus principais atributos.

Atributo	Descrição
id	identificador do componente
value	Valor dos itens selecionados (casa com o <code>itemValue</code> do item), pode ligar a uma propriedade de um MB, que deve ser uma lista.
size	Quantidade de elementos a serem mostrados, se não for especificado, todos serão igual ao número de opções). Se houver necessidade, um scrollbar é mostrado
readonly	true/false – somente leitura
disabled	true/false – elemento desabilitado (não recebe foco)
title	tool tip text
style	estilos CSS aplicados ao componente

Tabela 3.9
Atributos de
`<h:selectManyListbox>`.

Atributo	Descrição
styleClass	lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

O código a seguir mostra como um `<h:selectManyListbox>` é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Código em JSF

```
<h:outputLabel value="Cor: " for="cor" />
<h:selectManyListbox id="cor" >
    <f:selectItem id="M" itemLabel="Verde" itemValue="V" />
    <f:selectItem id="F" itemLabel="Azul" itemValue="A" />
</h:selectManyListbox>
```

Renderizado em HTML

```
<label for="j_idt5:cor">Cor: </label>
<select id="j_idt5:cor" name="j_idt5:cor" multiple="multiple" size="2">
    <option value="V">Verde</option>
    <option value="A">Azul</option>
</select>
```

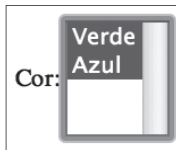


Figura 3.10
`<h:selectManyListbox>` em JSF, renderizado em HTML e no navegador.

Os códigos a seguir mostram o XHTML e o Managed Bean usados no exemplo.

```
<h:form>
    <h:panelGrid columns="2">
        <h:outputLabel value="Estados: " for="estadostr" />
        <h:outputText value="#{exemploMB.estados}" id="estadostr" />
        <h:outputLabel value="Estados: " for="estados" />
        <h:selectManyListbox id="estados" value="#{exemploMB.estados}"
            size="3" >
            <f:selectItem id="pr" itemLabel="Paraná" itemValue="PR" />
            <f:selectItem id="sc" itemLabel="Santa Catarina"
                itemValue="SC" />
            <f:selectItem id="rs" itemLabel="Rio Grande do Sul"
                itemValue="RS" />
            <f:selectItem id="sp" itemLabel="São Paulo" itemValue="SP" />
            <f:selectItem id="mg" itemLabel="Minas Gerais" itemValue="MG" />
        </h:selectManyListbox>
        <h:commandButton value="Enviar" />
    </h:panelGrid>
</h:form>
```

```

@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
    private List<String> estados;
    @PostConstruct
    public void init() {
        estados = new ArrayList<String>();
        estados.add("pr");
        estados.add("sc");
    }
    public List<String> getEstados() {
        return this.estados;
    }
    public void setEstados(List<String> estados) {
        this.estados = estados;
    }
}

```

3.1.4 @PostConstruct e @PreDestroy

No código acima foi usado uma anotação especial do CDI no Managed Bean, o @PostConstruct (javax.annotation.PostConstruct). Um método anotado dessa forma é chamado após a construção do objeto e após todas as injeções serem feitas, mas antes do objeto ser disponibilizado para operação. Assim, prefere-se iniciar os objetos dessa forma, pois todas as dependências já foram resolvidas e também porque há garantia de que esse método é chamado somente uma vez, o que não ocorre com o construtor, por causa do mecanismo de Proxy usado pelo CDI.

O método init() foi usado para inicializar alguns checkboxes, trazendo-os já marcados na primeira vez que a tela for mostrada ao usuário.

Analogamente, podemos anotar um método com @PreDestroy (javax.annotation.PreDestroy), que é invocando antes da instância ser destruída.

Somente pode haver um método anotado com @PostConstruct e um com @PreDestroy.

- @PostConstruct.
 - ▣ O método anotado é invocado logo após a criação do objeto;
 - ▣ Todas as injeções já foram feitas;
 - ▣ Sempre usado para inicializar o bean, ao invés do construtor.
- @PreDestroy.
 - ▣ O método anotado é invocado antes da destruição.



Exercícios

Caixa de Seleção: múltiplos Checkboxes

O componente <h:selectManyCheckbox> renderiza uma tabela contendo uma lista de checkbox para seleção do tipo SIM/NÃO, que poder ser vinculada a uma propriedade lista de um Managed

Bean. A tabela 3.10 apresenta seus principais atributos.

Atributo	Descrição
id	identificador do componente
value	(true/false) valor do componente, pode ligar a uma propriedade lista de um MB
layout	Aparência da tabela renderizada: ☐ Santa Catarina ☐ Paraná ☐ Rio Grande do Sul ☐ Santa Catarina ☐ Paraná ☐ Rio Grande do Sul
border	Tamanho (px) da borda da tabela renderizada
readonly	true/false – somente leitura
disabled	true/false – elemento desabilitado (não recebe foco)
title	tool tip text
style	estilos CSS aplicados ao componente
styleClass	lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

O código a seguir mostra como um `<h:selectManyCheckbox>` é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Tabela 3.10
Atributos de
`<h:selectManyCheckbox>`.

Código em JSF

```
<h:selectManyCheckbox id="teste" layout="pageDirection" >
  <f:selectItem id="sc" itemLabel="Santa Catarina"
    itemValue="SC" />
  <f:selectItem id="pr" itemLabel="Paraná" itemValue="PR" />
  <f:selectItem id="rs" itemLabel="Rio Grande do Sul"
    itemValue="RS" />
</h:selectManyCheckbox>
```

Renderizado em HTML

```
<table border="0" id="j_idt5:teste">
<tr><td>
  <input name="j_idt5:teste" id="j_idt5:teste:0" value="SC" type="checkbox" />
  <label for="j_idt5:teste:0" class=""> Santa Catarina</label></td>
</tr><tr><td>
  <input name="j_idt5:teste" id="j_idt5:teste:1" value="PR" type="checkbox" />
  <label for="j_idt5:teste:1" class=""> Paraná</label></td>
</tr><tr><td>
  <input name="j_idt5:teste" id="j_idt5:teste:2" value="RS" type="checkbox" />
  <label for="j_idt5:teste:2" class=""> Rio Grande do Sul</label></td>
</tr>
```

```
</table>
```

Figura 3.11
<h:selectMany
Checkbox> em
JSF, renderizado
em HTML e no
navegador.

☐	Santa Catarina
☐	Paraná
☐	Rio Grande do Sul

Os códigos a seguir mostram o XHTML e o Managed Bean usados no exemplo.

```
<h:form>
  <h:panelGrid columns="2">
    <h:outputLabel value="Estados" for="estados" />
    <h:selectManyCheckbox id="estados"
      value="#{exemploMB.escolhidos}"
      layout="pageDirection">
      <f:selectItem id="sc" itemLabel="Santa Catarina"
        itemValue="SC" />
      <f:selectItem id="pr" itemLabel="Paraná"
        itemValue="PR" />
      <f:selectItem id="rs"
        itemLabel="Rio Grande do Sul"
        itemValue="RS" />
    </h:selectManyCheckbox>

    <h:commandButton value="Enviar" />
  </h:panelGrid>
</h:form>
```

```
@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
  private List<String> escolhidos = new ArrayList<>();
  @PostConstruct
  public void init() {
    escolhidos.add("PR");
    escolhidos.add("SC");
  }
  // setters/getters
}
```

Caixa de Seleção: botões de Rádio

O componente <h:selectOneRadio> renderiza uma tabela contendo uma lista de botões de rádio para seleção mutuamente exclusiva (somente um pode estar selecionado em cada momento) do tipo SIM/NÃO, que poder ser vinculada a uma propriedade booleana de um

Managed Bean. A tabela 3.12 apresenta seus principais atributos.

Atributo	Descrição
id	identificador do componente
value	valor do componente, pode ligar a uma propriedade de um MB
layout	Aparência da tabela renderizada: ▪ lineDirection: para mostrar os elementos horizontalmente ▪ pageDirection: para mostrar os elementos verticalmente
border	Tamanho (px) da borda da tabela renderizada
readonly	true/false – somente leitura
disabled	true/false – elemento desabilitado (não recebe foco)
title	tool tip text
style	estilos CSS aplicados ao componente
styleClass	lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

O código a seguir mostra como um `<h:selectOneRadio>` é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Código em JSF

```
<h:selectOneRadio id="sexo" layout="pageDirection" >
    <f:selectItem id="M" itemLabel="Masculino" itemValue="M" />
    <f:selectItem id="F" itemLabel="Feminino" itemValue="F" />
</h:selectOneRadio>
```

Renderizado em HTML

```
<table id="j_idt5:sexo">
<tr><td>
    <input type="radio" name="j_idt5:sexo" id="j_idt5:sexo:0" value="M" />
    <label for="j_idt5:sexo:0"> Masculino</label></td>
</tr><tr><td>
    <input type="radio" name="j_idt5:sexo" id="j_idt5:sexo:1" value="F" />
    <label for="j_idt5:sexo:1"> Feminino</label></td>
</tr>
</table>
```



Tabela 3.11
Atributos de
`<h:selectOneRadio>`.

Figura 3.12
`<h:selectOneRadio>`
em JSF, renderizado
em HTML e no
navegador.

O código a seguir mostra o XHTML e o Managed Bean usados no exemplo.

```
<h:form>
  <h:panelGrid columns="2">
    <h:outputLabel value="Sexo: " for="sexostr" />
    <h:outputText value="#{exemploMB.sexo}"
                  id="sexostr" />

    <h:outputLabel value="Sexo: " for="sexo" />
    <h:selectOneRadio id="sexo"
                     value="#{exemploMB.sexo}"
                     layout="pageDirection">
      <f:selectItem id="M" itemLabel="Masculino"
                   itemValue="M" />
      <f:selectItem id="F" itemLabel="Feminino"
                   itemValue="F" />
    </h:selectOneRadio>

    <h:commandButton value="Enviar" />
  </h:panelGrid>
</h:form>
```

```
@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
    private String sexo;
    // setters/getters
}
```

3.1.7 Caixa de Seleção: listBox de Seleção Única

O componente `<h:selectOneListbox>` renderiza um `ListBox` contendo uma lista de opções para seleção, que poder ser vinculada a uma propriedade de um Managed Bean. Ele renderiza um componente `<select>` do HTML com atributo `size` igual ao número de opções sem o atributo `multiple`. A tabela 3.13 apresenta seus principais atributos.

Tabela 3.13
Atributos de
`<h:selectOneListbox>`.

Atributo	Descrição
id	identificador do componente
value	valor do componente, pode ligar a uma propriedade de um MB
Size	Quantidade de elementos a serem mostrados, se não for especificado, todos serão (igual ao número de opções). Se houver necessidade, um scrollbar é mostrado
readonly	true/false – somente leitura
disabled	true/false – elemento desabilitado (não recebe foco)
title	tool tip text
style	estilos CSS aplicados ao componente
styleClass	lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

O código a seguir mostra como um `<h:selectOneListbox>` é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Código em JSF

```
<h:outputLabel value="Cor: " for="cor" />
<h:selectOneListbox id="cor" >
    <f:selectItem id="M" itemLabel="Verde" itemValue="V" />
    <f:selectItem id="F" itemLabel="Azul" itemValue="A" />
</h:selectOneListbox>
```

Renderizado em HTML

```
<label for="j_idt5:cor">Cor: </label>
<select id="j_idt5:cor" name="j_idt5:cor" size="2">
    <option value="V">Verde</option>
    <option value="A">Azul</option>
</select>
```



Figura 3.13
`<h:selectOneListbox>` em JSF, renderizado em HTML e no navegador.

Os códigos a seguir mostram o XHTML e o Managed Bean usados no exemplo anterior.

```
<h:form>
    <h:panelGrid columns="2">
        <h:outputLabel value="Estado: " for="estadostr" />
        <h:outputText value="#{exemploMB.estado}"
            id="estadostr" />
        <h:outputLabel value="Estado: " for="estado" />
        <h:selectOneListbox id="estado"
            value="#{exemploMB.estado}"
            size="3" >
            <f:selectItem id="pr" itemLabel="Paraná"
                itemValue="PR" />
            <f:selectItem id="sc" itemLabel="Santa Catarina"
                itemValue="SC" />
            <f:selectItem id="rs"
                itemLabel="Rio Grande do Sul"
                itemValue="RS" />
            <f:selectItem id="sp" itemLabel="São Paulo"
                itemValue="SP" />
            <f:selectItem id="mg" itemLabel="Minas Gerais"
                itemValue="MG" />
        </h:selectOneListbox>
        <h:commandButton value="Enviar" />
    </h:panelGrid>
</h:form>
```



```

@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
    private String estado;
    // setters/getters
}

```

Caixa de Seleção: comboBox de Seleção Múltipla

O componente `<h:selectManyMenu>` renderiza um `ListBox` contendo uma lista de opções para seleção múltipla (pressionando-se a tecla CTRL ou CMD), que poder ser vinculada a uma propriedade de um Managed Bean. Ele renderiza um componente `<select>` do HTML com atributo `size` igual 1 e com o atributo `multiple`. A tabela 3.14 apresenta seus principais atributos.

Atributo	Descrição
id	identificador do componente
value	Valor dos itens selecionados (casa com o <code>itemValue</code> do item), pode ligar a uma propriedade de um MB, que deve ser uma lista.
readonly	true/false – somente leitura
disabled	true/false – elemento desabilitado (não recebe foco)
title	tool tip text
style	estilos CSS aplicados ao componente
styleClass	lista de classes de CSS aplicados ao componente (renderizados como atributo <code>class</code>)
Eventos do DOM	onfocus, onclick etc.

Tabela 3.14
Atributos de
`<h:selectManyMenu>`.

O código a seguir mostra como um `<h:selectManyListbox>` é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Código em JSF

```

<h:outputLabel value="Cor: " for="cor" />
<h:selectManyMenu id="cor" >
    <f:selectItem id="M" itemLabel="Verde" itemValue="V" />
    <f:selectItem id="F" itemLabel="Azul" itemValue="A" />
</h:selectManyMenu>

```

Renderizado em HTML

```

<label for="j_idt5:cor">Cor: </label>
<select id="j_idt5:cor" name="j_idt5:cor" multiple="multiple" size="1">
    <option value="V">Verde</option>
    <option value="A">Azul</option>
</select>

```

Figura 3.14
`<h:selectManyListbox>` em JSF, renderizado em HTML e no navegador..



Os códigos a seguir mostram o XHTML e o Managed Bean usados no exemplo.

```
<h:form>
  <h:panelGrid columns="2">
    <h:outputLabel value="Estados: " for="estadostr" />
    <h:outputText value="#{exemploMB.estados}" id="estadostr" />
    <h:outputLabel value="Estados: " for="estados" />
    <h:selectManyMenu id="estados" value="#{exemploMB.estados}" >
      <f:selectItem id="pr" itemLabel="Paraná" itemValue="PR" />
      <f:selectItem id="sc" itemLabel="Santa Catarina"
        itemValue="SC" />
      <f:selectItem id="rs" itemLabel="Rio Grande do Sul"
        itemValue="RS" />
      <f:selectItem id="sp" itemLabel="São Paulo" itemValue="SP" />
      <f:selectItem id="mg" itemLabel="Minas Gerais" itemValue="MG" />
    </h:selectManyMenu>
    <h:commandButton value="Enviar" />
  </h:panelGrid>
</h:form>
```

```
@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
    private List<String> estados;
    // setters/getters
}
```

Opções Estáticas x Dinâmicas

Os componentes que renderizam listas de opções podem ser carregados de forma estática e dinâmica e a seleção pode ser única ou múltipla.

Opções Estáticas

- Coloca opções através de tags <f:selectItem>
- Para cada ocorrência um item é renderizado
- Por exemplo:
 - ▣ <h:selectOneMenu value="#{pessoaMB.estado}">
 - ▣ <f:selectItem itemValue="PR" itemLabel="Paraná" />
 - ▣ <f:selectItem itemValue="SC" itemLabel="Santa Catarina" />
 - ▣ </h:selectOneMenu>
- Atributos
 - ▣ itemValue: o que será enviado se o item for selecionado
 - ▣ itemLabel: descrição, o que é mostrado para o usuário na opção



No exemplo anterior é renderizado um ComboBox (`<select>`) e dentro dele dois elementos do tipo `<option>`. A tabela 3.8 apresentada anteriormente mostra os atributos de `<f:selectItem>`.

Opções dinâmicas

- Alterações nas listas podem ser dinamicamente mostradas
- `<f:selectItems>`: executa um laço entre os vários elementos indicados em sua propriedade `values`, montando as opções da tag de seleção
- Por exemplo:
 - ▢ `<h:selectOneMenu value="#{pessoaMB.estado}">`
 - ▢ `<f:selectItems value="#{pessoaMB.listaEstados}"`
 - ▢ `var="estado"`
 - ▢ `itemValue="#{estado.sigla}"`
 - ▢ `itemLabel="#{estado.nome}" />`
 - ▢ `</h:selectOneMenu>`
- Atributos:
 - ▢ `value`: lista de itens a serem varridos
 - ▢ `var`: variável que assume um item em cada iteração sobre os itens especificados em `value`
 - ▢ `itemLabel`: o texto (rótulo) de cada opção
 - ▢ `itemValue`: valor do item que será submetido, quando esta opção estiver selecionada

Nesse caso acima, o ComboBox terá como elementos todo o conteúdo de `#{pessoaMB.listaEstados}`.

A tabela 3.15 mostra os atributos de `<f:selectItems>`.

Atributo	Descrição
<code>value</code>	Lista de itens a ser varrida. Para cada element dessa lista, um item será gerado.
<code>var</code>	Nome da variável que receberá um item da lista a cada iteração do laço
<code>itemLabel</code>	O texto (rótulo) de cada opção
<code>itemValue</code>	Valor do item que será submetido, quando esta opção estiver selecionada
<code>itemDisabled</code>	(true/false) Se o elemento deverá estar desabilitado
<code>noSelectionValue</code>	Se o elemento pode ser selecionado ou somente é um descritivo

Tabela 3.15
Atributos de
`<f:selectItems>`.

Os códigos a seguir mostram o XHTML de uma aplicação usando carga dinâmica de opções em um ComboBox (`<f:selectOneMenu>`), o Managed Bean e o Bean Estado, criado para armazenar os dados a serem mostrados.

```

<h:form>
  <h:panelGrid columns="2">
    <h:outputLabel value="Selecionado: " for="selecionadostr" />
    <h:outputText value="#{exemploMB.selecionado}" id="selecionadostr" />

    <h:outputLabel value="Estados: " for="estado" />
    <h:selectOneMenu id="estado" value="#{exemploMB.selecionado}" >
      <f:selectItems value="#{exemploMB.listaEstados}"
        var="estado"
        itemLabel="#{estado.nome}"
        itemValue="#{estado.sigla}" />
    </h:selectOneMenu>
    <h:commandButton value="Enviar" />
  </h:panelGrid>
</h:form>

```

```

@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
  private String selecionado;
  private List<Estado> listaEstados;

  @PostConstruct
  public void init() {
    listaEstados = new ArrayList<Estado>();
    Estado e = new Estado();
    e.setSigla("PR");
    e.setNome("Paraná");
    listaEstados.add(e);
    e = new Estado();
    e.setSigla("SC");
    e.setNome("Santa Catarina");
    listaEstados.add(e);
  }
  // setters/getters
}

```

```

public class Estado {
  private String sigla;
  private String nome;
  // setters/getters
}

```

Deve-se notar no exemplo anterior que a inicialização do Managed Bean foi feita no método `init()`, anotado com `@PostConstruct`.

Armazenar String x Objeto

No exemplo anterior o estado escolhido é setado no MB pela sua sigla, mas é possível também armazenar diretamente o objeto do tipo Estado selecionado. Para isso é necessário um Conversor Customizado que efetua duas operações básicas:

- Converte String (da requisição) para Objeto;
- Converte Objeto para String (para resposta).

Esses conversões serão vistas futuramente.

Seleção Única x Seleção Múltipla

Os componentes de seleção também podem ser divididos naqueles que permitem seleção única e seleção múltipla, como mostra a tabela 3.16.

	Seleção Única	Seleção Múltipla
<h:selectBooleanCheckbox>	X	
<h:selectManyCheckbox>		X
<h:selectOneRadio>	X	
<h:selectOneMenu>	X	
<h:selectOneListbox>	X	
<h:selectManyListbox>		X
<h:selectManyMenu>		X

Tabela 3.16
Seleção Única x
Seleção Múltipla..

Os componentes de seleção única precisam ligar seu atributo value a somente um elemento no Managed Bean, por exemplo, no XHTML:

```
<h:selectOneRadio id="sexo"
    value="#{exemploMB.sexo}"
    layout="pageDirection">
    <f:selectItem id="M" itemLabel="Masculino"
        itemValue="M" />
    <f:selectItem id="F" itemLabel="Feminino"
        itemValue="F" />
</h:selectOneRadio>
```

E no MB:

```
@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
    private String sexo;
    // setters/getters
}
```

Já os componentes que possuem seleção múltipla devem ter uma lista de elementos para receber os elementos selecionados e submetidos. Por exemplo, no XHTML:

```
<h:selectManyListbox id="estados"
    value="#{exemploMB.estados}"
    size="3" >
    <f:selectItem id="pr" itemLabel="Paraná"
        itemValue="PR" />
    <f:selectItem id="sc" itemLabel="Santa Catarina"
        itemValue="SC" />
    <f:selectItem id="rs"
        itemLabel="Rio Grande do Sul"
        itemValue="RS" />
    <f:selectItem id="sp" itemLabel="São Paulo"
        itemValue="SP" />
    <f:selectItem id="mg" itemLabel="Minas Gerais"
        itemValue="MG" />
</h:selectManyListbox>
```

E no MB:

```
@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
    private List<String> estados;
    // setters/getters
}
```

Opção Não Selecionada

Para que um componente com múltiplas opções apresente uma opção não selecionada, usa-se o atributo `noSelectionOption` em algum item, conforme o seguinte exemplo:

```
<h:selectOneMenu value="#{pessoaMB.estados}">
    <f:selectItem itemLabel="Nenhum"
        noSelectionOption="true" />
    <f:selectItems value="#{pessoaMB.estados}"
        var="estado"
        itemValue="#{estado.sigla}"
        itemLabel="#{estado.nome}" />
</h:selectOneMenu>
```

4

JSF – Componentes visuais

objetivos

Aprender sobre os demais componentes visuais, para criar interfaces gráficas em JSF.

Botões e links; Textos; Imagens; Biblioteca de recursos; Versionamento: Javascript e CSS; Mensagens.

conceitos

Botões e links

O JSF possui cinco tipos de botões e links, com variações nos métodos de submissão (POST ou GET) e no destino das páginas. São eles:

Botões:

- **<h:commandButton>**: renderiza um botão que, quando clicado, submete um formulário via POST;
- **<h:button>**: renderiza um botão que realiza uma requisição via GET para uma página do sistema quando clicado.

Links:

- **<h:commandLink>**: renderiza um link que, quando clicado, submete um formulário via POST;
- **<h:link>**: renderiza um link que realiza uma requisição via GET para uma página do sistema quando clicado.

Link externo:

- **<h:outputLink>**: renderiza um link externo para uma URL completa.

A tabela 4.1 mostra um comparativo entre os componentes de botões e links, no que diz respeito ao tipo, método de submissão e destino das páginas.

Componente	Tipo	Submissão	Destino
<h:commandButton>	BOTÃO	POST	JSF
<h:button>	BOTÃO	GET	JSF
<h:commandLink>	LINK	POST	JSF
<h:link>	LINK	GET	JSF
<h:outputLink>	LINK	GET	Externas

Tabela 4.1
Comparativo entre
botões e links.

Cada uma destas variantes de botões e links será explorada em maior detalhe a seguir.

Botão de Ação <h:commandButton>

O componente <h:commandButton> é um botão com ação de submissão, para um formulário via POST. Ele renderiza um <input type="submit" />. A tabela 317 apresenta seus principais atributos.

Atributos	Descrição
value	Contém o texto do botão
action	Contém o método do MB a ser invocado. Exemplo: #{mb.acao}
actionListener	Contém o método de um ActionListener, que será invocado quando o botão for pressionado. Exemplo: #{mb.acaoListener}
type	O tipo do botão renderizado (submit/button/reset). Por default, gera um botão do tipo submit
image	A URL ou caminho da imagem que será renderizada como um botão de submit (renderiza um <input type="image" src="..." />)
disabled	(true/false) Se o botão estará desabilitado
style	Estilos CSS aplicados ao componente
styleClass	Lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

A figura a seguir mostra como um <h:commandButton> é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Tabela 4.2

Atributos de <h:commandButton>.

Código em JSF

```
<h:commandButton action="#{exemploMB.acaoBotao}"
                 value="Botão de Comando" />
```

Renderizado em HTML

```
<input type="submit" name="j_idt5:j_idt10" value="Botão de Comando" />
```

Botão de Comando

Figura 4.1

<h:commandButton> – JSF, HTML e imagem renderizada.

Link de ação: <h:commandLink>

O componente <h:commandLink> é um link com ação de submissão, para um formulário via POST. Ele renderiza um , mas com scripts específicos para submissão do formulário. A tabela 4.3 apresenta seus principais atributos.

Atributos	Descrição
value	Contém o texto do botão
action	Contém o método do MB a ser invocado. Exemplo: #{mb.acao}
actionListener	Contém o método de um ActionListener, que será invocado quando o botão for pressionado. Exemplo: #{mb.acaoListener}

Tabela 4.3
Atributos de
<h:commandLink>.

Atributos	Descrição
type	Tipo do recurso do link. Exemplo: stylesheet
disabled	(true/false) Se o botão estará desabilitado
style	Estilos CSS aplicados ao componente
styleClass	Lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

A figura a seguir mostra como um <h:commandLink> é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Código em JSF

```
<h:commandLink action="#{exemploMB.acaoLink}"
value="Link de Comando"/>
```

Renderizado em HTML

```
<script type="text/javascript" src="/BotoesLinks/faces/javax.faces.resource/jsf.
js?ln=javax.faces&stage=Development">
</script>
<a href="#" onclick="mojarra.jsfcljs(document.getElementById('j_idt5'),{'j_idt5:j_
idt11':'j_idt5:j_idt11'},');return false">Link de Comando</a>
```

Figura 4.2
<h:commandLink> –
JSF, HTML e imagem
renderizada.

Link de Comando

Botão <h:button>

Tabela 4.4
Atributos de
<h:button>.

O componente <h:button> é um botão que direciona para uma página da aplicação. Ele renderiza um <input type="button"> e seu redirecionamento é um método GET. A tabela 4.4 apresenta seus principais atributos.

Atributos	Descrição
value	Contém o texto do botão
outcome	O nome da página (sem .xhtml) para aonde a aplicação será direcionada
disabled	(true/false) Se o botão estará desabilitado
image	A URL ou caminho da imagem que será renderizada como um botão de submit (renderiza um <input type="image" src="..." />)
style	Estilos CSS aplicados ao componente
styleClass	Lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

O código a seguir mostra como um <h:button> é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Código em JSF

```
<h:button outcome="teste" value="Botão"/>
```

Renderizado em HTML

```
<input type="button" onclick="window.location.href='/BotoesLinks/faces/teste.xhtml'; return false;" value="Botão" />
```



Figura 4.3
<h:button> – JSF, HTML e imagem renderizada.

Link <h:link>

O componente <h:link> é um link que direciona para uma página da aplicação. Ele renderiza um e seu redirecionamento é um método GET. A tabela 4.5 apresenta seus principais atributos.

Atributos	Descrição
value	Contém o texto do botão
outcome	O nome da página (sem .xhtml) para aonde a aplicação será direcionada
disabled	(true/false) Se o botão estará desabilitado
style	Estilos CSS aplicados ao componente
styleClass	Lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

A figura 4.4 mostra como um <h:link> é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Tabela 4.5
Atributos de <h:link>.

Código em JSF

```
<h:link outcome="teste" value="Link"/>
```

Renderizado em HTML

```
<a href="/BotoesLinks/faces/teste.xhtml">Link</a>
```



Figura 4.4
<h:link> – JSF, HTML e imagem renderizada.

Link Externo: <h:outputLink>

O componente <h:outputLink> é um link que direciona para uma página externa à aplicação, não JSF. Ele renderiza um , e seu redirecionamento é um método GET. O texto do link deve ser colocado dentro do outputLink. A tabela 4.6 apresenta seus principais atributos.

Atributos	Descrição
value	Para aonde será redirecionado
disabled	(true/false) Se o botão estará desabilitado
style	Estilos CSS aplicados ao componente
styleClass	Lista de classes de CSS aplicados ao componente (renderizados como atributo class)
Eventos do DOM	onfocus, onclick etc.

Tabela 4.6
Atributos de
<h:outputLink>.

A figura a seguir mostra como um <h:outputLink> é escrito em JSF, como é renderizado em HTML e como é apresentado no navegador.

Código em JSF

```
<h:outputLink value="http://www.google.com.br">
    <h:outputText value="Google" />
</h:outputLink>
```

Renderizado em HTML

```
<a href="http://www.google.com.br">Google</a>
```

Figura 4.5
<h:outputLink> –
JSF, HTML e imagem
renderizada.



Exemplo com botões e links

Os códigos a seguir apresentam um exemplo com os cinco tipos de botões e links.

Index.xhtml

```
<h:form>
    <h:panelGrid columns="2">
        <h:outputLabel value="Ação: " for="txtacao"/>
        <h:outputText id="txtacao" value="#{exemploMB.acao}" />
        <h:outputLabel value="Texto Antigo: " for="txtantigo"/>
        <h:outputText id="txtantigo" value="#{exemploMB.texto}" />
        <h:outputLabel value="Texto Novo: " for="txtnovo"/>
        <h:inputText id="txtnovo" value="#{exemploMB.texto}" />
        <h:commandButton action="#{exemploMB.acaoBotao}" value="Ok" />
        <h:commandLink action="#{exemploMB.acaoLink}" value="Ok"/>
        <h:button outcome="teste" value="Ok"/>
        <h:link outcome="teste" value="Ok"/>
        <h:outputLink value="http://www.google.com.br">
            <h:outputText value="Google" />
        </h:outputLink>
    </h:panelGrid>
</h:form>
```

Managed Bean

```
@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
    private String texto;
    private String acao;
    // setters/getters
    public void acaoBotao() {
        this.acao = "Pressionou um BOTÃO de ação";
    }
    public void acaoLink() {
        this.acao = "Pressionou um LINK de ação";
    }
}
```

Teste.xhtml

```
Teste de Links <br/>
<h:link outcome="index" value="Voltar" />
```

Textos

Para inserir textos nas páginas XHTML, temos dois componentes:

- **<h:outputText>**: o que for escrito no atributo value é apresentado;
- **<h:outputFormat>**: o atributo value é um texto formatado onde se pode inserir fragmentos via parâmetros.



Textos simples

O `<h:outputText>` é recomendado quando se precisa fazer uma renderização condicional (usando o atributo `rendered`), quando precisamos fazer formatações (um elemento `<span>` é renderizado) ou quando se necessita alguma funcionalidade via AJAX. Outra utilidade que torna o uso desta tag extremamente recomendado é o fato de o JSF já ter embutido um mecanismo de prevenção de ataques do tipo XSS (Cross-Site Scripting), que é uma vulnerabilidade onde um hacker consegue colocar códigos Javascript dentro de campos no site, fazendo com que ele obtenha controle do sistema, como por exemplo, monitorando entradas de usuário ou senha. A figura a seguir traz um exemplo simples utilizando `<h:outputText>`.

Código em JSF

```
<h:outputText value="Teste" styleClass="classecss" />
```

Renderizado em HTML

```
<span class="classecss">Teste</span>
```

Teste

Figura 4.6
`<h:outputText>` –
JSF, HTML e texto
renderizado.

Textos formatados

O `<h:outputFormat>` é usado para criar textos formatados através de parametrização. Também é recomendando quando se deseja concatenar textos com variáveis, quando se precisa fazer uma renderização condicional (usando-se o atributo `rendered`), quando se precisa fazer formatações (um elemento `<span>` é renderizado) ou quando se necessita alguma funcionalidade via AJAX.

Para criar os textos formatados usam-se parâmetros que são adicionados dentro da string final. Os parâmetros começam em `{0}`, depois `{1}` e assim por diante. Para preencher esses parâmetros na string, usamos a tag `<f:param>`, que possui os seguintes atributos:

Atributo	Descrição
name	Nome do parâmetro. No caso do <code><h:outputFormat></code> , não precisa ser utilizado, pois a ordem em que os parâmetros são colocados é a ordem em que serão inseridos na string
value	Dado a ser inserido na string

Tabela 4.7
Atributos de
`<f:param>`.

O código a seguir mostra como um `<h:outputFormat>` é apresentado.

Código em JSF

```
<h:outputFormat value="Meu nome é {0} e tenho {1} anos."  
                styleClass="classecss" >  
    <f:param value="Maria" />  
    <f:param value="20" />  
</h:outputFormat>
```

Renderizado em HTML

```
<span class="classecss">Meu nome é Maria e tenho 20 anos.</span>
```

Figura 4.7
`<h:outputFormat>`
– JSF, HTML e texto
renderizado.

Meu nome é Maria e tenho 20 anos.

Imagens

Tabela 4.8
Atributos de
`<h:graphicImage>`.

O componente do JSF usado para inserir imagens é o `<h:graphicImage>`. Esse componente renderiza um `<img src="" />`. A tabela 4.8 mostra alguns de seus atributos.

Atributo	Descrição
value	A "url" é a imagem a ser mostrada. Se iniciar com "/", procura a imagem no contexto da aplicação. Pode-se colocar uma URL completa da imagem. Também é possível que aponte para um recurso (imagem) diretamente retornado por um MB
width	Sobrescreve o atributo de largura
height	Sobrescreve o atributo de altura
usemap	O nome do mapa que foi criado no HTML
alt	Texto alternativo se a imagem não for encontrada
library	Nome da biblioteca de imagens sendo utilizada. Deve ser um subdiretório do diretório "/resources", que está na raiz da aplicação
name	Usado juntamente com o atributo library. Nome da imagem que está dentro da biblioteca

O código a seguir mostra como o `<h:graphicImage>` é apresentado.

Código em JSF

```
<h:graphicImage value="http://docs.oracle.com/javaee/6/tutorial/doc/graphics/
javaologo.png" />
```

Renderizado em HTML

```

```



Figura 4.8
`<h:graphicImage>`
– JSF, HTML e texto
renderizado.

Biblioteca de recursos

As imagens podem estar armazenadas em diretórios específicos chamados de bibliotecas. Uma biblioteca de imagens deve ser um subdiretório dentro do diretório `“/resources”`, que se situa na raiz da aplicação. Para adicionar uma imagem de uma biblioteca, usa-se o atributo `library` e o nome da imagem deve ser configurado com o atributo `name`. Por exemplo:

```
<h:graphicImage library="fotos" name="foto.jpg" />
```

No exemplo, a imagem `foto.jpg` é obtida de dentro da biblioteca `fotos` (diretório `“/resources/fotos”`). A figura 4.9 mostra como uma biblioteca de imagens fica no sistema de arquivos. No caso, é mostrado o diretório do projeto do Netbeans. A biblioteca se chama `tema1` e a imagem é `img/tux.jpg`, e poderia ser carregada com o seguinte comando:

```
<h:graphicImage library="tema1" name="img/tux.jpg" />
```

Nome	Tamanho
▶ build	3 itens
▶ dist	1 item
▶ nbproject	6 itens
▶ src	2 itens
▼ web	4 itens
▶ META-INF	1 item
▼ resources	1 item
▼ tema1	1 item
▼ img	1 item
tux.jpg	8,0 kB
▶ WEB-INF	1 item
ABC index.xhtml	413 bytes
build.xml	3,4 kB

Figura 4.9
Biblioteca de
Imagens.

A estrutura de bibliotecas também pode ser usada para carregar Scripts personalizados e estilos CSS, usando-se as tags `<h:outputScript>` e `<h:outputStyleSheet>`, respectivamente, desde que dentro do diretório `tema1` sejam criados os diretórios `js` e `css`. Segue um exemplo:

```
<h:outputScript library="tema1" name="js/func.js" />
<h:outputStyleSheet library="tema1"
name="css/estilo.css" />
```

Versionamento de Recursos

É possível armazenar diferentes versões de bibliotecas através de um padrão numérico em dois níveis no formato `DIGITOS_DIGITOS`, conforme o seguinte exemplo:

```
1_0 ou 2_1
```

O JSF sempre referencia a versão mais atual (maior) para acessar os recursos. Esse versionamento é opcional, bastando omiti-lo, como no exemplo anterior (`tema1`). Na imagem a seguir temos a biblioteca default (versões `1_0` e `2_0`) e `tema1` (sem versionamento).

Nome	Tamanho
▶ build	3 itens
▶ dist	1 item
▶ nbproject	6 itens
▶ src	2 itens
▼ web	4 itens
▶ META-INF	1 item
▼ resources	2 itens
▼ default	2 itens
▼ 1_0	1 item
▼ img	1 item
tux.png	49,1 kB
▼ 2_0	1 item
▼ img	1 item
tux.png	85,7 kB
▼ tema1	1 item
▼ img	1 item
tux.jpg	8,0 kB
▶ WEB-INF	1 item
index.xhtml	413 bytes
build.xml	3,4 kB

Figura 4.10
Biblioteca de
Imagens com
versões.

Para acessar a biblioteca de nome default, usamos:

- `<h:graphicImage library="default" name="img/tux.png" />`
- Os recursos da versão `2_0` serão mostrados;
- Não é possível escolher a versão dos recursos a ser utilizado, sendo que o JSF sempre utiliza a última versão.



JavaScript e CSS

Scripts em JavaScript e CSS adicionais (leiautes) podem ser adicionados usando as tags comuns: `<script>` e `<link>`. O ideal é usar o mecanismo de bibliotecas (o mesmo mostrado para imagens) e, neste caso, usamos as seguintes tags do JSF:

`<h:outputScript>`: inclui um JS e usa o mesmo esquema de bibliotecas:

- Atributo `name`: nome do script sendo adicionado;
- Atributo `library`: nome da biblioteca (subdiretório) onde o script se encontra;
- Atributo `target`: onde o script deve ser colocado: `head` ou `body`.

`<h:outputStyleSheet>`: inclui um CSS, usa o mesmo esquema de bibliotecas:

- Atributo `name`: nome do CSS sendo adicionado;
- Atributo `library`: nome da biblioteca (subdiretório) onde o CSS se encontra.

Por exemplo:

```
<h:outputScript name="teste.js" library="js"
               target="head" />
<h:outputStyleSheet name="estilo.css"
                   library="css" />
```

Atributo Rendered

Todos os componentes visuais do JSF possuem o atributo booleano `rendered`. Seu objetivo é fazer a renderização condicional, isto é, se o seu valor for verdadeiro (`true`), o componente é mostrado; caso contrário, não é mostrado. Isso evita a proliferação de comandos condicionais, comuns no desenvolvimento com JSTL.

Todos os componentes do JSF possuem o atributo booleano `rendered`:

- Usado para dizer se o componente deve ou não ser apresentado na fase de `Render Response`;
- Seu valor default é `true`.

Exemplo:

```
<h:outputFormat value="Aluno: {0} Média: {1}"
               rendered="#{alunoMB.mostrarAluno}">
  <f:param value="#{alunoMB.nome}" />
  <f:param value="#{alunoMB.media}" />
</h:outputFormat>
```

Dentro do atributo `rendered`, também podemos colocar uma expressão complexa, por exemplo, queremos mostrar um determinado `<h:outputText>` caso:

```
(alunoMB.media >= 7.0 e alunoMB.mostrarAluno) ou (alunoMB.media < 7.0)
```

Assim, pode-se escrever:

```
<h:outputText value="01"
              rendered="#{(alunoMB.media ge 7.0 and alunoMB.mostrarAluno) or alunoMB.media lt
7.0}" />
```

Os códigos a seguir mostram um exemplo para testar o atributo `rendered`.


```

<h:outputText value="01"
               rendered="#{(alunoMB.media ge 7.0 and alunoMB.mostrarAluno) or
(alunoMB.media lt 7.0)}" />
<h:form>
    Media: <h:inputText value="#{alunoMB.media}" /> <br/>
    <h:selectBooleanCheckbox value="#{alunoMB.mostrarAluno}" /> Mostrar? <br/>
    <h:commandButton value="OK" />
</h:form>
@Named(value = "alunoMB")
@RequestScoped
public class AlunoMB {
    private boolean mostrarAluno = false;
    private double media = 0.0;

    // setters/getters
}

```

Componentes de organização

Para composição de Grids usando os componentes visuais, podemos usar dois componentes de organização:

- **<h:panelGrid>**: organiza os elementos em uma grade ou tabela. Atributo columns: se coloca a quantidade de colunas na tabela, cada componente do JSF será colocado em uma coluna. Renderiza uma tabela com os elementos dentro;
- **<h:panelGroup>**: permite que vários componentes sejam tratados visualmente como um só, na tabela.

O código a seguir mostra como um <h:panelGrid> é apresentado.

Código em JSF

```

<h:panelGrid columns="2">
    <h:outputLabel value="Nome: " for="nome" />
    <h:inputText value="João" id="nome"/>

    <h:outputLabel value="Idade: " for="idade" />
    <h:inputText value="18" id="idade"/>
</h:panelGrid>

```

Renderizado em HTML

```

<table><tbody>
<tr><td><label for="nome">Nome: </label></td>
<td><input id="nome" type="text" name="nome" value="João" /></td></tr>
<tr><td><label for="idade">Idade: </label></td>
<td><input id="idade" type="text" name="idade" value="18" /></td></tr>
</tbody></table>

```

Figura 4.11
<h:panelGrid> –
JSF, HTML e texto
renderizado.

Nome:	João
Idade:	18

O código a seguir mostra um exemplo de `<h:panelGroup>`.

```
<h:panelGrid columns="2">
  <h:outputLabel value="Nome: " for="nome" />
  <h:inputText value="#{exemploMB.nome}" id="nome" />
  <h:panelGroup>
    <h:selectBooleanCheckbox />
    <h:outputLabel value="Idade: " for="idade" />
  </h:panelGroup>
  <h:inputText value="#{exemploMB.idade}" id="idade" />
  <h:commandButton value="Enviar" />
</h:panelGrid>
```

A seguir, é mostrado como o `<h:panelGroup>` é renderizado. Sua função é inserir todos os componentes no mesmo `<td>` da tabela gerada pelo `<h:panelGrid>`.

```
<table>
<tbody>
<tr>
<td><label for="j_idt5:nome">Nome: </label></td>
<td><input id="j_idt5:nome" type="text" name="j_idt5:nome" /></td>
</tr>
<tr>
<td><input type="checkbox" name="j_idt5:j_idt9" /><label for="j_idt5:idade">Idade:
</label></td>
<td><input id="j_idt5:idade" type="text" name="j_idt5:idade" value="0" /></td>
</tr>
<tr>
<td><input type="submit" name="j_idt5:j_idt11" value="Enviar" /></td>
</tr>
</tbody>
</table>
```

A figura 4.12 mostra como fica a tabela.

Figura 4.12
`<h:panelGroup>`
renderizado.

Tabelas

O componente para criação de tabelas é o `<h:dataTable>`.

- Principais atributos `<h:dataTable>`:
 - ▣ Atributo `value`: uma lista contendo os elementos da tabela, gera uma linha para cada item da lista;
 - ▣ Atributo `var`: a cada volta do laço, representa um elemento da lista.
- Colunas da tabela são definidas por `<h:column>` e
- Cabeçalhos ou rodapés são definidos por `<f:facet>`.



Os códigos a seguir mostram um exemplo de <h:dataTable> e como a tabela é renderizada em HTML. A figura 4.13 mostra como é apresentada no navegador.

Index.xhtml

```
<h:dataTable value="#{alunoMB.alunos}" var="aluno">
  <h:column>
    #{aluno.nome}
  </h:column>
  <h:column>
    #{aluno.idade}
  </h:column>
  <h:column>
    #{aluno.email}
  </h:column>
</h:dataTable>
```

Managed Bean

```
@Named(value = "alunoMB")
@RequestScoped
public class AlunoMB {
    private List<Aluno> alunos;
    @PostConstruct
    public void init() {
        alunos = new ArrayList<Aluno>();
        Aluno a = new Aluno();
        a.setNome("João");
        a.setIdade(18);
        a.setEmail("joao@joao.com");
        alunos.add(a);
        a = new Aluno();
        a.setNome("Maria");
        a.setIdade(20);
        a.setEmail("maria@maria.com");
        alunos.add(a);
    }
    // setters/getters
}
```

Bean Aluno

```
public class Aluno {
    private String nome;
    private int idade;
    private String email;

    //setters/getters
}
```

Renderizado em HTML

```
<table>
<tbody>
  <tr>
    <td>João</td>
    <td>18</td>
    <td>joao@joao.com</td>
  </tr>
  <tr>
    <td>Maria</td>
    <td>20</td>
    <td>maria@maria.com</td>
  </tr>
</tbody>
</table>
```

João	18	joao@joao.com
Maria	20	maria@maria.com

Figura 4.13
Apresentação
da tabela no
navegador.

Para criar cabeçalhos e rodapés na tabela ou coluna usamos a tag da biblioteca core `<f:facet>`. O seu atributo name determina o tipo e, para tabelas, temos dois: header ou footer. Os códigos a seguir mostram um exemplo de tabela com cabeçalhos e rodapés, e como a renderização fica em HTML. A figura 4.14 mostra como é apresentado no navegador.

```
<h:dataTable value="#{alunoMB.alunos}" var="aluno" border="1">
  <f:facet name="header">Lista de Alunos</f:facet>
  <h:column>
    <f:facet name="header">Nome</f:facet>
    <f:facet name="footer">x</f:facet>
    #{aluno.nome}
  </h:column>
  <h:column>
    <f:facet name="header">Idade</f:facet>
    #{aluno.idade}
  </h:column>
  <h:column>
    <f:facet name="header">E-mail</f:facet>
    #{aluno.email}
  </h:column>
  <f:facet name="footer">Fim da Lista</f:facet>
</h:dataTable>
```

Renderizado em HTML

```
<table border="1">
<thead>
  <tr><th colspan="3" scope="colgroup">Lista de Alunos</th></tr>
  <tr>  <th scope="col">Nome</th><th scope="col">Idade</th>
    <th scope="col">E-mail</th>  </tr>
</thead>
<tfoot>
  <tr>  <td>x</td> <td></td> <td></td>  </tr>
  <tr><td colspan="3">Fim da Lista</td></tr>
</tfoot>
<tbody>
  <tr> <td> João </td>  <td> 18 </td> <td> joao@joao.com </td>
  </tr>
  <tr> <td> Maria </td> <td> 20 </td> <td> maria@maria.com </td>
  </tr>
</tbody>
</table>
```

Figura 4.14
Apresentação
no navegador de
uma tabela com
cabeçalhos e
rodapés.

Lista de Alunos		
Nome	Idade	email
João	18	joao@joao.com
Maria	20	maria@maria.com
X		
Fim da lista		

Mensagens

Pode-se adicionar mensagens no processamento de uma requisição para que sejam mostradas na próxima página a ser exibida (resposta). O código a seguir mostra como adicionar uma mensagem dentro de um Managed Bean:

```
FacesMessage mensagem = new FacesMessage(
    "Aluno Removido");
mensagem.setSeverity(FacesMessage.SEVERITY_ERROR);
FacesContext.getCurrentInstance().addMessage(
    null, mensagem);
```

O método `addMessage()`, que faz a adição da mensagem, tem dois parâmetros:

- **String:** no exemplo está sendo passado `null`. É o ID do componente usando-se a sintaxe de formulário: `"formId:componentId"`;
- **FacesMessage:** a mensagem propriamente dita.

Em um XHTML, para mostrar Mensagens, usamos:

```
<h:messages />
```



A classe `FacesMessage` é a que representa uma mensagem a ser mostrada e possui dois construtores úteis e um método para definir a severidade da mensagem. São eles:

- `FacesMessage(String resumo)`: cria um `FacesMessage` com uma mensagem resumida;
- `FacesMessage(String resumo, String detalhes)`: cria um `FacesMessage` com uma mensagem resumida e com uma mensagem detalhada;
- `setSeverity(FacesMessage.Severity)`: configura a severidade da mensagem, que pode ser:
 - `FacesMessage.SEVERITY_INFO`
 - `FacesMessage.SEVERITY_WARN`
 - `FacesMessage.SEVERITY_ERRO`
 - `FacesMessage.SEVERITY_FATAL`



A tag `<h:messages/>` possui os seguintes atributos principais:

- `showDetail`: `true/false`, se deve ou não mostrar os detalhes da mensagem. Default `false`;
- `showSummary`: `true/false`, se deve ou não mostrar a mensagem resumida. Default `true`;
- `style`, `styleClass`, `errorClass`, `errorStyle`, `fatalClass`, `fatalStyle`, `infoClass`, `infoStyle`, `warnClass`, `warnStyle`: para definir a classe CSS de apresentação da mensagem, em cada uma das severidades.



Os códigos a seguir mostram o XHTML exemplo e o Managed Bean exemplo de uso de mensagens.

XHTML

```
Aqui aparece a mensagem:
<h:messages style="color: red" showDetail="true"/>
<h:form>
    <h:commandButton action="#{mensagemMB.criarMensagem}"
        value="Mensagem"/>
</h:form>
```

Managed Bean

```
@Named(value = "mensagemMB")
@RequestScoped
public class MensagemMB {
    public MensagemMB() {
    }

    public void criarMensagem() {
        FacesMessage mensagem = new FacesMessage("Aluno Removido", "O aluno foi
removido com sucesso");
        mensagem.setSeverity(FacesMessage.SEVERITY_ERROR);
        FacesContext.getCurrentInstance().addMessage(
            null, mensagem);
    }
}
```

Repetição

O JSF também disponibiliza uma tag de repetição para que uma operação possa ser feita mediante uma lista de itens. A tag é `<ui:repeat>`, que se encontra na biblioteca de tags do facelets.

- Para repetir um código sobre uma lista de elementos, usa-se `<ui:repeat>`
 - ▢ Atributo `value`: coleção de elementos a serem varridos;
 - ▢ Atributo `var`: variável que recebe cada um dos elementos em cada passo da iteração.



Exemplo:

XHTML

```
<h:outputText value="Alunos" />
<ul>
  <ui:repeat value="#{alunosMB.alunos}" var="aluno">
    <li>
      <h:outputText value="#{aluno.nome}" />
    </li>
  </ui:repeat>
</ul>
```

Bean Aluno

```
public class Aluno {
    private String nome;
    // setters/getters
}
```

Managed Bean

```
@Named(value = "alunosMB")
@RequestScoped
public class AlunosMB {
    private List<Aluno> alunos;
    @PostConstruct
    public void init() {
        alunos = new ArrayList<>();
        Aluno a = new Aluno();
        a.setNome("João");
        alunos.add(a);
        a = new Aluno();
        a.setNome("Maria");
        alunos.add(a);
        a = new Aluno();
        a.setNome("José");
        alunos.add(a);
    }
    // setters/getters
}
```



5

JSF – Tratamento de dados e eventos

objetivos

Conhecer o tratamento de dados e eventos no JSF com recursos disponíveis e através de customização.

conceitos

Templates; Conversores; Validadores; Eventos; Atributo Immediate.

Páginas e templates

Um dos grandes benefícios em se utilizar o facelets, que é o motor padrão de tratamento de telas em JSF, é o uso de templates. Um template é um modelo de tela que deve ser usado por toda (ou grande parte) da aplicação. Em um template temos as porções fixas, que não mudam conforme muda-se de tela. São, por exemplo, o cabeçalho da aplicação, que é fixo, seguido de partes variáveis que mudam a cada tela da aplicação.

Entre os benefícios dessa abordagem estão o reuso de porções fixas das páginas da aplicação, a prevenção da duplicação de páginas similares e a manutenção de um padrão de aparência entre todas as telas do sistema.

Para o reuso de páginas existem duas alternativas: inclusão de páginas e templates.

- Facelets: engine padrão de tratamento de telas;
- Templates: definição de um modelo de tela (ou parte).
 - ▣ “Esqueleto” para as telas;
 - ▣ Define elementos fixos – Exemplo: imagem de cabeçalho;
 - ▣ Evita a redefinição de páginas similares;
 - ▣ Incentiva o reuso de páginas;
 - ▣ Ajuda a manter um padrão de aparência no sistema como um todo.
- Duas alternativas para reuso: inclusão de páginas e templates.



Inclusão de páginas

Em JSF podemos incluir arquivos XHTML dentro de outros de modo a evitar duplicação e diminuir o esforço de manutenção quando for preciso modificar alguma coisa. Esse recurso é interessante, porque páginas grandes podem ser separadas ou trechos de páginas que são usados em várias partes do sistema podem ser isolados.

- Para inclusão de arquivos XHTML dentro de outros XHTML, usa-se a tag `<ui:include>`;
- O arquivo a ser incluído deve iniciar com `<ui:composition>` (sem o atributo `template`);
- No arquivo que receberá a inclusão, usa-se `<ui:include>`.



No exemplo a seguir, o cabeçalho de uma página é incluído através desse recurso.

A página que possui a imagem de cabeçalho é incluída exatamente no ponto em que a tag `<ui:include>` é colocada.

index.xhtml

```
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:ui="http://java.sun.com/jsf/facelets" >
<h:head><title>JSF</title></h:head>
<h:body>
  <ui:include src="cabecalho.xhtml" />
  <hr/>
  Conteúdo da página
  <hr/>
  <div id="footer" style="text-align: center">Rodapé</div>
</h:body>
</html>
```

cabecalho.xhtml

```
<ui:composition
  xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="http://java.sun.com/jsf/html"
  xmlns:ui="http://java.sun.com/jsf/facelets">

  <div id="header">
    <h:graphicImage library="tema1" name="cabecalho.jpg" />
  </div>
</ui:composition>
```

Templates

Para definir um template em JSF, deve-se escrever um arquivo XHTML que contém a porção fixa do template. Esse arquivo será usado nas demais páginas da aplicação. Para indicar onde a porção dinâmica da aplicação é inserida nesse template, usa-se a tag `<ui:insert>`. Cada tela da aplicação deverá indicar qual template será usado e isso é feito usando a tag `<ui:composition>` e `<ui:define>`, que insere o conteúdo dinâmico em determinado espaço do template.

Templates são arquivos XHTML com tags especiais do facelets:

- **<ui:insert>**: usada no arquivo de template, indica que será feita a troca por um arquivo específico;
- O atributo `name` é usado para nomear os trechos dinâmicos;
- **<ui:define>**: define um conteúdo a ser inserido em um local definido por `<ui:insert>`;
- **<ui:composition>**: define um grupo de elementos que será inserido em um template. O atributo `template` deve ser usado. O conteúdo fora dessa tag é ignorado.



Os códigos a seguir mostram o arquivo de template (template.xhtml) contendo as porções fixas e a posição onde são inseridas as porções dinâmicas das página e uma página da aplicação (index.xhtml) que usa o template.

template.xhtml

```
<?xml version='1.0' encoding='UTF-8' ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/
TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://xmlns.jcp.org/jsf/html"
      xmlns:ui="http://xmlns.jcp.org/jsf/facelets">
<h:head><title>JSF</title></h:head>
<h:body>
  <div id="header">
    <h:graphicImage library="tema1" name="cabecalho.jpg" />
  </div>
  <hr/>
  <ui:insert name="corpo">Conteúdo da tela</ui:insert>
  <hr/>
  <div id="footer" style="text-align: center">Rodapé</div>
</h:body>
</html>
```

Para usar um template:

- Escreve-se os arquivos com elementos dinâmicos;
- Em geral, um arquivo para cada página que usa o template;
- Usa-se a tag <ui:composition> para que o template seja carregado.
 - O atributo template indica qual é o arquivo de template a ser usado.
- Dentro deve-se criar trechos com <ui:define>
 - Esses trechos serão colocados nas posições dinâmicas do template;
 - O local a ser inserido depende do atributo name.



tela.xhtml

```
<?xml version='1.0' encoding='UTF-8' ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/
TR/xhtml1/DTD/xhtml1-transitional.dtd">
<ui:composition template="/template.xhtml"
      xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:ui="http://java.sun.com/jsf/facelets">
  <ui:define name="corpo">
    <h:form>
      <h:outputLabel for="nome" value="Nome: " />
      <h:inputText id="nome" value="#{exemploMB.nome}" />
      <h:commandButton value="Salvar"
        action="#{exemploMB.salvar}" />
    </h:form>
  </ui:define>
</ui:composition>
```



Na definição do template, podemos ter vários pontos de inserção de conteúdo dinâmico e eles são identificados pelo atributo name da tag `<ui:insert>`. Ao criar uma página usando determinado template, deve-se ter atenção na colocação das tags `<ui:define>`, pois cada uma delas representa uma porção dinâmica indicada no template pela tag `<ui:insert>`.



O atributo name usado na tag `<ui:define>` dentro do conteúdo dinâmico deverá casar com algum atributo name da tag `<ui:insert>` dentro do template.

Conversores

Os dados que trafegam de um formulário HTML para uma aplicação web estão sempre em formato de String e, portanto, podem demandar conversão para serem usados. Mais do que isso, pode ser que precisem ser também validados para garantir estejam semanticamente corretos.

Da mesma forma, os dados dentro da aplicação estão em seus formatos originais (exemplo: `java.util.Date`) e precisam ser convertidos em String para serem exibidos em uma página.

Conversão ou Validação:

- Dados que trafegam de um formulário para a aplicação são sempre texto:
 - Na aplicação, deve-se converter para o tipo desejado;
 - Deve-se fazer uma validação para saber se o dado segue o padrão desejado (exemplo: tamanho de uma string).
- Dados que são obtidos da aplicação nem sempre estão no formato certo a ser apresentado:
 - Deve-se converter para apresentar ao usuário.



O JSF já traz diversos conversores padrão, que não precisam ser assinalados na aplicação. Basta que o MB tenha um atributo daquele tipo para que a conversão seja feita automaticamente.

Conversores aplicados Automaticamente:

- `BigDecimal` e `BigInteger`;
- `Boolean` e `boolean`;
- `Byte` e `byte`;
- `Character` e `char`;
- `Double` e `double`;
- `Float` e `float`;
- `Integer` e `int`;
- `Long` e `long`;
- `Short` e `short`.



A seguir, temos um exemplo de conversão automática, onde o texto digitado no componente é automaticamente convertido para inteiro.

XHTML

```
<h:inputText value="#{pessoaMB.idade}" />
```

Managed Bean

```
@Named
@RequestScoped
public class PessoaMB {
    private int idade;
    // setters/getters
}
```



É possível personalizar os conversores através das tags:

- ▣ **<f:convertNumber />**: para personalizar conversão para java.lang.Number;
- ▣ **<f:convertDateTime />**: para personalizar conversão para java.util.Date ou java.sql.Date.

Conversão de números

Tabela 5.1
Atributos da tag
<f:convert
Number />.

Atributo	Descrição
maxFractionDigits	Com quantas casas decimais, no máximo, o número é apresentado
minFractionDigits	Com quantas casas decimais, no mínimo, o número é apresentado
maxIntegerDigits	Número máximo de números em sua porção inteira
minIntegerDigits	Número mínimo de números em sua porção inteira
pattern	Padrão de formatação do número (definido no DecimalFormat)
type	Tipo como o número é apresentado (number, percent, currency)
currencySymbol	Símbolo a ser usado quando o tipo (type) do número for currency

Os códigos a seguir apresentam um primeiro exemplo de conversão de números.

index.xhtml

```
Normal: <h:outputText value="#{exemploMB.decimal}" /><br/>
Convertido: <h:outputText value="#{exemploMB.decimal}" >
    <f:convertNumber maxFractionDigits="2" />
</h:outputText>
<h:form>
    <h:inputText value="#{exemploMB.decimal}" >
        <f:convertNumber maxFractionDigits="4" />
    </h:inputText> <br/>
    <h:commandButton value="Alterar" />
</h:form>
```

ExemploMB

```
@Named
@RequestScoped
public class ExemploMB {
    private double decimal;
    // setters/getters
}
```

Os códigos a seguir apresentam um segundo exemplo de conversão de números. O código é o mesmo, mas foi mudada a conversão do número decimal para currency.

index.xhtml

```
Normal: <h:outputText value="#{exemploMB.decimal}" /><br/>
Convertido: <h:outputText value="#{exemploMB.decimal}" >
    <f:convertNumber type="currency" />
</h:outputText>
<h:form>
    <h:inputText value="#{exemploMB.decimal}" >
```



```

        <f:convertNumber type="currency"/>
    </h:inputText><br/>
    <h:commandButton value="Alterar" />
</h:form>

```

ExemploMB

```

@Named
@RequestScoped
public class ExemploMB {
    private double decimal;
    // setters/getters
}

```

As figuras a seguir apresentam as diferenças entre os dois exemplos, quando entrado com o valor 100,87878787.

Normal: 100.87878787 Convertido: 100,88 100,8788 Alterar	Normal: 100.87878787 Convertido: R\$ 100,88 R\$ 100,88 Alterar
---	---

Figura 5.1
Diferença entre os dois exemplos.

No primeiro exemplo, o valor é convertido com dois dígitos após a vírgula no texto e com quatro dígitos no componente de entrada de dados. No segundo exemplo, o valor foi convertido para currency e, portanto, deve conter a sigla da moeda do país corrente no navegador (no caso, R\$).

Conversão de datas ou horas

A tabela 4.2 mostra os atributos da tag <f:convertDateTime>.

Atributo	Descrição
pattern	Padrão para apresentação da data (definido em SimpleDateFormat) (exemplo: dd/MM/yyyy)
type	Para especificar se a data, a hora ou os dois serão apresentados (date (default), time, both)
dateStyle	Estilo da data a ser apresentada (default, short, medium, long, full)
timeStyle	Estilo de apresentação da hora (default, short, medium, long, full), somente se type for time ou both
Locale	Preferências regionais (do local) a serem utilizadas nas conversões. (exemplo: pt_BR)

Os códigos a seguir mostram um exemplo de uso da conversão de data ou hora.

index.xhtml

```

Normal: <h:outputText value="#{exemploMB.data}" /><br/>
Padrão dd.MM.yy: <h:outputText value="#{exemploMB.data}" >
    <f:convertDateTime pattern="dd.MM.yy" />
</h:outputText><br/>
Estilo Long: <h:outputText value="#{exemploMB.data}" >
    <f:convertDateTime dateStyle="long" />

```

Tabela 5.2
Atributos de
<f:convertDate
Time>.

```

</h:outputText>
<h:form>
    <h:inputText value="#{exemploMB.data}" >
        <f:convertDateTime pattern="dd/MM/yyyy"/>
    </h:inputText> <br/>
    <h:commandButton value="Alterar" />
</h:form>

```

ExemploMB

```

import java.util.Date;
@Named
@RequestScoped
public class ExemploMB {
    private Date data;
    // setters/getters
}

```

A figura a seguir mostra a execução do exemplo anterior, quando a data entrada é 10/01/2015.

Figura 5.2
Exemplo com
data de entrada
10/01/2015.

Conversor personalizado

Se os conversores padrão não forem suficientes, podemos criar seu próprio conversor. Isso é usado quando se quer disponibilizar conversões para objetos que não são os padrões que o JSF já oferece. Devemos seguir os seguintes passos:

1. A classe cujos objetos serão armazenados deve implementar a interface `Serializable` e os métodos `equals()` e `hashCode()`;
2. Criar uma classe que implementa a interface `javax.faces.convert.Converter`;
3. Nesta classe, adicionar a anotação `@FacesConverter`, indicando a classe associada

```
@FacesConverter(forClass=Telefone.class)
```

Assim, o conversor é atribuído automaticamente à classe `Telefone`. Também é possível definir o conversor da seguinte forma:

```
@FacesConverter(value="telefoneConverter")
```

Onde o atributo `value` indica o id do conversor, a ser usado no XHTML.

4. Implementar os métodos: `getAsObject()` e `getAsString()`.
 - ▣ **getAsObject():** transforma uma `STRING` em `OBJETO` (nesse exemplo `Telefone`);
 - ▣ **getAsString():** transforma um `OBJETO` em `STRING`.
5. Se a string recebida em `getAsObject()` não respeita as regras de conversão, deve-se criar uma mensagem de erro e uma exceção.

```

FacesMessage mensagem = new FacesMessage(
    "Telefone Inválido");
mensagem.setSeverity(
    FacesMessage.SEVERITY_ERROR);
throw new ConverterException(mensagem);

```

Os códigos a seguir mostram um exemplo de uso de um conversor customizado e um conversor customizado para a classe Estado.

```

<h:form>
  <h:panelGrid columns="2">
    <h:outputLabel value="Selecionado: "
      for="estadostr" />
    <h:outputText value="#{exemploMB.selecionado.nome}"
      id="selecionadostr" />
    <h:outputLabel value="Estados: " for="estado" />
    <h:selectOneMenu id="estado"
      value="#{exemploMB.selecionado}"
      converter="estadoConverter">
      <f:selectItems value="#{exemploMB.listaEstados}"
        var="estado"
        itemLabel="#{estado.nome}"
        itemValue="#{estado}" />
    </h:selectOneMenu>
    <h:commandButton value="Enviar" />
  </h:panelGrid>
</h:form>

```

Dica: o ambiente de desenvolvimento facilita a criação dessa classe.

```

@FacesConverter(value="estadoConverter", forClass=Estado.class)
public class EstadoConverter implements Converter {
    public Object getAsObject(FacesContext context,
        UIComponent component,
        String value) {
        return Estado.buscar(value);
    }
    public String getAsString(FacesContext context,
        UIComponent component,
        Object value) {
        return ((Estado) value).getSigla();
    }
}

```

Armazenar um objeto em um MB em vez de uma string

Nos exemplos mostrados de entrada de dados em formulário, principalmente para os componentes de seleção, quando o usuário selecionava um elemento de uma lista, sempre era armazenado uma string correspondendo ao elemento.

Mesmo quando a lista era formada por objetos (exemplo: estado), somente a sigla era armazenada. Mas essa não é a situação ideal para um sistema orientado a objetos. O que se quer é que um objeto do tipo Estado (ou aquele que seja o assunto da escolha) seja armazenado, em vez de uma string.

Esse é o uso mais comum para conversores customizados, que será mostrado nos exemplos a seguir:

index.xhtml: página do usuário.

```
<h:form>
  <h:panelGrid columns="2">
    <h:outputLabel value="Selecionado: "
                  for="selecionadostr" />
    <h:outputText value="#{exemploMB.selecionado.nome}"
                  id="selecionadostr" />

    <h:outputLabel value="Estados: " for="estado" />
    <h:selectOneMenu id="estado"
                    value="#{exemploMB.selecionado}"
                    converter="estadoConverter">
      <f:selectItems value="#{exemploMB.listaEstados}"
                    var="estado"
                    itemLabel="#{estado.nome}"
                    itemValue="#{estado}" />
    </h:selectOneMenu>
    <h:commandButton value="Enviar" />
  </h:panelGrid>
</h:form>
```

managedbeans.ExemploMB: MB que contém a lista de estados a serem mostrados e um estado selecionado pelo usuário.

```
package managedbeans;
// imports
@Named
@RequestScoped
public class ExemploMB {
  private Estado selecionado;
  private List<Estado> listaEstados;
  @PostConstruct
  public void init() {
    // busca todos os estados
    listaEstados = Estado.buscarTodos();
  }
  // setters/getters
}
```

beans.Estado : setters/getters, métodos de acesso ao banco de dados, Serializable, hashCode() e equals().

```
package beans;
import java.util.ArrayList;
import java.util.List;
public class Estado {
    private String sigla;
    private String nome;
    public Estado() { }
    public Estado(String s, String n) {
        this.sigla = s;
        this.nome = n;
    }
    // setters/getters
    public boolean equals(Object e) {
        return (this.sigla.equalsIgnoreCase(
            ((Estado)e).getSigla()));
    }
    public int hashCode() {
        return this.sigla.hashCode();
    }
    // simulando uma busca no banco de dados
    public static Estado buscar(String sigla) {
        if (sigla.equals("PR"))
            return new Estado("PR", "Paraná");
        if (sigla.equals("SC"))
            return new Estado("SC", "Santa Catarina");
        if (sigla.equals("RS"))
            return new Estado("RS", "Rio Grande do Sul");
        if (sigla.equals("MG"))
            return new Estado("MG", "Minas Gerais");
        return null;
    }
    // Simulando uma busca no banco de dados
    public static List<Estado> buscarTodos() {
        List<Estado> listaEstados = new
            ArrayList<Estado>();
        Estado e = new Estado();

        e.setSigla("PR");
        e.setNome("Paraná");
        listaEstados.add(e);

        e = new Estado();
        e.setSigla("SC");
        e.setNome("Santa Catarina");
        listaEstados.add(e);

        return listaEstados;
    }
}
```

converter.EstadoConverter: usado para converter:

- String(sigla) > Estado;
- Estado > String.

```
package converter;
// imports
import beans.Estado;
@FacesConverter(value="estadoConverter", forClass=Estado.class)
public class EstadoConverter implements Converter {
    public Object getAsObject(FacesContext context,
                             UIComponent component,
                             String value) {
        return Estado.buscar(value);
    }
    public String getAsString(FacesContext context,
                              UIComponent component,
                              Object value) {
        return ((Estado) value).getSigla();
    }
}
```

Mensagem de erro de conversão

Se o dado preenchido pelo usuário não for adequado, não permitindo a conversão, podemos apresentar mensagem para avisar o usuário do problema ocorrido. Para tal, usa-se <h:message> para mostrar a mensagem específica de um campo da tela:

```
<h:inputText value="#{pessoaMB.idade}" id="idade" />
<h:message for="idade"/>
```

As mensagens possuem uma versão detalhada e uma resumida, que podem ser mostradas através dos atributos showSummary e showDetails, por exemplo:

```
<h:message for="idade" showSummary="true"
           showDetail="false"/>
```

Para apresentar mensagem de todos os campos, usa-se <h:messages />, cujo valor default de showSummary e showDetails é true e false, respectivamente

Para definir uma mensagem de erro de conversão, usa-se o atributo converterMessage nos campos, como por exemplo:

```
<h:inputText value="#{alunoMB.idade}" id="idade"
             converterMessage="Digite um número" />
<h:message for="idade" />
```

Podemos também alterar globalmente as mensagens de conversão, criando-se um arquivo de mensagens contendo todas as mensagens personalizadas. Esse arquivo precisa ser registrado no faces-config.xml.

Arquivo de mensagens:

- Arquivo.properties (mensagens.properties);
- Coloca-se em um pacote dos fontes;
- Conjunto de CHAVE=VALOR.
 - CHAVE: usado para recuperar a mensagem;
 - VALOR: mensagem a ser apresentada.
- Exemplo:

```
javax.faces.converter.NumberConverter.NUMBER=0 valor {0} não é adequado.  
javax.faces.converter.NumberConverter.NUMBER_detail={0} não é número ou é  
inadequado.
```

Para registrar o arquivo de mensagens, adiciona-se uma referência a seu nome no arquivo *faces-config.xml*, omitindo-se o sufixo.properties. Por exemplo:

```
<application>  
  <message-bundle>pacote.mensagens</message-bundle>  
</application>
```

Validadores

Após a conversão, podemos verificar se os valores passados para a aplicação seguem determinadas regras, por exemplo, se a idade entrada pelo usuário foi um número positivo. Essas regras não são verificadas na fase de conversão.

- Regra não é verificada na fase de conversão;
- Atributo required dos campos indica se o mesmo é obrigatório;
- Exemplo:

```
<h:inputText value="#{alunoMB.idade}"  
             id="idade" required="true" />  
<h:message for="idade" />
```

As validações podem ser feitas através das tags:

<f:validateLongRange>: número inteiro entre uma faixa de valores.

```
<h:inputText value="#{alunoMB.idade}"  
             id="idade" required="true" >  
  <f:validateLongRange minimum="18"  
                       maximum="150" />  
</h:inputText>  
<h:message for="idade" />
```

<f:validateDoubleRange>: número real entre uma faixa de valores.

```
<h:inputText value="#{produtoMB.preco}"  
             id="preco" required="true" >  
  <f:validateDoubleRange minimum="10.5"  
                       maximum="450.87" />  
</h:inputText>  
<h:message for="preco" />
```

<f:validateLength>: string com uma determinada quantidade de caracteres.

```
<h:inputText value="#{alunoMB.nome}" id="nome"
    required="true" >
    <f:validateLength minimum="4"
        maximum="12" />
</h:inputText>
<h:message for="nome" />
```

<f:validateRegex>: validar se um texto segue uma expressão regular. No exemplo a seguir, verifica se a string possui de 6 a 20 caracteres e é formado somente por letras.

```
<h:inputText value="#{alunoMB.codigo}"
    id="codigo" required="true" >
    <f:validateRegex pattern="[a-zA-Z]{6,20}" />
</h:inputText>
<h:message for="codigo" />
```

A seguir, exemplo de validação de e-mail com Expressão Regular:

index.xhtml

```
<h:body>
    <h:messages />
    E-mail: <h:outputText value="#{exemploMB.email}"/>
    <h:form>
        <h:inputText value="#{exemploMB.email}" >
            <f:validateRegex pattern="^[_a-z0-9-]+(\\.[_a-z0-9-]+)*@[a-z0-9-]+(\\.[a-z0-9-]+)*(\\.[a-z]{2,6})$" />
        </h:inputText>
        <h:commandButton value="Atualizar"/>
    </h:form>
</h:body>
```

ExemploMB

```
@Named
@RequestScoped
public class ExemploMB {
    private String e-mail;
    // setters/getters
}
```

As mensagens de erro de validação estão contidas nas bibliotecas do JSF, mas podem ser customizadas através da criação de um arquivo.properties. Esse arquivo deve conter o seguinte formato:

```
javax.faces.converter.DateTimeConverter.DATE={2}: ''{0}'' não parece ser uma data.
javax.faces.converter.DateTimeConverter.DATE_detail=Formato de data inválido.
javax.faces.validator.LengthValidator.MINIMUM=''{0}'' não tem o tamanho mínimo
requerido.
```

Onde do lado esquerdo estão as validações feitas e do lado direito os textos a serem apresentados. Esse arquivo também deve ser registrado no `faces-config.xml`, da seguinte forma:

```
<application>
  <message-bundle>
    com.app.mensagens
  </message-bundle>
</application>
```

Onde `com.app.mensagens` é o nome do arquivo *mensagens.properties*, que está no pacote `com.app`.

Bean Validation

No JSF 2 foram introduzidas regras de validação nos beans, que são usadas para restringir os valores setados em atributos. Essas regras de validação não são feitas nas telas, mas as complementam.

A preferência de uso entre validações nos beans e no XHTML depende da aplicação. Se a validação deve ser feita a cada `set()` executado no bean, ou se esse bean será usado em vários subsistemas e a validação é requerida, então Bean Validation deve ser usado. Mas se a validação depende mais do formulário, isto é, dependente da visão, então a validação no XHTML deve ser usada.

Para usar Bean Validation, adiciona-se anotações antes dos atributos que recebem a validação. No exemplo a seguir, o atributo `nome` não pode ser nulo:

```
public class Aluno {
    @NotNull
    private String nome;
}
```

Podemos desabilitar a validação por bean de um determinado componente, na tela, usando:

```
<f:validateBean disabled="true" />
```

Por exemplo:

```
<h:inputText value="#{alunoMB.nome}" >
  <f:validateBean disabled="true" />
</h:inputText>
```

As validações definidas são:

- **@AssertFalse**: verifica se possui valor false;
- **@AssertTrue**: verifica se possui valor true;
- **@DecimalMax**: define o valor real máximo:
@DecimalMax(value="300.5")
- **@DecimalMin**: define o valor real mínimo:
@DecimalMin(value="13.8")
- **@Digits**: define a quantidade de dígitos da parte inteira (integer) e da fracionária (fraction): @Digits(integer="3", fraction="2")
- **@Future**: verifica se a data é posterior à data atual;
- **@Past**: verifica se a data é anterior à data atual;





- **@Max:** número inteiro máximo:
@Max(value="200")
- **@Min:** número inteiro mínimo:
@Min(value="10")
- **@NotNull:** obriga o valor a não ser nulo;
- **@Null:** obriga o valor a ser nulo;
- **@Size:** define tamanho mínimo e máximo para Collections, Arrays ou Strings:
@Size(min="2", max="10")
- **@Pattern:** se o atributo segue uma expressão regular:
@Pattern(regexp="\\(\\d{3}\\)\\d{4}-\\d{4}")

Podemos customizar as mensagens de erro diretamente nas anotações, com o atributo `message`. Por exemplo:

```
public class Aluno {
    @NotNull(message="Nome não pode ser nulo")
    private String nome;
}
```

Validador Personalizado

Caso as validações padrão não sejam suficientes para a aplicação, podemos criar um validador customizado. Por exemplo, validação de data inicial e data final e-mail etc. A criação é análoga à criação de um Conversor Customizado, e seguimos os seguintes passos:

Passos para criar Validador Customizado:

1. Criar uma classe que implementa a interface `javax.faces.validator.Validator`;
2. Anotar a classe com `@FacesValidator("com.curso.EmailValidator")`;
3. Sobrescrever o método `validate()`.

Para usar o validador nas tags de tela, usa-se:

```
<h:inputText value="#{alunoMB.email}" >
    <f:validator
        validatorId="com.curso.EmailValidator" />
</h:inputText>
```

Se a validação resultar em erro, cria-se uma mensagem e lança-se uma exceção, como no seguinte exemplo:

```
FacesMessage mensagem = new FacesMessage(
    "Email Inválido");
mensagem.setSeverity(FacesMessage.SEVERITY_ERROR);
throw new ValidatorException(mensagem);
```

Se forem necessárias mais informações para a validação, passa-se parâmetros do campo com `<f:attribute />`:

```
<h:inputText value="#{alunoMB.data}" >
    <f:attribute name="inicio" value="10/10/2010" />
    <f:attribute name="fim" value="11/11/2011" />
</h:inputText>
```



No método `validate()` pegamos esses valores usando:

```
String strInicio = (String)component.getAttributes()
    .get("inicio");
String strFim = (String)component.getAttributes()
    .get("fim");
```

Os códigos a seguir mostram um exemplo de validador customizado.

```
<h:outputLabel for="email" value="E-mail:" />
<h:inputText id="email"
    value="#{pessoaMB.pessoa.email}"
    required="true"
    requiredMessage="E-mail é obrigatório">
    <f:validator validatorId="emailValidator"/>
</h:inputText>
```

```
@FacesValidator("emailValidator")
public class EmailValidator implements Validator {
    public void validate(FacesContext context,
        UIComponent component, Object value)
        throws ValidatorException {
        String email = (String)value;
        Pattern p = Pattern.compile(
            "^([\\w-]+(\\.([\\w-]+)*)@([\\w-]+\\.)+[a-zA-Z]{2,7})$");
        Matcher m = p.matcher(email);
        if (!m.find()) {
            FacesMessage fm = new FacesMessage("E-mail inválido.");
            throw new ValidatorException(fm);
        }
    }
}
```

Eventos

Aplicações JSF são baseadas em eventos:

- Eventos gerados pelos usuários;
- Eventos gerados pelo próprio JSF.

Três categorias de eventos podem ser observados e tratados:

- **FacesEvent**: principalmente eventos de controles;
- **PhaseEvent**: interceptam as diversas fases do JSF;
- **SystemEvent**: eventos em outros pontos, que não são cobertos pelos **PhaseEvents**.

A seguir, será detalhada apenas a categoria **FacesEvent**. As demais categorias podem ter seus detalhes consultados no material adicional do AVA.

FacesEvent

Os eventos do tipo FacesEvent são:

- **ActionEvent**: ocorre quando um botão ou link é pressionado;
 - `<h:commandButton>`
 - `<h:commandLink>`
- **ValueChangeEvent**: ocorre quando um valor de uma caixa de texto ou de uma caixa de seleção é alterado.

Botões ou links, ao serem clicados, automaticamente disparam o submit do formulário. Podemos associar um método de um MB, que será invocado quando esse evento ocorrer. Para isso, usa-se os atributos `action` ou `actionListener` dos componentes `<h:commandButton>` e `<h:commandLink>` para atribuir os métodos.

O atributo `action` é associado a um método público de um MB que retorna `void` ou `String`:

- Se retornar `String`, o resultado indica a navegação entre telas;
- Se retornar `void` (ou se o método for `String` e retornar `null`), permanece na mesma tela.

Por exemplo, no XHTML:

```
<h:commandButton value="Inserir"
    action="#{alunoMB.salvar}" />
```

E no Managed Bean:

```
@ManagedBean
public class AlunoMB {
    public String salvar() {
        ""
        return "tela1";
    }
}
```

Atributo `actionListener`:

- Associado a um método público de um MB que retorna `void`;
- O método recebe como parâmetro um `ActionEvent`.

Obtém-se o componente que gerou o evento com:

```
UICommand c = (UICommand)e.getComponent();
```

Por exemplo, no XHTML:

```
<h:commandButton value="Inserir"
    actionListener="#{alunoMB.mudar}" />
```

E no Managed Bean:

```
@Named
@RequestScoped
public class AlunoMB {
    public void mudar(ActionEvent e) {
        UICommand c = (UICommand)e.getComponent();
        c.setValue("Alterado");
    }
}
```

Os atributos `action` e `actionListener` permitem que no máximo dois métodos sejam invocados. Se for necessário mais, deve-se ter uma classe que implementa a interface `ActionListener`. Essa classe deve implementar o método `processAction()` e, para invocá-lo, usa-se `<f:actionListener>` no `commandButton` ou `commandLink`. Por exemplo, no XHTML:

```
<h:commandLink value="Enviar" action="..."
               actionListener="...">
  <f:actionListener type="com.curso.Alterar" />
</h:commandLink>
```

E no listener fica:

```
public class Alterar implements ActionListener {
    public void processAction(ActionEvent e) {
        UICommand c = (UICommand)e.getComponent();
        c.getAttributes().put("style", "color: red");
    }
}
```

Os métodos associados a um evento de ação são executados na seguinte ordem:

1. Método associado com o atributo `actionListener`;
2. Métodos associados com `<f:actionListener>`, de acordo com a ordem em que aparecem no XHTML;
3. Método associado com o atributo `action`.

Esses métodos são chamados na fase `Invoke Application`, como pode ser observado na figura a seguir.

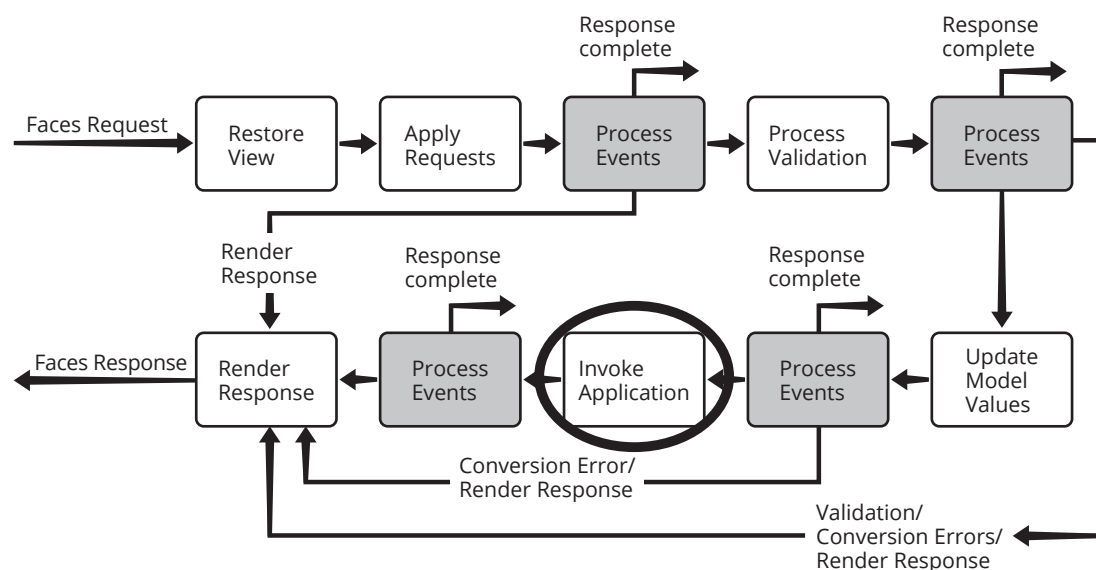


Figura 5.3
FacesEvent:
action Event.

ValueChangeEvent

Esse evento ocorre quando uma caixa de texto ou seleção são alterados. Podemos associar um método de um MB, que será invocado quando o evento ocorrer. Os métodos devem ser públicos e receber um parâmetro do tipo `ValueChangeEvent`. Usa-se o atributo `valueChangeListener` dos componentes ou a tag `<f:valueChangeListener>`.

Por exemplo, no XHTML:

```
<h:outputLabel value="Preço: " for="preco" />
<h:inputText valueChangeListener=
    "#{produtoMB.mudarPreco}"
    id="preco" />
```

E no Managed Bean:

```
@Named
@RequestScoped
public class ProdutoMB {
    public void mudarPreco(ValueChangeEvent e) {
        System.out.println("Antigo: " +
            e.getOldValue());
        System.out.println("Novo: " +
            e.getNewValue());
    }
}
```

Usando a tag <f:valueChangeListener>, podemos apontar para uma classe que implementa interface ValueChangeListener, que implementa o método processValueChange.

Por exemplo, no XHTML:

```
<h:outputLabel value="Preço: " for="preco" />
<h:inputText id="preco" >
    <f:valueChangeListener type=
        "com.curso.AlteracaoPreco" />
</h:inputText>
```

E o Listener:

```
public class AlteracaoPreco
    implements ValueChangeListener {
    public void processValueChange(
        ValueChangeEvent e) {
        System.out.println("Antigo: " +
            e.getOldValue());
        System.out.println("Novo: " +
            e.getNewValue());
    }
}
```

Os métodos são executados na seguinte ordem:

- Método associado com o atributo valueChangeListener;
- Métodos associados com a tag <f:valueChangeListener>, na ordem em que aparecem no XHTML.

Esses métodos são executados na fase Process Validations, conforme pode ser observado na figura a seguir.

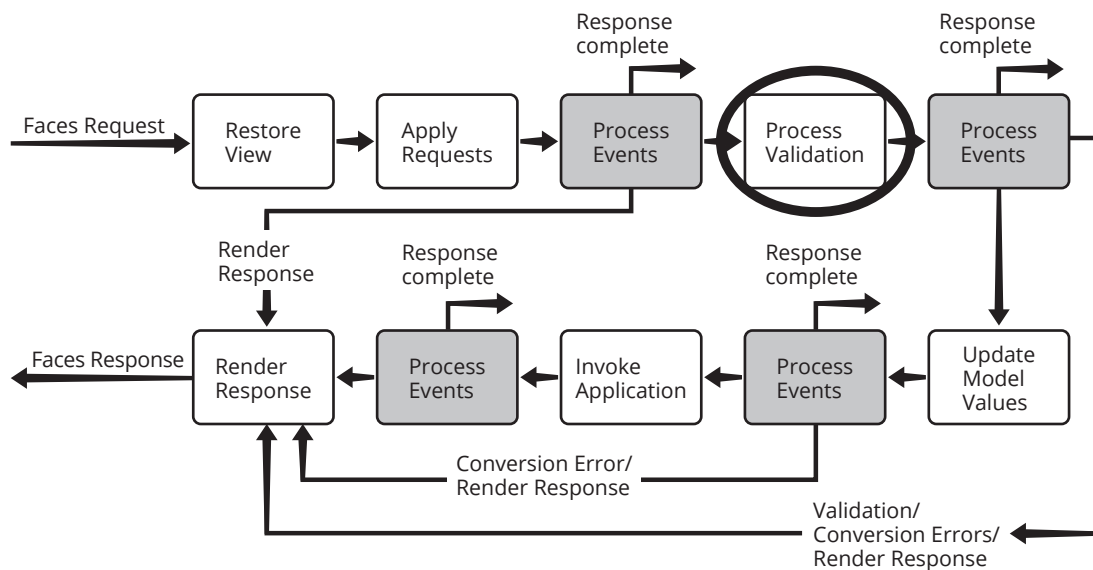


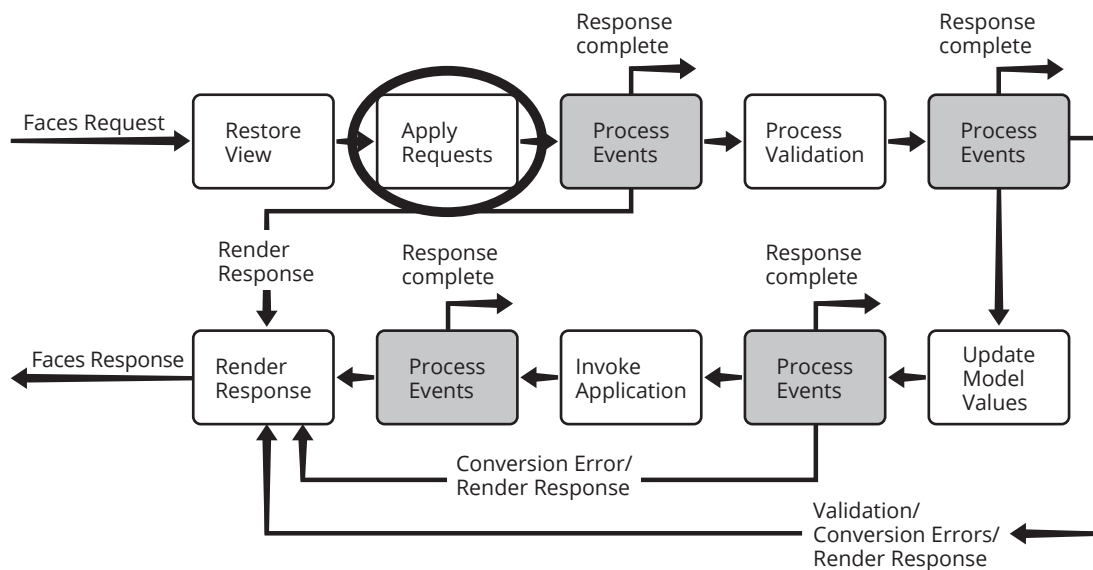
Figura 5.4
facesEvent:
valueChangeEvent.

Atributo immediate

Praticamente todos os componentes visuais possuem esse atributo. Por padrão (sem nada ou `immediate="false"`):

- A validação dos dados de um componente de entrada (exemplo: `<h:inputText>`) ocorre na fase `Process Validations`;
- Eventos de mudança de valor (`ValueChangeEvent`) também acontecem na fase `Process Validation`, conforme figura a seguir;
- Eventos de ação (`ActionEvent`: `<h:commandButton>` e `<h:commandLink>`) ocorrem no final da fase `Invoke Application`.

Figura 5.5
Com `immediate="true"` eventos acontecem na na fase `Apply Request`



O atributo `immediate` pode alterar esse comportamento. Se `immediate="true"`:

- Conversão e Validação ocorrem na fase `Apply Request Values`;
- Eventos de mudança de valor também ocorrem nessa fase;
- Botões e Links: eventos de ação ocorrem no final da fase `Apply Request Values`.

A seguir, alguns exemplos de uso:

- Fazer um `commandLink` ou `commandButton` navegar para outra página sem processar qualquer dado que esteja nos campos de entrada. Exemplo: botão de Cancelamento, wizard;
- Fazer um `commandLink` ou `commandButton` disparar um código da aplicação (exemplo: preencher algum dado da tela), sendo que para isso ainda não se faz necessário (ou não se pode) validar os dados de entrada. Exemplo: combo categoria que, quando selecionado, carrega as subcategorias;
- Fazer um ou mais campos de entrada terem sua validação priorizada, sendo feita antes dos demais, reduzindo o número de mensagens de erro mostradas.



6

JSF – Internacionalização, AJAX e Primefaces

objetivos

Conhecer conceitos de internacionalização; Aprender sobre o uso de Javascript com XML de forma assíncrona; Entender biblioteca Primefaces.

Managed Bean de Internacionalização, requisições, eventos e componentes em Ajax; Componentes ricos, tais como layout de páginas, datatable, picklist, growl e outros.

conceitos

Internacionalização

Internacionalização é a capacidade de uma aplicação se adaptar ao idioma e padrões de exibição de data, hora etc. de diferentes localidades. Em geral, isso afeta a forma de exibir textos (principalmente dos componentes de tela) e outros símbolos de acordo com um padrão pré-definido ou escolhido pelo usuário.

Mecanismos do JSF para internacionalização:

- Arquivos de mensagens em vários idiomas;
- XHTML preparado para apresentar mensagens internacionalizadas;
- Managed Bean na sessão que contém a configuração atual do idioma.

A seguir, cada um desses mecanismos será examinado com mais detalhes.

Arquivos de mensagens

Os arquivos de mensagens são arquivos .properties que residem em algum pacote do código fonte da aplicação.

- Pacote: com.cursojava.messages
- Arquivos:
 - messages_pt_BR.properties: Português (pt) do Brasil (BR);
 - messages_en_US.properties: Inglês (en) dos Estados Unidos (US).

Esses arquivos possuem um conjunto de itens do tipo chave/valor, onde chave é o nome da string a ser usada nos arquivos XHTML e o valor é a string a ser mostrada naquele idioma. A tabela 6.1 mostra os arquivos messages_pt_BR.properties e messages_en_US.properties, com as strings para o português e inglês.



Arquivo messages_pt_BR.properties	Arquivo messages_en_US.properties
portuguese=Português	portuguese=Portuguese
english=Inglês	english=English
save=Salvar	save=Save
delete=Excluir	Delete=Delete
name=Nome	name=Name
title=Formulário	title=Form

Tabela 6.1
Arquivos de mensagens em português e inglês.

Para que esses arquivos de mensagens estejam disponíveis na aplicação configura-se o arquivo faces-config.xml para indicar:

- Qual é o idioma padrão (default);
- Quais são os idiomas disponíveis;
- Qual variável será usada para apresentar as mensagens na tela.

A figura 6.1, a seguir, apresenta as configurações necessárias no *faces-config.xml* para disponibilizar a internacionalização na aplicação.

```
<application>
  <resource-bundle>
    <base-name>
      com.cursojava.messages.messages
    </base-name>
    <var>msgs</var>
  </resource-bundle>
  <locale-config>
    <default-locale>pt_BR</default-locale>
    <supported-locale>en_US</supported-locale>
  </locale-config>
  <message-bundle>
    com.cursojava.messages.messages
  </message-bundle>
</application>
```

Figura 6.1
Configurações do arquivo faces-config.xml para internacionalização.

Managed Bean de internacionalização

O mecanismo de utilizar um MB para internacionalização exige que este tenha escopo da seção, para que todas as páginas e componentes da aplicação possam referenciá-lo.

A seguir, figura 6.2 mostra como um MB pode ser usado para internacionalização de aplicações. Este MB possui um atributo `currentLocale`, que guarda a configuração atual. Possui também dois métodos, (`englishLocale()`) e (`portugueseLocale()`), usados para alterar o idioma em tempo de execução.


```

@Named
@SessionScoped
public class LocaleMB {
    private Locale currentLocale = new Locale("pt", "BR");
    public void englishLocale() {
        UIViewRoot viewRoot = FacesContext.getCurrentInstance().
                                                    getViewRoot();

        currentLocale = Locale.US;
        viewRoot.setLocale(currentLocale);
    }
    public void portugueseLocale() {
        UIViewRoot viewRoot = FacesContext.getCurrentInstance().
                                                    getViewRoot();

        currentLocale = new Locale("pt", "BR");
        viewRoot.setLocale(currentLocale);
    }
    public Locale getCurrentLocale() {
        return currentLocale;
    }
}

```

Figura 6.2
Configurações do
arquivo faces-
config.xml para
internacionalização.

XHTML com Internacionalização

Todos os XHTML que forem usar o recurso da internacionalização precisam ser preparados para tal. São dois pontos a serem observados:

- Usar <f:view> para indicar a internacionalização e qual o idioma padrão. Por exemplo:


```
<f:view locale="#{localeMB.currentLocale}">
```
- Todas as mensagens a serem apresentadas devem estar no arquivo de mensagens configurado no *faces-config.xml*. Por exemplo:


```
<h:outputText value="#{msgs.name}" />
```

O código apresentado na figura 6.3, a seguir, mostra um XHTML preparado para internacionalização contendo dois botões que, ao serem pressionados, alteram o idioma em que a aplicação deve ser mostrada.

```

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core">
  <f:view locale="#{localeMB.currentLocale}">
    <h:head>
      <title>#{msgs.title}</title>
    </h:head>
    <h:body>
      <h:form>
        <h:outputText value="#{msgs.name}" />
        <h:inputText id="nome" /> <br/>
        <h:commandLink value="English"
          action="#{localeMB.englishLocale()}"
        /><br/>
        <h:commandLink value="Português"
          action="#{localeMB.portugueseLocale()}"
        />
      </h:form>
    </h:body>
  </f:view>
</html>

```

Figura 6.3
Configurações
do arquivo
faces-config.xml
para
internacionalização.

AJAX

AJAX significa Asynchronous Javascript And XML. Traduz-se em uma maneira de fazer requisições na internet através de Javascript, de forma assíncrona, sem que a página toda precise ser recarregada. As grandes vantagens no seu uso estão na atualização de trechos da página sem recarregar toda a tela e na realização de requisições sem interromper a navegação dos usuários.

Asynchronous Javascript And XML. Uso:

- ▀ Atualização de trechos da página sem recarregar toda a tela.
- ▀ Realização de requisições sem interromper a navegação dos usuários.

Usando AJAX com JSF, podemos definir:

- ▀ O **evento** de um componente para fazer a requisição.
- ▀ Que **componentes** da tela serão avaliados no servidor.
- ▀ Que componentes da tela serão **atualizados** após a requisição.
- ▀ Que **método** do MB será invocado na requisição.

A tag usada para invocar uma requisição AJAX é <f:ajax>, e cada um dos elementos citados pode ser configurado através de atributos específicos.

Eventos

Uma requisição AJAX deve ser invocada através de um evento, geralmente relacionado aos eventos visuais, de tela, invocados pelo usuário.

Exemplo de requisição AJAX:

```
<h:inputText >
  <f:ajax />
</h:inputText>
```

Esse código indica que a requisição AJAX ocorrerá sempre que o conteúdo do inputText for modificado (evento default é onchange).

No exemplo a seguir, explicita-se o evento que dispara a requisição usando o atributo event:

```
<h:inputText >
  <f:ajax event="keyup" />
</h:inputText>
```



Esse código deixa claro que a requisição só será acionada quando o evento onkeyup for disparado.

Uma outra situação é configurar um componente, por exemplo, <h:commandButton>, para realizar algo somente quando ocorrer um evento de ação. No próximo exemplo, a ação do commandButton (invocação do action) será executada via AJAX quando o botão for clicado (onclick, que é o evento default).

```
<h:commandButton action="...">
  <f:ajax />
</h:commandButton>
```

Os eventos default (quando event não é especificado) são:

- Para componentes de Entrada de Dados: valueChange
- Para componentes de Ação: action

O AJAX do JSF Suporta todos os eventos do DOM:

- Os eventos do DOM são todos suportados com os mesmos nomes, mas sem o prefixo "on";
- valueChange: para elementos de entrada de dados, alteração de valor;
- action: para elementos de ação, por exemplo, botões, ação efetuada.

Para agrupar vários componentes, usa-se a tag <f:ajax> ao redor de todos os componentes agrupados.

Exemplo de agrupamento de componentes:

```
<f:ajax >
  <h:inputText />
  <h:inputSecret />
  <h:commandButton value="OK" />
</f:ajax>
```

Nesse caso, os eventos recém-tratados são os defaults:

- inputText e inputSecret – valueChange (onchange);
- commandButton – action (onclick).

Mais informações sobre o DOM: http://en.wikipedia.org/wiki/DOM_events



Componentes

Para determinar quais componentes devem ser enviados e avaliados no servidor, usamos o atributo `execute`. Esse atributo pode conter uma lista com todos os IDs dos campos que serão avaliados (separados por espaços). O default é ser enviado somente o próprio elemento.

Atributo `execute`:

- Determina quais componentes devem ser avaliados no servidor;
- Forma de uso:
 - Lista com todos os IDs dos campos que serão avaliados.
 - Default é o próprio elemento;
 - Podemos usar o padrão de IDs para campos de formulário: `id1:id2`.

O exemplo a seguir mostra o envio de tudo: o formulário, o componente de formulário e todos os seus filhos. Todos serão avaliados no ciclo de vida do JSF, após a requisição AJAX.

```
<h:form id="formulario">
  <h:inputText />
  <h:inputSecret />
  <h:commandButton value="Enviar">
    <f:ajax event="click" execute="formulario" />
  </h:commandButton>
</h:form>
```



Pode-se usar o padrão de id's para campos de formulário: `id1:id2`.
como por exemplo:
`"formId:textId"`



O `execute` indica quais valores (componentes) serão enviados para processamento via AJAX e pode ter os seguintes valores:

- **@all**: todos os componentes da tela serão enviados;
- **@none**: nenhum componente da tela será enviado;
- **@this**: somente o componente que disparou a requisição AJAX será enviado;
- **@form**: todos os componentes do formulário que contém o componente que disparou a requisição AJAX serão enviados.

Também pode ser atribuído com:

- Lista de IDs, separados por espaço;
- Expressão EL, que resulta em uma coleção de Strings contendo os IDs dos elementos a serem processados.



Atualização

Quando se faz uma requisição AJAX, parte da (ou toda) a tela deve ser atualizada. O atributo `render` indica se a recarga é de parte ou de toda a tela.

No exemplo a seguir, é indicado que o texto com `id numero1` e o texto com `id numero2` devem ser recarregados após a chamada AJAX.

```

<h:commandButton value="Processar">
    <f:ajax event="click" render="numero1 numero2"/>
</h:commandButton>
<h:outputText id="numero1"
    value="#{alunoMB.matricula}" />
<h:outputText id="numero2"
    value="#{alunoMB.matricula}" />

```

O render pode receber os seguintes valores:

- **@all**: todos os componentes da tela serão atualizados;
- **@none**: nenhum componente da tela será atualizado;
- **@this**: somente o componente que disparou a requisição AJAX será atualizado;
- **@form**: todos os componentes do formulário que contém o componente que disparou a requisição AJAX serão atualizados.

Também pode ser atribuído com:

- Lista de IDs, separados por espaço;
- Expressão EL que resulta em uma coleção de Strings, contendo os IDs dos elementos a serem processados.



Método Invocado

Através do atributo listener, se define qual método responde a uma requisição AJAX. Esse método será executado na fase Invoke Application e deve ser public void.

No exemplo a seguir, é efetuada a chamada do método salvar() de aluno MB, após o evento onclick do botão, quando são enviados todos os dados do formulário para atualização.

```

<h:commandButton value="Salvar">
    <f:ajax event="click" execute="formulario"
        render="formulario"
        listener="#{alunoMB.salvar}" />
</h:commandButton>

```

Resumo

Atributos de <f:ajax>:

- **Atributo event**: evento para o qual a requisição AJAX deve ser disparada;
- **Atributo execute**: lista com os IDs dos componentes que serão avaliados no servidor;
- **Atributo render**: lista com os IDs dos componentes que serão atualizados após a requisição ser executada;
- **Atributo listener**: método do MB que será executado quando a requisição AJAX ocorrer.



Palavras-chave associadas a grupos de componentes que podem ser usadas nos atributos render e execute:

- **@all**: todos os componentes da tela;
- **@none**: nenhum componente da tela;
- **@this**: componente que disparou a requisição AJAX;
- **@form**: todos os componentes do formulário que contém o componente que disparou a requisição AJAX;
- Lista de IDs, separados por espaço;
- Expressão EL que resulta em uma coleção de Strings.

Por exemplo:

```
<h:form id="formulario">
  <h:inputText />
  <h:inputSecret />
  <h:commandButton value="Enviar">
    <f:ajax event="click" execute="@form" />
  </h:commandButton>
</h:form>
```

Neste exemplo, ao ser pressionado o botão "Enviar", uma requisição Ajax é invocada para o sistema, sendo enviados todos os dados do formulário para serem processados.

Primefaces

O Primefaces é uma Biblioteca de Componentes Ricos completamente integrada ao ciclo de vida do JSF. O próprio JSF já possui todos os componentes básicos do HTML, mas faltam configurações, flexibilidade e características que tornariam o desenvolvimento de uma aplicação mais rápido. Primefaces não é a única biblioteca para JSF, mas é uma das mais utilizadas.

O JSF possui todos os componentes básicos do HTML. Bibliotecas que contêm componentes mais elaborados ou sofisticados:

- **PrimeFaces**: PrimeTek Informatics;
- **RichFaces**: Jboss/Red Hat;
- **IceFaces**: IceSoft.

O Primefaces pode ser encontrado no site <http://primefaces.org>, e é a biblioteca que contém o maior conjunto de componentes entre as bibliotecas mencionadas.

Neste curso, vamos analisar alguns dos componentes disponíveis no Primefaces

Características do Primefaces:

- OpenSource;
- Suporte nativo a AJAX;
- Mais de 100 componentes JSF;
- Baseado em jQuery;
- Suporte a HTML5;
- Site: <http://primefaces.org>.



Um comparativo entre essas bibliotecas pode ser encontrado no AVA (<http://www.master-theboss.com/jboss-web/richfaces/primefaces-vs-richfaces-vs-icefaces>).

É preciso tomar algumas providências para utilizar os componentes disponíveis no Primefaces. Primeiro deve-se adicionar suas bibliotecas ao projeto (no Netbeans, elas já estão integradas ao ambiente e podem ser facilmente adicionadas). É preciso também ajustar o namespace. O XHTML com suporte a Primefaces fica como no exemplo a seguir:

```
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://java.sun.com/jsf/html"
      xmlns:f="http://java.sun.com/jsf/core"
      xmlns:ui="http://java.sun.com/jsf/facelets"
      xmlns:p="http://primefaces.org/ui">
```

São vários componentes disponíveis no Primefaces, os quais destacamos:

- **Layout:** elemento que define um leiaute de página;
- **DataTable:** um componente para criar tabelas de dados que suporta paginação, filtro, ordenação;
- **Calendar:** entrada de datas em forma de calendário;
- **InputMask:** entrada de dados formatados, com máscara;
- **Editor:** entrada de um texto com formatação;
- **PickList:** seleção de elementos usando transferência entre ListBoxes;
- **Gmap:** apresentação de um mapa do Google Maps;
- **Accordion:** elementos mostrados em abas de um accordion;
- **Menu:** definição de menus;
- **ItemMenu:** itens de menu;
 - ▣ **SubMenu:** submenus, agrupando opções;
 - ▣ **MenuBar:** barra de menus;
 - ▣ **MenuButton:** botão de menu, com várias opções;
- **Growl:** mensagens no estilo de notificações.

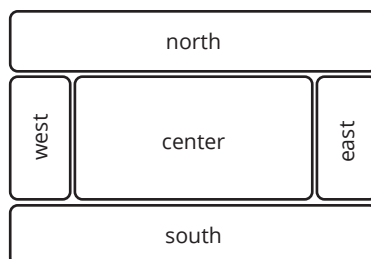
Cada um dos componentes acima listados será apresentado a seguir.

Layout – <p:layout>

O componente de Layout é usado para criar leiautes de páginas. É formado por componentes do tipo <p:layout>, que define um layout na página e que pode usar a página toda (parâmetro fullPage) ou somente parte dela. Também é formado por componentes do tipo <p:layoutUnit>, que define uma porção específica do layout.

Um layout é composto por cinco regiões (unidades – Layout Units): north, west, east, South e center. A figura 6.4 mostra a posição geográfica das regiões de um Layout, enquanto que as tabelas 6.2 e 6.3 apresentam, respectivamente, alguns atributos de <p:layout> e <p:layoutUnit>.

Figura 6.4
Regiões de um
Layout.



Atributo	Descrição
rendered	Se o componente será mostrado
fullPage	(true/false) se o componente ocupará toda a página
style, styleClass	CSS usado no componente
onResize	script JS chamado quando uma Layout Unit é redimensionada
onClose	script JS chamado quando uma Layout Unit é fechada
onToggle	script JS chamado quando uma Layout Unit é escondida/mostrada

Tabela 6.2
Atributos de
<p:layout>.

Atributo	Descrição
header	Texto do cabeçalho da unidade
rendered	Se o componente será mostrado
position	Posição da unidade: north, west, east, south, center
resizable	(true/false) se a unidade é redimensionável
closable	(true/false) se a unidade pode ser fechada
collapsible	(true/false) se a unidade pode ser escondida/mostrada
style, styleClass	CSS usado na unidade de layout

Tabela 6.3
Atributos de
<p:layoutUnit>.

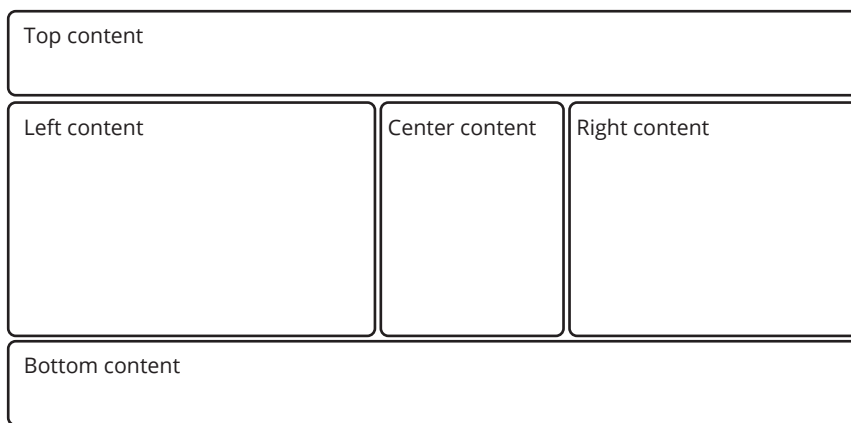
Apresentamos o Exemplo de Layout 1, cujo código fonte pode ser visto a seguir, para em seguida mostrar como este é renderizado, por meio da figura 6.5.

```

<h:body>
<p:layout fullPage="true">
  <p:layoutUnit position="north" size="50">
    <h:outputText value="Top content." />
  </p:layoutUnit>
  <p:layoutUnit position="south" size="100">
    <h:outputText value="Bottom content." />
  </p:layoutUnit>
  <p:layoutUnit position="west" size="300">
    <h:outputText value="Left content" />
  </p:layoutUnit>
  <p:layoutUnit position="east" size="200">
    <h:outputText value="Right Content" />
  </p:layoutUnit>
  <p:layoutUnit position="center">
    <h:outputText value="Center Content" />
  </p:layoutUnit>
</p:layout>
</h:body>

```

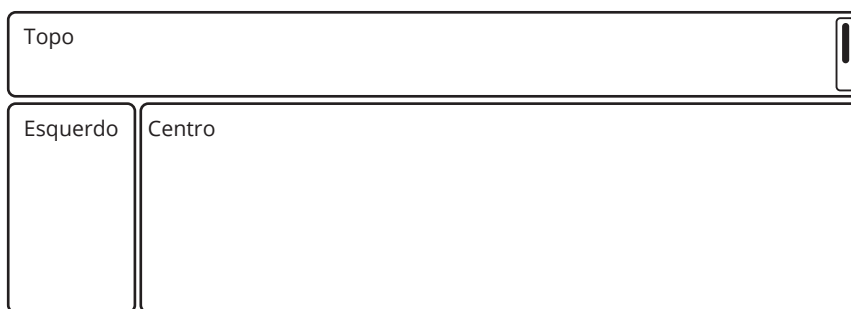

Figura 6.5
Renderização do
Exemplo de
Layout 1.



No Exemplo de Layout 2, podemos ver que não é obrigatória a declaração de todas as regiões, podendo-se criar combinações diferentes, conforme a necessidade do sistema. No código a seguir, vemos uma configuração sem a coluna da esquerda e sem rodapé, com o resultado da renderização mostrado na figura 6.6.

```
<h:body>
<p:layout fullPage="true">
  <p:layoutUnit position="north">
    Topo
  </p:layoutUnit>
  <p:layoutUnit position="west">
    Esquerdo
  </p:layoutUnit>
  <p:layoutUnit position="center">
    Centro
  </p:layoutUnit>
</p:layout>
</h:body>
```

Figura 6.6
Renderização do
Exemplo de
Layout 2 (Parcial).



Finalmente, também é possível programar páginas mais sofisticadas aninhando elementos do tipo layout. Nesse caso, os elementos de layout internos devem conter o atributo `fullPage` como `false`. No código do Exemplo de Layout 3, temos dois layouts aninhados, com a renderização sendo mostrada na figura 6.7.

```

<h:body>
<p:layout fullPage="true">
  <p:layoutUnit position="north">
    Topo
  </p:layoutUnit>
  <p:layoutUnit position="west">
    Esquerdo
  </p:layoutUnit>
  <p:layoutUnit position="center">
    <p:layout fullPage="false">
      <p:layoutUnit position="north">
        Topo 2
      </p:layoutUnit>
      <p:layoutUnit position="west">
        Esquerdo 2
      </p:layoutUnit>
      <p:layoutUnit position="east">
        Direito 2
      </p:layoutUnit>
      <p:layoutUnit position="center">
        Centro 2
      </p:layoutUnit>
      <p:layoutUnit position="south">
        Base 2
      </p:layoutUnit>
    </p:layout>
  </p:layoutUnit>
</p:layout>
</h:body>

```

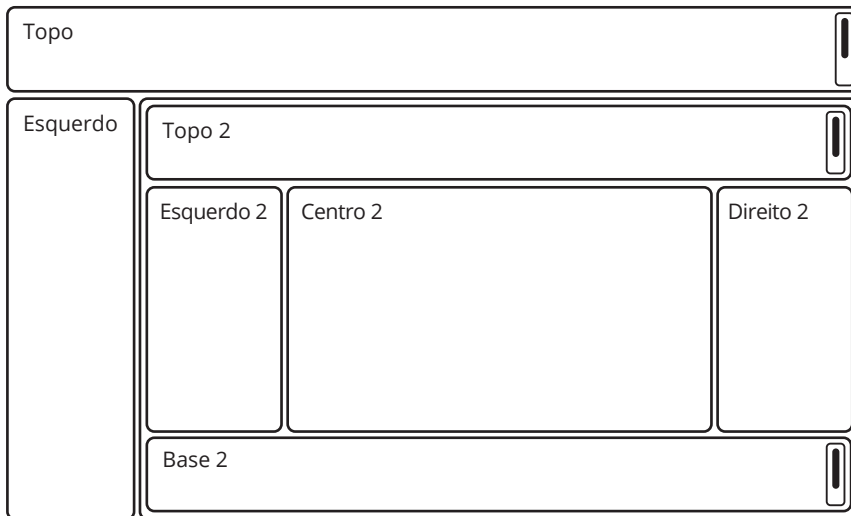


Figura 6.7
Renderização do
Exemplo de Layout 3
(Aninhado).

DataTable – <p:dataTable>

O DataTable é equivalente ao DataTable puro do JSF, mas aumenta suas funcionalidades. Ele apresenta as seguintes características:

- Parecido com o dataTable do JSF;
- Faz paginação entre elementos;
- Faz ordenação de elementos (colunas);
- Permite o uso de Filtros;
- Deve ser colocado dentro de um <h:form>;
- Faz carga preguiçosa de elementos (Lazy Load).
 - Para usar paginação, o MB deve ter escopo maior que da requisição

A tabela 6.4 mostra alguns dos atributos de <p:dataTable>, enquanto que a figura 6.8 mostra como fica, visualmente, um DataTable.

Atributo	Descrição
rendered	Se o componente será mostrado
value	Dados a serem apresentados
var	Nome da variável que receberá os dados na iteração usada para preencher a tabela
rows	Número de linhas a serem mostradas por página
paginator	(true/false) se haverá paginação
style, styleClass	CSS usado na tabela

Tabela 6.4
Atributos de
<p:dataTable>.

Search all fields:			
Model	Year	Manufacturer	Color
		Select v	
9f1e82ad	1989	Volkswagen	Black
c1362b1d	1968	Mercedes	Blue
ec1f0bb1	1962	Renault	Green
9b0b3fe3	2001	Mercedes	Yellow
de0517b3	2002	Volkswagen	Green
3e702116	1972	BMW	Blue
49612134	1994	Ford	Black
bf19778d	1983	Audi	Red
4ecd938b	1962	Opel	Yellow
contains	startsWith	exact	endsWith

Figura 6.8
Exemplo de
DataTable.

O código fonte a seguir exemplifica o uso de <p:dataTable>, com a definição de alguns de seus atributos. Não é o código utilizado para montar a dataTable da figura 6.8.

```

<h:form>
<p:dataTable id="dataTable" var="p" value="#{exemploMB.pessoas}"
    paginator="true" rows="10">
    <f:facet name="header">
        Exemplo
    </f:facet>

    <p:column>
        <f:facet name="header">
            <h:outputText value="Nome" />
        </f:facet>
        <h:outputText value="#{p.nome}" />
    </p:column>

    <p:column>
        <f:facet name="header">
            <h:outputText value="E-mail"/>
        </f:facet>
        <h:outputText value="#{p.email}" />
    </p:column>
</p:dataTable>
</h:form>

```

Calendar – <p:calendar>

O componente <p:calendar> tem por objetivo oferecer um método para a entrada de datas de forma mais prática do que digitá-las em algum formato específico. A tabela 6.5 mostra alguns dos seus atributos, enquanto a figura 6.9 demonstra a aparência desse componente.

Tabela 6.5
Atributos de
<p:calendar>.

Atributo	Descrição
value	A propriedade em que a data escolhida será setada também indica a data inicial em que o calendário estará setado
rendered	Se o calendário será mostrado
mindate	Data mínima mostrada no calendário
maxdate	Data máxima mostrada no calendário
pattern	Padrão das datas, como elas são representadas
showOn	inline: o calendário é renderizado na própria página, ocupando espaço considerável popup: ao clicar no componente (ou imagem), o calendário é apresentado em uma nova janela



Figura 6.9
Exemplo de
Calendar.

O código a seguir traz um exemplo de uso de usando <p:calendar>.

```
<h:form id="form">

    <h3>Inline</h3>
    <p:calendar value="#{exemploMB.date1}"
               mode="inline" id="inlineCal"/>

    <h3>Popup</h3>
    <p:calendar value="#{exemploMB.date2}"
               id="popupCal" />

    <h3>Popup Button</h3>
    <p:calendar value="#{exemploMB.date3}"
               id="popupButtonCal" showOn="button" />

</h:form>
```

Para usar o calendário em português do Brasil, deve-se criar uma localização no Primefaces (usando JavaScript) e adicionar o atributo locale ao <p:calendar>. O trecho a seguir é o código que deve ser inserido para que os textos do calendário fiquem em português.

```
PrimeFaces.locales['pt_BR'] = {
    closeText: 'Fechar',
    prevText: 'Anterior',
    nextText: 'Próximo',
    currentText: 'Começo',
    monthNames: ['Janeiro', 'Fevereiro', 'Março', 'Abril', 'Maio', 'Junho', 'Julho', 'Agosto', 'Setembro', 'Outubro', 'Novembro', 'Dezembro'],
    monthNamesShort: ['Jan', 'Fev', 'Mar', 'Abr', 'Mai', 'Jun', 'Jul', 'Ago', 'Set', 'Out', 'Nov', 'Dez'],
    dayNames: ['Domingo', 'Segunda', 'Terça', 'Quarta', 'Quinta', 'Sexta', 'Sábado'],
    dayNamesShort: ['Dom', 'Seg', 'Ter', 'Qua', 'Qui', 'Sex', 'Sáb'],
    dayNamesMin: ['D', 'S', 'T', 'Q', 'Q', 'S', 'S'],
    weekHeader: 'Semana',
    firstDay: 1,
    isRTL: false,
    showMonthAfterYear: false,
    yearSuffix: '',
    timeOnlyTitle: 'Só Horas',
    timeText: 'Tempo',
    hourText: 'Hora',
    minuteText: 'Minuto',
    secondText: 'Segundo',
    currentText: 'Data Atual',
    ampm: false,
    month: 'Mês',
    week: 'Semana',
    day: 'Dia',
    allDayText: 'Todo Dia'
};
```

E no componente usa-se o atributo locale, conforme a seguir:

```
<h:form id="form">
  <h3>Inline</h3>
  <p:calendar value="#{exemploMB.date1}"
              locale="pt_BR"
              mode="inline" id="inlineCal"/>
</h:form>
```



Não custa lembrar: o texto pt_BR usado no componente é o mesmo que foi setado no JavaScript, na linha PrimeFaces.locales['pt_BR'].

InputMask – <p:inputMask>

O componente <p:inputMask> é usado para a entrada de dados com máscara, ou seja, que possuem uma combinação específica de dígitos, símbolos etc. Exemplos de dados com máscara são um número de telefone ou CEP. A tabela 6.6 a seguir apresenta alguns dos atributos desse componente.

Tabela 6.6
Atributos de
<p:inputMask>.

Atributo	Descrição
rendered	Se o componente será mostrado
mask	A máscara a ser usada no componente, por exemplo: 9999-9999, indicando que deve ser entrado com quatro números, depois tem um hífen e depois mais quatro números
value	O valor do campo, que pode ser ligado a um atributo de um MB

A seguir um exemplo de uso do InputMask.

```
<p:inputMask value="#{exemploMB.data}"
             mask="99/99/9999"/>
```

As máscaras são formadas por caracteres pré-definidos que indicam o padrão permitido.

Caracteres utilizados em máscaras:

- # – indica um número de 0-9
- A – indica um caractere alfanumérico: 0-9, a-Z
- ? – indica um caractere: a-Z

Exemplos:

- Telefone: (##) ####-####
- Placa de carro: ???-####

O código a seguir apresenta um formulário contendo campos com vários tipos de máscaras.

```
<h:form>
  <p:panel header="Masks">
    <h:panelGrid columns="2" cellpadding="5">
      <h:outputText value="Date: " />
      <p:inputMask value="#{maskController.date}"
                  mask="99/99/9999"/>
    </h:panelGrid>
  </p:panel>
</h:form>
```



```

<h:outputText value="Phone: " />
<p:inputMask value="#{maskController.phone}"
             mask="(999) 999-9999"/>

<h:outputText value="Phone with Ext: " />
<p:inputMask
             value="#{maskController.phoneExt}"
             mask="(999) 999-9999? x99999"/>

<h:outputText value="taxId: " />
<p:inputMask value="#{maskController.taxId}"
             mask="99-9999999"/>

<h:outputText value="SSN: " />
<p:inputMask value="#{maskController.ssn}"
             mask="999-99-9999"/>

<h:outputText value="Product Key: " />
<p:inputMask
             value="#{maskController.productKey}"
             mask="a*-999-a999"/>

<p:commandButton value="Reset"
                  type="reset" />
<p:commandButton value="Submit"
                  update="display"
                  oncomplete="PF('dlg').show()"/>

</h:panelGrid>
</p:panel>
</h:form>

```

Editor – <p:editor>

O componente <p:editor> é usado para que o usuário possa entrar textos com formatação rica: negrito, fonte, cores etc.

Atributo	Descrição
rendered	Se o componente será mostrado
value	Texto editado, pode estar ligado a um atributo String de um MB, em format HTML
controls	Lista com os controles que aparecem no componente, por exemplo: bold, italic, underline, color
style, styleClass	CSS usado no componente

Tabela 6.7
Atributos de
<p:editor>.

Um editor é inserido em um formulário através da tag <p:editor>, como no exemplo a seguir.

```

<h:form>
    <p:editor id="editor"
        value="#{editorBean.value}"
        width="600"/>
</h:form>

```

A figura 6.10 mostra como o Editor é exibido na tela, com uma série de ícones na parte superior representando comandos de formatação de texto.

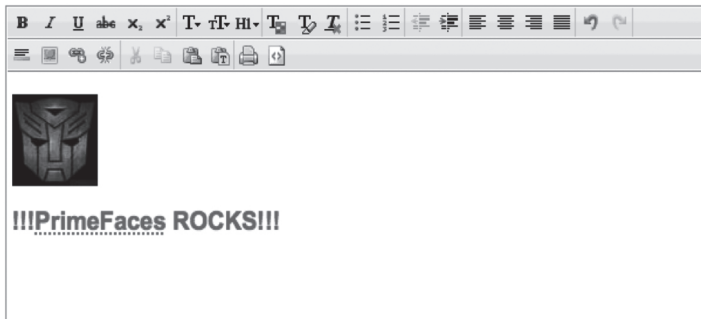


Figura 6.10
Exemplo de Editor.

PickList – <p:pickList>

O PickList é um componente de escolhas, onde são mostradas ao usuário duas listas. A primeira contém os elementos disponíveis. Na segunda estão os elementos que o usuário já escolheu. Também são disponibilizados botões para transferência dos elementos entre as listas e funcionalidades de drag-n-drop.

Um exemplo de PickList é visto a seguir.

```

<p:pickList id="pickList"
    value="#{pickListBean.cities}"
    var="city"
    itemLabel="#{city}"
    itemValue="#{city}" />

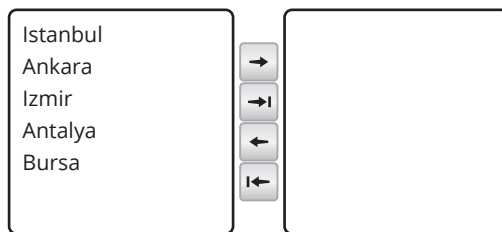
```

A tabela 6.8 apresenta alguns atributos do componente PickList, enquanto que a Figura 6.11 exibe um exemplo do mesmo renderizado.

Tabela 6.8
Atributos de
<p:pickList>.

Atributo	Descrição
rendered	Se o componente será mostrado
value	Valor do componente, com as opções a serem mostradas e a lista de elementos escolhidos. Usa a interface DualListModel.
itemLabel	O texto que será mostrado para cada item
itemValue	O valor que será submetido para cada item
var	Nome da variável de iteração entre todos os elementos a serem mostrados nas listas

Figura 6.11
Renderização de
um <p:pickList>.



A interface `DualListModel` contém os dados das duas listas de um `PickList`. Podemos criá-las dentro do método anotado com `@PostConstruct`.

```
cities = new DualListModel<String>(source, target);
```

Quando o formulário é submetido, essas listas são atualizadas no MB e podem ser acessadas com os métodos:

- **getSource():** obtém a lista de origem dos dados (lista esquerda);
- **getTarget():** obtém a lista de destino (escolha) dos dados (lista direita).

O código a seguir apresenta um exemplo de Managed Bean que instancia uma `DualListModel` e preenche a lista da esquerda com várias strings.

```
@Named
public class PickListBean {
    private DualListModel<String> cities;
    @PostConstruct
    public void init() {
        List<String> source = new ArrayList<String>();
        List<String> target = new ArrayList<String>();
        source.add("Istanbul");
        source.add("Ankara");
        source.add("Izmir");
        source.add("Antalya");
        source.add("Bursa");
        cities = new DualListModel<String>(source,
                                           target);
    }
    public DualListModel<String> getCities() {
        return cities;
    }
    public void setCities(DualListModel<String> cities) {
        this.cities = cities;
    }
}
```

Google Maps – <p:gmap>

O componente <p:gmap> é usado para apresentar uma localização no Google Maps exibido na tela do usuário. Com esse componente é possível mostrar localizações, criar marcas, usar o street view, entre outras facilidades.

A tabela 6.9 apresenta alguns atributos do <p:gmap>, enquanto a que figura 6.12 mostra como este é apresentado ao usuário, assim como trecho de código exemplificando o uso do componente.

Atributo	Descrição
rendered	Se o componente será mostrado
center	Define em que local o mapa estará centralizado
model	Atributo que define a propriedade do MB que receberá a localização
zoom	Define o zoom inicial
streetView	(true/false) se vai ser habilitado o street view neste mapa
type	Tipo do mapa (hybrid, satellite, terrain)
style, styleClass	CSS aplicado ao componente

Tabela 6.9
Atributos do
<p:gmap>.



Figura 6.12
Exemplo de
<p:gmap>.

Para utilizar o Google Maps deve-se adicionar no <h:head> da página o seguinte script:

```
<script type="text/javascript" src="http://maps.google.com/maps/api/
js?sensor=false"></script>
```

O MB que manipula o mapa pode ser implementado como o código abaixo. O atributo que recebe a localização deve ser do tipo MapModel.

```
@Named
@RequestScoped
public class ExemploMB {
    private MapModel localizacao = new DefaultMapModel();

    @PostConstruct
    public void init() {
        localizacao = new DefaultMapModel();
    }

    // setters/getters
}
```

Accordion Panel – <p:accordionPanel>

O Accordion Panel é um componente usado para mostrar diversas informações em formatos de abas, com efeito de acordeão.

Usa-se o componente <p:accordionPanel> para definir o Accordion Panel, e suas abas são definidas com o componente <p:tab>. Os principais atributos de cada um desses componentes são mostrados, respectivamente, nas tabelas 6.10 e 6.11, a seguir.

Tabela 6.10
Atributos de
<p:accordion
de Panel>.

Atributo	Descrição
rendered	Se o componente será mostrado
onTabChange	script (JS) usado para tratar a mudança de abas pelo cliente
style, styleClass	CSS usado no componente

Tabela 6.11
Atributos de
<p:tab>.

Atributo	Descrição
rendered	Se a aba será mostrada
title	Título da aba
tooltip	Texto de ajuda da aba
titleStyle, titleStyleClass	CSS usado para para a aba
disable	(true/false) se a aba está desabilitada
closable	(true/false) torna a aba fechável

A figura 6.13 mostra como um Accordion Panel é apresentado ao usuário.

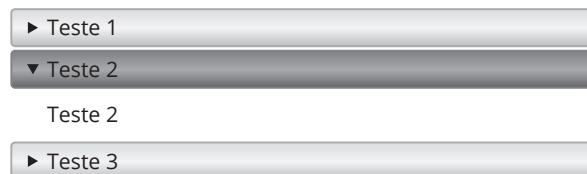


Figura 6.13
Exemplo de
Accordion Panel.

O código a seguir corresponde ao Accordion Panel, apresentado na figura anterior.

```
<p:accordionPanel>
  <p:tab title="Teste 1">
    Teste1
  </p:tab>
  <p:tab title="Teste 2">
    Teste2
  </p:tab>
  <p:tab title="Teste 3">
    Teste3
  </p:tab>
</p:accordionPanel>
```

Menus – <p:menu>

O componente <p:menu> é usado para criar menus de navegação. Os itens de menu são criados com <p:menuItem>. Os atributos de cada um desses componentes são apresentados nas tabelas 6.12 e 6.13 respectivamente.

Atributo	Descrição
rendered	Se o componente será mostrado
style, styleClass	CSS usado no componente

Tabela 6.12
Atributos de
<p:menu>.

Atributo	Descrição
rendered	Se o item será mostrado
value	O texto do item
action, actionListener	Métodos a serem invocados quando o item é pressionado
disabeld	Se o item estará desabilitado
url	URL para onde a aplicação será redirecionada quando o item for pressionado
icon	Imagem do item de menu
style, styleClass	CSS usado no componente

Tabela 6.13
Atributos de
<p:menuItem>.

O código a seguir mostra a criação de um menu com quatro itens, enquanto a figura 6.14 mostra como é exibido.

```
<p:menu>
  <p:menuitem value="Google" url="http://www.google.com" />
  <p:menuitem value="Hotmail" url="http://www.hotmail.com" />
  <p:menuitem value="Youtube" url="http://www.youtube.com" />
  <p:menuitem value="Facebook" url="http://www.facebook.com" />
</p:menu>
```

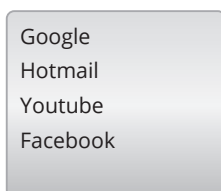


Figura 6.14
Exemplo de
apresentação
de menu e item
de menu.

Já a figura 6.15 mostra como este mesmo menu é apresentado dentro de um componente de Layout.

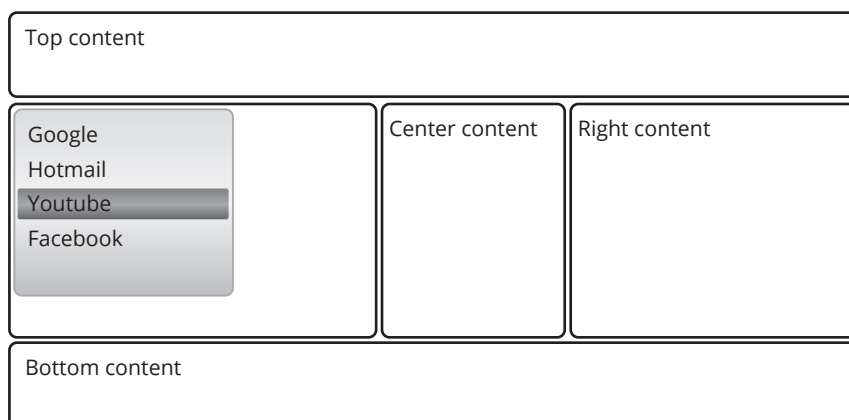


Figura 6.15
menu dentro
de um layout.

Cabe destacar que `<p:menuItem>` pode ser utilizado em diversos outros componentes de menu, tais como `MenuBar` e `MenuButton`, que serão vistos mais à frente. Chamamos atenção também para o atributo `icon`, através do qual podemos adicionar uma imagem a um item de menu.

Exemplos de imagens de itens de menu do Primefaces:

- `ui-icon-disk`  `ui-icon-trash` 
- `ui-icon-print` 
- `ui-icon-search` 

Exemplo de uso:

```
<p:menuItem value="Google"
  url="http://www.google.com"
  icon="ui-icon-search" />
```

Para personalizar uma imagem a ser utilizada em um Item de Menu, devemos:

- Ter um CSS com uma classe, que será usada no item, exemplo, `estilo.css`, com a classe `.teste`;

```
.teste {
  background-image:
    url("#{resource['img/pessoa.gif']}")
  !important;
  background-size: 16px 16px;
}
```

- Ter criado o diretório `resources` contendo: `css` e `img`;
- Dentro de `css`, colocar `estilo.css` que contém o CSS para a imagem;
- Dentro do diretório `img`, colocar a imagem, por exemplo, `pessoa.gif`;
- Adicionar o CSS na página XHTML com `<h:outputStylesheet>`;

```
<h:head>
  <title>Facelet Title</title>
  <h:outputStylesheet name="estilo.css"
    library="css" />
</h:head>
```

- Usar o nome da classe CSS no atributo `icon` de `<p:menuItem>`

```
<p:menuItem value="Gmail"
  url="http://www.google.com"
  icon="teste" />
```

A figura 6.16 mostra um item de menu com imagem personalizada.

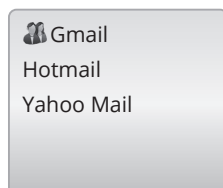


Figura 6.16
Exemplo de Item de
Menu com imagem
personalizada.

SubMenus – <p:submenu>

Para acrescentar agrupamentos ou submenus, usa-se a tag <p:submenu>. A tabela 6.14 apresenta alguns atributos de SubMenu.

Atributo	Descrição
rendered	Se o componente será mostrado
label	O título do submenu
icon	Ícone para o submenu, nos mesmos moldes de menu item
style, styleClass	CSS usado no componente

Tabela 6.14
Atributos de
<p:submenu>.

O código a seguir mostra um exemplo de Menu com Itens de Menu e Submenus. A figura 6.17 mostra como este código é renderizado.

```
<p:menu>
  <p:submenu label="Mail">
    <p:menuitem value="Gmail"
      url="http://www.google.com" />
    <p:menuitem value="Hotmail"
      url="http://www.hotmail.com" />
    <p:menuitem value="Yahoo Mail"
      url="http://mail.yahoo.com" />
  </p:submenu>
  <p:submenu label="Videos">
    <p:menuitem value="Youtube"
      url="http://www.youtube.com" />
    <p:menuitem value="Break"
      url="http://www.break.com" />
    <p:menuitem value="Metacafe"
      url="http://www.metacafe.com" />
  </p:submenu>
  <p:submenu label="Social Networks">
    <p:menuitem value="Facebook"
      url="http://www.facebook.com" />
    <p:menuitem value="Linkedin"
      url="http://www.linkedin.com" />
  </p:submenu>
</p:menu>
```

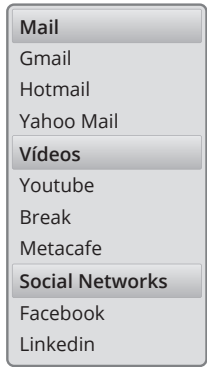


Figura 6.17
Exemplo de Menu
com SubMenu.

Barra de Menus – <p:menubar>

Para a criação de uma barra de menus horizontal, usa-se o componente <p:menubar>. A Tabela 6.15 mostra alguns atributos de MenuBar.

Atributo	Descrição
rendered	Se o componente será mostrado
autoDisplay	(true/false) se o primeiro nível de menus será mostrado automaticamente quando o mouse passa por cima
style, styleClass	CSS usado no componente

Tabela 6.15
Atributos de
MenuBar.

A código a seguir exemplifica a construção de uma barra de menu, com o resultado final sendo apresentado na figura 6.18.

```
<p:menubar>
  <p:submenu label="Mail">
    <p:menuitem value="Gmail"
      url="http://www.google.com" />
    <p:menuitem value="Hotmail"
      url="http://www.hotmail.com" />
    <p:menuitem value="Yahoo Mail"
      url="http://mail.yahoo.com" />
  </p:submenu>
  <p:submenu label="Videos">
    <p:menuitem value="Youtube"
      url="http://www.youtube.com" />
    <p:menuitem value="Vimeo"
      url="http://www.vimeo.com" />
  </p:submenu>
</p:menubar>
```



Figura 6.18
Exemplo de
MenuBar.

Botão de Menus – <p:menuButton>

Para se criar um Botão de Menu, usa-se o componente <p:menuButton>. Dentro, coloca-se componentes do tipo <p:menuItem> para as opções clicáveis. A tabela 6.16 mostra alguns atributos do Botão de Menu.

Atributo	Descrição
rendered	Se o componente será mostrado
value	O texto do botão
disabled	Para desabilitar o botão
style, styleClass	CSS usado no componente

Tabela 6.16
Atributos de
Botão de Menu.

O código a seguir traz como exemplo a construção de um Botão de Menu com três itens, sendo que a figura 6.19 exibe o resultado da renderização desse código.

```
<p:menuButton value="Options">
    <p:menuitem value="Save" url="/home.jsf" />
    <p:menuitem value="Update" url="/home.jsf" />
    <p:menuitem value="Go Home" url="/home.jsf" />
</p:menuButton>
```

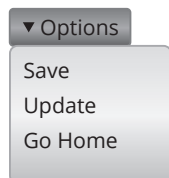


Figura 6.19
Exemplo de
Botão de Menu.

Growl – <p:growl>

O componente <p:growl> é usado para apresentar mensagens em formato de notificações. As mensagens mostradas são as produzidas por FacesMessage e, portanto, funcionam como o <h:messages>.

Podemos também direcionar as mensagens como neste exemplo:

```
<p:messages for="mensagens1" />
<p:growl for="mensagens2" />
```

E no ManagedBean podemos usar:

```
FacesContext context =
    FacesContext.getCurrentInstance();
context.addMessage("mensagens1", facesMessage1);
context.addMessage("mensagens1", facesMessage2);
context.addMessage("mensagens2", facesMessage3);
```

Nesse exemplo, facesMessage1 e facesMessage2 serão apresentados no <p:messages> e facesMessage3 no <p:growl>.

A figura 6.20 mostra como Growl é apresentado.

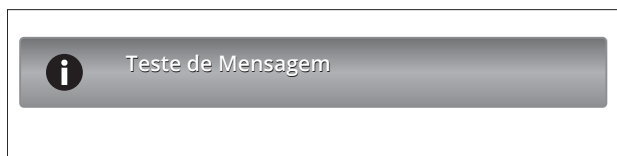


Figura 6.20
Exemplo de Growl.

7

Hibernate

objetivos

Conhecer o Hibernate como ferramenta de mapeamento Objeto/Relacional em Java com JSF.

Mapeamento Objeto/Relacional; Ciclo de Vida de Persistência; Unidade de Trabalho; Contexto de Persistência; Estados, atributos transientes e interfaces do Hibernate.

conceitos

Introdução

Hibernate é uma ferramenta de Mapeamento Objeto/Relacional em Java, isto é, transforma dados das tabelas em um grafo de objetos.

- Ferramenta de Mapeamento Objeto/Relacional em Java;
- Transforma dados das tabelas em um grafo de objetos;
- Mascara o acesso ao banco, facilitando a interface com ele;
- Aumenta a velocidade de desenvolvimento;
- Mascara o uso de SQL;
- Indicado para aplicações com uma modelagem rica de Objetos.



Um de seus principais objetivos é mascarar o acesso ao banco, facilitando a interface com ele, aumentando a velocidade de desenvolvimento, evitando o uso de SQL. Por suas características e funcionalidades, é fortemente indicado para aplicações com uma modelagem rica de Objetos.

Basicamente o que se quer é transformar:

- Um registro de uma tabela no banco de dados em Objeto do sistema; ou
- Objetos do sistema em registros no BD.

Em geral, se pensa em:

- Corresponder cada tabela a uma classe;
- Corresponder cada registro da tabela a um objeto.



Problemas podem ocorrer:

- Herança;
- Objetos com coleção de outros objetos.
- Etc.

A figura 7.1 mostra uma correspondência simples.

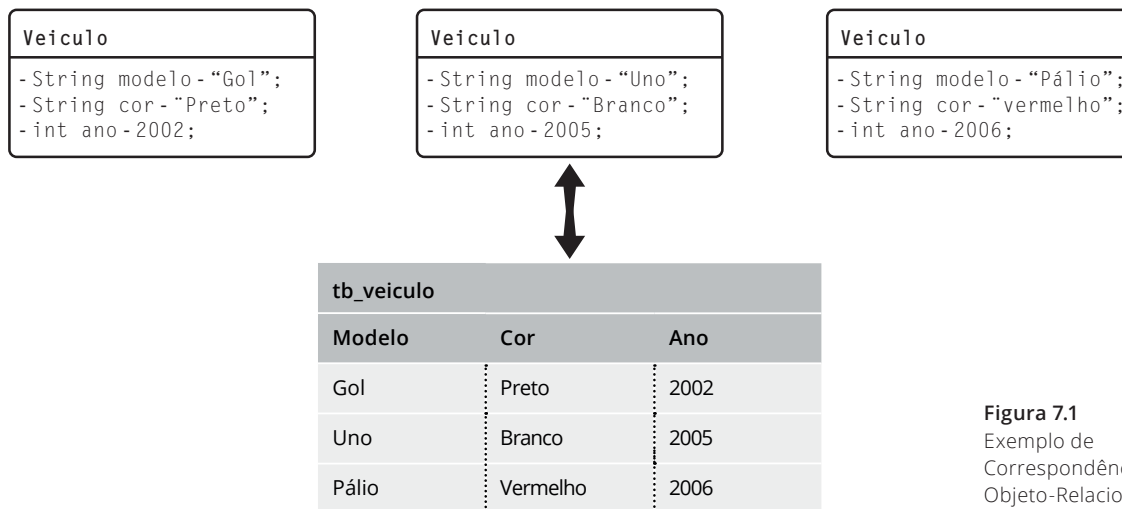


Figura 7.1
Exemplo de
Correspondência
Objeto-Relacional.

Para usar o Hibernate, precisa-se:

- Banco de Dados.
 - Tabelas criadas.
- Arquivo de configuração: `hibernate.cfg.xml`.
 - Contém referência para todas as classes persistentes.
- Hibernate Session Factory.
 - Uma classe utilitária que retorna uma sessão para usar o hibernate.
- Classes Persistentes.
 - Classes que serão mapeadas para tabelas.

Os passos para criação de um projeto Hibernate no Netbeans estão a seguir, adicionando esse Framework na criação do projeto, juntamente com a adição do Framework JSF.

1. Criar o Projeto, adicionando o framework Hibernate;
2. Criar `HibernateUtil.java`;
3. Criar Classe Persistente;
4. Criar Arquivo de Configuração;
5. Classe de Teste (Programa Principal).

Exercício de Fixação

Acompanhar o passo a passo apresentado em sala de aula para criar um primeiro projeto utilizando o Hibernate no Netbeans. O código a seguir será utilizado no exercício e traz um exemplo de classe persistente e uma classe principal, sem usar ainda o JSF.

Os códigos a seguir apresentam um exemplo de classe persistente e um exemplo de classe principal, sem JSF.

```

package model;
import javax.persistence.Entity;
import javax.persistence.Table;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.SequenceGenerator;
@Entity
@Table(name="tb_teste")
@SequenceGenerator(name="sequencia", sequenceName="seq_id")
public class Pessoa {
    private Long id;
    private String nome;
    public Pessoa() {
    }
    @Id
    @GeneratedValue(strategy=GenerationType.SEQUENCE, generator="sequencia")
    public Long getId() {
        return this.id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getNome() {
        return this.nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
}

```

```

import org.hibernate.Session;
import model.Pessoa;
import util.HibernateUtil;
public class Teste {
    public static void main(String[] args) {
        Pessoa p = new Pessoa();
        p.setNome("João");
        Session session = HibernateUtil.
                                sessionFactory().
                                openSession();
        session.beginTransaction();
        session.save(p);
        session.getTransaction().commit();
        System.out.println(p.getNome() +
                                " Inserido com sucesso.");
    }
}

```

Classe persistente

Uma classe persistente deve ser anotada para que seja devidamente utilizada pelo Hibernate.

As principais anotações são:

- **@Entity**: indica que é persistente;
- **@Id**: indica que uma propriedade é Primary Key;
- **@Table**: indica a tabela correspondente;
- **@SequenceGenerator/@GeneratedValue**: indicam atributos autoincrementáveis.
Serão vistos em seção específica no decorrer do material.

O código a seguir implementa a classe persistente Aluno.java.

```
@Entity
@Table (name="tb_aluno")
@SequenceGenerator(name="sequencia", sequenceName="seq_id")
public class Aluno {
    private Long id;
    private String nome;
    public Aluno() {
    }
    @Id
    @GeneratedValue(strategy=GenerationType.SEQUENCE,
                    generator="sequencia")
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getNome() {
        return this.nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
}
```

Essa classe pode estar relacionada à seguinte tabela, definida pelo seu DDL:

```
CREATE TABLE tb_aluno (
    id            integer NOT NULL,
    nome          character varying(30),
    CONSTRAINT pk_id PRIMARY KEY (id)
)
CREATE SEQUENCE seq_id
    INCREMENT 1
    MINVALUE 1
    MAXVALUE 9223372036854775807
    START 10
    CACHE 1;
```

Acesso simples ao banco de dados: inserção e consulta

Para se fazer uma inserção em uma base de dados usando hibernate, deve-se seguir uma sequência simples de passos:

1. Cria-se um objeto (se não existir) de uma classe persistente;
2. Obtém-se a sessão hibernate corrente, ou abre-se uma nova;
3. Cria-se uma transação;
4. Salva-se o objeto;
5. Efetiva-se a transação (commit);
6. Fecha-se a sessão.

O código a seguir mostra os passos para inserção.

```
// Criação do objeto
Aluno aluno = new Aluno();
aluno.setNome(nomeAluno);
// Obter sessão hibernate corrente
Session session = HibernateUtil.getSessionFactory()
    .openSession();

// Inicialização da transação
session.beginTransaction();
// Salvamento do Objeto
session.save(aluno);
// Efetivação da transação
session.getTransaction().commit();
session.close();
```

Para se fazer uma consulta, seguimos os seguintes passos:

1. Obtém-se a sessão hibernate corrente ou abre-se uma nova;
2. Cria-se uma transação;
3. Consulta-se a base via HQL ou Criteria;
4. Efetiva-se a transação (commit);
5. Fecha-se a sessão.

O código a seguir apresenta um exemplo de consulta.

```
// Obtem-se sessão corrente do hibernate
Session session = HibernateUtil.getSessionFactory()
    .openSession();

// Inicia-se a transação
session.beginTransaction();
// Efetua-se a consulta (aqui em HQL)
Query select = session.createQuery("from Aluno");
List objetos = select.list();
// Efetiva-se a transação
session.getTransaction().commit();
session.close();
```

Managed Bean e XHTMLs

Dependendo da arquitetura escolhida para o desenvolvimento do sistema, o acesso às classes anotadas pode ser feito de várias formas. Nesse material será mostrada a maneira mais simples, onde o Managed Bean possui um atributo do tipo da classe persistente e, em métodos específicos para acesso ao banco, essa classe é persistida ou recuperada, usando-se a API do Hibernate.

A seguir, é apresentado o Managed Bean `AlunoMB`, que gerencia os dados das telas.

```
@Named(value = "alunoMB")
@RequestScoped
public class AlunoMB {
    private Aluno aluno;
    private List<Aluno> listaAlunos;
    public AlunoMB() {
    }
    @PostConstruct
    public void init() {
        this.aluno = new Aluno();
    }
}
```

Nesse MB, o método `salvar()` usa a API do Hibernate para salvar a classe persistente `aluno` que já está preenchida no MB.

```
    public String salvar() {

        // salva o aluno
        Session session = HibernateUtil.getSessionFactory()
                                                    .openSession();

        session.beginTransaction();
        session.save(this.aluno);
        session.getTransaction().commit();

        session.beginTransaction();
        // lista todos os alunos
        Query query = session.createQuery("from Aluno");
        this.listaAlunos = query.list();

        session.getTransaction().commit();

        session.close();
        return "listar";
    }
```

O método `listar()` busca todos os alunos da base e preenche a lista `Alunos` para ser mostrada no XHTML e redireciona para `listar.xhtml`.

```

public String listar() {
    Session session = HibernateUtil.getSessionFactory()
                                                .openSession();

    session.beginTransaction();
    // lista todos os alunos
    Query query = session.createQuery("from Aluno");
    this.listaAlunos = query.list();

    session.getTransaction().commit();

    session.close();
    return "listar";
}

// setters/getters
public Aluno getAluno() {
    return aluno;
}

public void setAluno(Aluno aluno) {
    this.aluno = aluno;
}

public List<Aluno> getListaAlunos() {
    return listaAlunos;
}

public void setAluno(List<Aluno> listaAlunos) {
    this.listaAlunos = listaAlunos;
}
}

```

O `indexemplo.xhtml` é um formulário com dois botões. Quando o botão “Salvar” é pressionado, o método `salvar()` é invocado. Quando o botão “Listar” é pressionado, o método `listar()` é invocado. A seguir, o arquivo `indexemplo.xhtml`.

```

<h:body>
    <h:form>
        Nome: <h:inputText value="#{alunoMB.aluno.nome}" />
        <h:commandButton action="#{alunoMB.salvar()}"
                        value="Salvar" />
        <h:commandButton action="#{alunoMB.listar()}"
                        value="Listar" />
    </h:form>
</h:body>

```

A seguir o arquivo `listar.xhtml`, que apresenta todos os dados retornados pelo método `listar()`.

```

<h:body>
    <h:dataTable value="#{alunoMB.listaAlunos}" var="a">
        <f:facet name="header">
            <h:outputText value="LISTA DE ALUNOS" />
        </f:facet>
        <h:column>
            <f:facet name="header">

```

```

        <h:outputText value="ID" />
    </f:facet>
    #{a.id}
</h:column>
<h:column>
    <f:facet name="header" >
        <h:outputText value="NOME" />
    </f:facet>
    #{a.nome}
</h:column>
</h:dataTable>
<h:link outcome="index" value="Início" />
</h:body>

```

Arquivo de Configuração: hibernate.cfg.xml

O arquivo *hibernate.cfg.xml* é o arquivo que contém as configurações de acesso ao banco de dados e informações sobre as classes que são persistentes. É um arquivo XML, onde tags específicas indicam informações como: banco de dados, usuário, senha, dialeto SQL etc.

A seguir, um exemplo de arquivo *hibernate.cfg.xml*, que configura a conexão com um banco de dados.

```

<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate Configuration DTD
3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
<session-factory>
    <property name="hibernate.connection.driver_class">
        org.postgresql.Driver
    </property>
    <property name="hibernate.connection.url">
        jdbc:postgresql://localhost:5432/curso
    </property>
    <property name="hibernate.connection.username">postgres</property>
    <property name="hibernate.connection.password">postgres</property>
    <property name="hibernate.connection.pool_size">1</property>
    <property name="hibernate.dialect">
        org.hibernate.dialect.PostgreSQLDialect
    </property>
    <property name="hibernate.current_session_context_class">
        thread
    </property>
    <property name="hibernate.cache.provider_class">
        org.hibernate.cache.NoCacheProvider
    </property>

```



```

</property>
<property name="hibernate.show_sql">true</property>
<!-- Class Mapping -->
<mapping package="com.cursojava.model"/>
<mapping class="com.cursojava.model.Aluno"/>
</session-factory>
</hibernate-configuration>

```

Conteúdo de uma aplicação

A figura 7.2 mostra os diretórios e seus conteúdos para uma aplicação usando Hibernate.

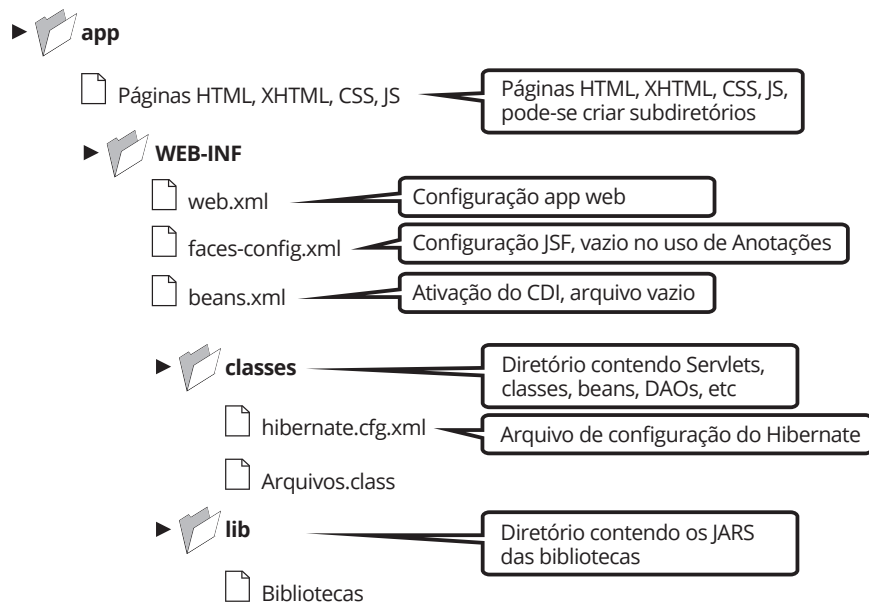


Figura 7.2
Conteúdo de
uma aplicação.

Exercício de Fixação

Criar a aplicação contendo a classe persistente Aluno, o Managed Bean AlunoMB, o arquivo *indexemplo.xhtml* e o arquivo *listar.xhtml*, conforme mostrados anteriormente.

Conceitos e Ciclo de Vida

Alguns conceitos são de extrema importância para o entendimento do Hibernate:

- Ciclo de Vida de Persistência: estados e transições de estados que dizem respeito à persistência de um objeto;
- Unidade de Trabalho: conjunto de operações que são consideradas como um grupo (algumas vezes atômico);
- Contexto de Persistência: mecanismo que mantém todos os objetos persistentes, seus estados e modificações. Mantido por uma Session em Hibernate (ou EntityManager no JPA);
- Estados: situações que um objeto pode assumir no ciclo de vida de persistência: transiente, Persistente, Removido, Desligado;



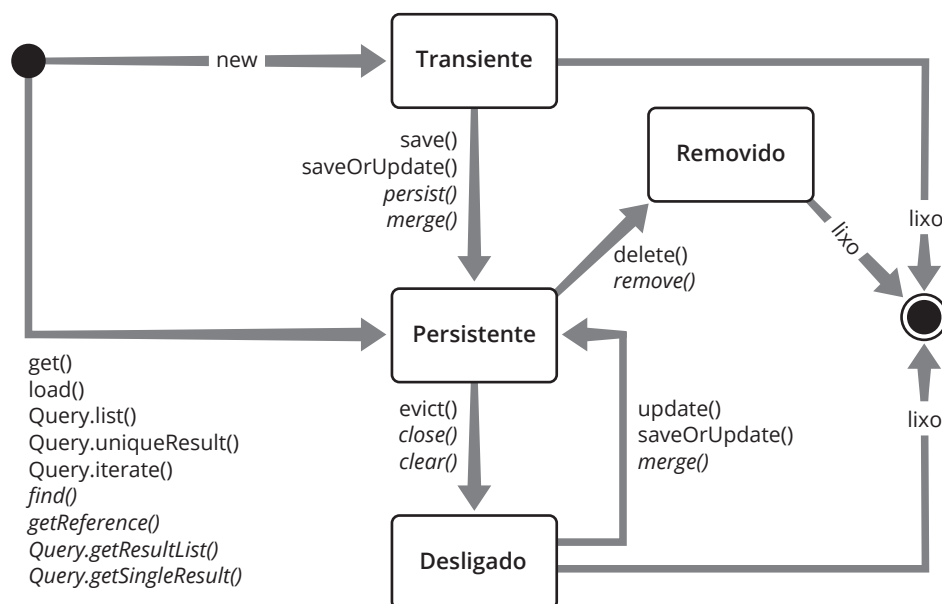


Figura 7.3
Ciclo de Vida de Objetos.

A figura 7.3 mostra os estados e os métodos que fazem com que um objeto transite entre esses estados. Os estados são:

- **Transientes:** instanciados com `new`. Não são persistidos imediatamente;
- **Persistentes:** objetos que possuem um correspondente na base e já estão persistidos;
- **Removidos:** objeto marcado para remoção no final do processo corrente;
- **Desligados:** objetos persistidos, mas que estão fora de um contexto de persistência. Exemplo: objetos que ainda existem depois que uma sessão foi fechada. Não há garantia de sincronismo desses com a base.



Anotações

Para anotar uma entidade, usa-se `@Entity`. Por exemplo, a classe `Pessoa` a seguir será persistente:

```
@Entity
public class Pessoa {
    ...
}
```

Para criar um identificador único, que no banco de dados se reflete como uma chave primária, usa-se `@Id`, como no exemplo a seguir:

```
@Entity
public class Pessoa {
    @Id
    private Long id;
}
```

Para customizar o nome da tabela no banco de dados e o nome da coluna, usa-se `@Table` e `@Column`. No exemplo a seguir, o nome da tabela foi alterado. Se não for usado, o nome da tabela deve ser o mesmo nome da classe:

```
@Entity
@Table(name = "tb_pessoa")
public class Pessoa {
    @Id
    private Long id;
}
```

No exemplo a seguir, o nome da coluna também foi customizado. Se não for usado, o nome da coluna deve ser o mesmo nome do atributo da classe.

```
@Entity
@Table(name = "tb_pessoa")
public class Pessoa {
    @Id
    @Column(name="pess_id")
    private Long id;
}
```

Alguns atributos de @Column são:

- **length**: limita a quantidade de caracteres em uma string;
- **nullable**: se o campo pode ter null ou não;
- **unique**: se a coluna pode ter campos repetidos ou não;
- **precision**: quantidades de dígitos de um número decimal;
- **scale**: quantidade de casas decimais de um número decimal.

E seu uso pode ser observado no exemplo a seguir.

```
@Entity
public class Pessoa {
    @Id
    private Long id;
    @Column(length=30, nullable=false, unique=true)
    private String nome;
    @Column(precision=3, scale=2)
    private BigDecimal altura;
}
```

Deve-se manter a consistência entre os tamanhos na anotação e nas tabelas. O Hibernate possui ferramentas de Engenharia Reversa que conseguem criar as classes persistentes a partir de um banco de dados já criado, e vice-versa.

- Deve-se manter a consistência entre o banco de dados e as anotações;
- Hibernate tem ferramentas de Engenharia Reversa.
 - Dado uma Base de Dados, gera todas as Classes Persistentes;
 - Dadas as Classes Persistentes, gera o Banco de Dados.

Chaves primárias

Para chaves primárias simples, anota-se com @Id o atributo cujo campo é a chave.

São tipos permitidos para chaves primárias:

- Tipos primitivos (e seus wrappers);
- String;
- java.util.Date;
- java.sql.Date;
- java.math.BigDecimal;
- java.math.BigInteger.

Para chaves autogeradas, usa-se tipos discretos (inteiros). Para chaves compostas, cria-se uma classe contendo as respectivas chaves:

- Classe que contém as chaves deve ser anotada com @Embeddable. Deve implementar os métodos equals() e hashCode();
- Na entidade, anota-se o atributo contendo a classe das chaves com @EmbeddedId (javax.persistence.EmbeddedId).

Os exemplos a seguir mostram o uso de chave composta.

```
@Embeddable
public class PessoaPK implements Serializable {
    @Column(name="pess_idade")
    protected Integer idade;
    @Column(name="pess_cpf")
    protected String CPF;
    public PessoaPK() {}
    // equals, hashCode
}

@Entity
public class Pessoa implements Serializable {
    @EmbeddedId
    private PessoaPK pessoaPK;
    private String endereco;
    // setters/getters
}
```

Características de uma Classe de Chave Primária (composta):

- Classe deve ser pública;
- Propriedades devem ser públicas ou protegidas;
- Deve ter um construtor público, sem argumentos;
- Deve implementar hashCode() e equals(Object other);
- Deve implementar Serializable;
- Deve representar múltiplos atributos da classe de entidade;
- Os nomes da classe de chave primária e dos atributos na entidade que são chaves primária, devem coincidir;

Para geração automática de chaves primárias, usa-se `@GeneratedValue`, que possui os seguintes atributos:

strategy: maneira de gerar a chave primária, que pode ser:

- **GenerationType.AUTO**: (default) usa o tipo de geração definido no banco de dados subjacente;
- **GenerationType.IDENTITY**: atribui uma chave primária usando o campo identity da tabela;
- **GenerationType.SEQUENCE**: atribui uma chave primária usando uma sequence;
- **GenerationType.TABLE**: usa uma tabela criada para manter as chaves primárias.

Quando a estratégia definida é via o uso de uma sequência, é preciso utilizar a anotação `@SequenceGenerator` e alguns atributos específicos.

`@GeneratedValue(strategy=GenerationType.SEQUENCE)`

- Cria um sequenciador que atribui valores à chave primária na hora da persistência (antes do `commit()`);
- Pode ser um contador ou um elemento sequence do banco;
- É global à aplicação: várias entidades podem usá-lo.

Atributos de `@SequenceGenerator`:

- name: nome da sequence, usado internamente pelas entidades na anotação `@GeneratedValue`;
- initialValue: (default 1) valor inicial das sequências geradas;
- allocationSize: (default 50) incremento usado na geração dos números;
- sequenceName: o nome do elemento Sequence no banco de dados.

Os exemplos a seguir mostram dois tipos de sequenciadores. O primeiro inicia seu valor em 1 e retorna valores espaçados em 100.

```
@Entity
@SequenceGenerator(name="seq", initialValue=1,
                  allocationSize=100)

public class Pessoa {
    @GeneratedValue(strategy=GenerationType.SEQUENCE,
                  generator="seq")

    @Id
    private long id;
}
```

O próximo sequenciador usa uma entidade sequence do banco de dados para obter o próximo valor a ser gerado como Id.

```
@Entity
@SequenceGenerator(name="seq", sequenceName="seq_pessoa")

public class Pessoa {
    @GeneratedValue(strategy=GenerationType.SEQUENCE,
                  generator="seq")

    @Id
    private long id;
}
```

Datas

Para persistência de Datas, usa-se em Java:

- `java.util.Date;`
- `java.util.Calendar.`

Esses tipos são mapeados automaticamente para os tipos de data/hora dos bancos de dados. Por default, a data e a hora são armazenados no banco. Para mudar o comportamento, usa-se:

`@Temporal`, que pode receber um dos seguintes valores:

- **`TemporalType.DATE`**: armazena apenas data;
- **`TemporalType.TIME`**: armazena apenas horário;
- **`TemporalType.TIMESTAMP`**: (default) armazena data e hora.

O exemplo a seguir mostra o uso de datas.

```
@Entity
public class Pessoa {
    @Id
    @GeneratedValue
    private Long id;
    @Temporal(TemporalType.DATE)
    private Calendar nascimento;
}
```

LOB: Large Objects

Para persistência de objetos grandes (Large Objects: LOB), anota-se o atributo com `@Lob`.

São tipos permitidos:

- `String;`
- `byte[];`
- `Byte[];`
- `char[];`
- `Character[].`

O exemplo a seguir mostra um exemplo de uso de LOB.

```
@Entity
public class Pessoa {
    @Id
    @GeneratedValue
    private Long id;
    @Lob
    private byte[] avatar;
}
```

Atributos transientes

Quando algum atributo de uma entidade não deve ser persistido, usa-se a anotação `@Transient` ou marca-se o atributo como `transient`. No exemplo a seguir, o atributo `idade` não será persistido.

```
@Entity
public class Pessoa {
    @Id
    @GeneratedValue
    private Long id;
    @Temporal(TemporalType.DATE)
    private Calendar nascimento;
    @Transient
    private int idade;
}
```

Método de acesso aos atributos

O JPA, e portanto o Hibernate, fornece dois tipos de acesso aos atributos de uma entidade.

Tipos de Acesso:

- **Field Access:** quando as anotações são colocadas nos atributos; Hibernate acessa os dados direto, via reflection. Não usa os métodos getters/setters (são ignorados pelo Hibernate)
- **Property Access:** quando as anotações são colocadas nos métodos `get()`. O Hibernate usa os métodos getters/setters para acessar os dados que devem estar obrigatoriamente implementados

O Hibernate usa o acesso definido pela anotação `@Id` para definir qual é o tipo que será usado na classe persistente. O programador deve usar sempre o mesmo tipo de acesso, durante a mesma classe, sob a pena de o Hibernate ignorar determinadas consistências que estejam atreladas a acessos diferentes do determinado pelo `@Id`.

Interfaces do Hibernate

Qualquer tipo de acesso ao banco de dados pode ser feito com o Hibernate, e isso é provido por suas interfaces.

Admite-se:

- Operações CRUD: inserções, Atualizações, Remoções e Consultas;
- Execução de Queries: consultas via HQL ou Criteria;
- Controle de Transações: `commit/Rollback`;
- Gerenciamento do Contexto de Persistência: manutenção e gerência dos estados dos objetos.

Em termos de código, temos as seguintes interfaces:

Interfaces do Hibernate:

- **Session**: interface principal. Mantém o contexto de persistência.
 - ▣ **Automatic Dirty Checking**: só atualiza objetos alterados;
 - ▣ **Cache**: mantém um cache com os objetos persistentes.
- **Query**: efetua consultas via HQL;
- **Criteria**: efetua consultas programaticamente;
- **Transaction**: controla as transações.



Iniciando uma Unidade de Trabalho

Usa-se um SessionFactory, por exemplo:

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
```

Uma aplicação pode ter várias fábricas (SessionFactory), se acessa vários bancos de dados. Essas fábricas são obtidas através da classe HibernateUtil. Pela fábrica, obtém-se uma sessão (chamada de openSession()) e é na sessão que as operações são efetuadas.

Características desses objetos:

- **SessionFactory**: é caro de criar e portanto não deve ser criada a cada requisição;
- **Session**: é um objeto leve que não obtém uma conexão de imediato, por isso mesmo podendo ser criada a cada conexão.



Todas as operações ocorrem dentro de uma transação (Transaction) (tanto leitura como escrita).

Persistindo um objeto

Usa-se o método save(), como no exemplo a seguir:

```
Pessoa p = new Pessoa();
p.setNome("João");
p.setIdade(18);
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
Serializable pessoaId = session.save(p);
tx.commit();
session.close();
```

O método save() retorna o Id do objeto persistido. O objeto é sincronizado com o banco na chamada do commit().

Recuperando um objeto persistente

Usa-se ou o método get() ou o load(), como no exemplo a seguir:

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
Pessoa p = session.load(Pessoa.class, new Long(1));
// Pessoa p = session.get(Pessoa.class, new Long(1));
tx.commit();
session.close();
```




Diferenças entre `get()` e `load()`:

- **`get()`**: retorna o NULL se o objeto não existe;
- **`load()`**: lança `ObjectNotFoundException` se o objeto não existe.

O objeto ficará em estado persistente até que a sessão seja fechada (entra em estado desligado).

Modificando um objeto persistente

Basta alterar uma propriedade de um objeto persistente e efetuar o `commit()` para sincronizar com o banco de dados.

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
Pessoa p = session.get(Pessoa.class, new Long(1));
p.setNome("Maria");
tx.commit();
session.close();
```

Quando ocorre o `commit()`, o objeto é sincronizado, isto é, se houver alguma alteração ela é efetuada no banco de dados (Automatic Dirty Checking).

Remoção de um objeto

Usa-se `delete()` para remover o estado persistente do objeto, como no exemplo a seguir:

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
Pessoa p = session.get(Pessoa.class, new Long(1));
session.delete(p);
tx.commit();
session.close();
```

Após o `delete()`, o objeto passa para o estado removido e não deve mais ser manipulado. Quando ocorre o `commit()`, o SQL (`delete`) é executado e o objeto não terá mais estado persistente.

Religar um objeto desligado

Religar um objeto desligado se refere a objetos que já estão na base de dados, mas não estão na sessão do hibernate (portanto desligados). Por exemplo, isso ocorre quando um objeto, que já está na base de dados, é alterado e é entregue para um procedimento abrir a sessão do hibernate e realizar as alterações na base de dados. Como o objeto alterado (seu Id) já está na base de dados e uma nova sessão do hibernate está sendo aberta, esse objeto precisa ser religado na sessão para que haja um agendamento de um `UPDATE` na base de dados.

Deve ser feito quando:

- Tem-se um objeto que já está na base de dados;
- Esse objeto possui valores diferentes em atributos (portanto precisa ser atualizado);
- Ele não está na sessão do hibernate (exemplo: sessão recém-aberta).

O processo é:

- Abrir ou obter a sessão do hibernate;
- Chamar método `update()`.
 - Agenda um `UPDATE` na base de dados.



Para resolver esta questão, deve-se abrir ou obter a sessão do hibernate e chamar o método `update()` passando como parâmetro o objeto a ser atualizado. Esse procedimento coloca o objeto na sessão e agenda o UPDATE necessário.

```
p.setNome("Teste"); // objeto fora da sessão mas já existe na base
Session session2 = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
session2.update(p);
p.setIdade(28);
tx.commit();
session2.close();
```

O `update()` religa a instância para uma nova sessão. Como a sessão é nova, não tem informação se o objeto está ou não sujo (isto é, se foi alterado), portanto, não importa se houve alterações antes ou depois do `update()`, o objeto sempre será persistido.

Se há certeza absoluta que o objeto não foi alterado, usa-se o método `lock()`, como no exemplo:

```
Session session2 = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
session2.lock(p, LockMode.NONE);
p.setIdade(28);
tx.commit();
session2.close();
```

O `lock()` religa a instância (não alterada) para uma nova sessão. Tudo o que foi modificado antes do `lock()` não é persistido, tudo que for alterado depois de sua chamada é persistido.

Removendo objeto Desligado

Um objeto Desligado passa a ser Transiente através do uso do método `delete()`, como no exemplo:

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
session.delete(p);
tx.commit();
session2.close();
```

O `delete()`, nesse caso, religa o objeto com o contexto de persistência e agenda a deleção do mesmo no `commit()`.

8

Hibernate – Associações

objetivos

Aprender as configurações necessárias no Hibernate para se trabalhar com relacionamento entre tabelas e sua contrapartida, como associações entre classes; Conhecer os quatro tipos mais comuns de associações.

Associações Um-para-Um, Um-para-Muitos, Muitos-para-Um e Muitos-para-Muitos; Cascadeamento e Lazy Fetch.

conceitos

Associações

Existem quatro tipos de Associações (em orientação a objetos) que são mapeadas para Relacionamentos (no modelo relacional). Vejamos cada um deles no exemplo a seguir. Tipos de Associação entre Pessoa e Endereço:

- **um-para-um (@OneToOne)**: uma pessoa tem um só endereço. Um endereço pertence a somente uma pessoa;
- **um-para-muitos (@OneToMany)**: uma pessoa tem vários endereços. Um endereço pertence a somente uma pessoa;
- **muitos-para-um (@ManyToOne)**: uma pessoa tem somente um endereço. Um endereço pode ser de várias pessoas. Basicamente é uma inversão da um-para-muitos;
- **muitos-para-muitos (@ManyToMany)**: uma pessoa tem vários endereços. Um endereço pode ser de várias pessoas.

Para entender as associações, dois conceitos são importante: cascadeamento e Lazy Load.

Cascadeamento

Cascadeamento é quando se indica que uma operação aplicada em um objeto será também aplicada aos objetos a ele associados. Os tipos são definidos na classe CascadeType. A seguir são apresentados alguns deles:

- **CascadeType.PERSIST**: cascadeia a operação de persistência (salvar);
- **CascadeType.MERGE**: cascadeia a operação que religa um objeto ao contexto de persistência;
- **CascadeType.REMOVE**: cascadeia a operação de remoção;

- **CascadeType.REFRESH**: cascadeia a operação de refresh;
- **CascadeType.DETACH**: cascadeia a operação que desliga um objeto do contexto de persistência;
- **CascadeType.ALL**: cascadeia todas as operações.



Lazy Load

O Load ou Fetch é a indicação de como a associação deve ser recuperada, definindo se, ao carregar um objeto que possui outros objetos associados, os dados das associações também devem ser recuperados.

Temos dois tipos:

- **FetchType.EAGER**: ao carregar um objeto, carrega todas as associações imediatamente;
- **FetchType.LAZY**: ao carregar um objeto, não carrega as associações imediatamente, somente nas chamadas dos getters.



Em toda associação @OneToOne, @OneToMany e @ManyToMany, podemos definir o tipo de Fetch, sendo que o default é FetchType.LAZY. Para trocar o tipo de Fetch em uma associação, usa-se o atributo fetch das anotações.

- Em toda associação: @OneToOne, @OneToMany e @ManyToMany;
- Default é FetchType.LAZY;
- Para trocar o tipo de Fetch em uma associação, usa-se:

```
@OneToOne(fetch=FetchType.EAGER)
@JoinColumn(name="user_id")
private Profile getUser() {
    return user;
}
```



A escolha de qual estratégia usar depende do sistema sendo desenvolvido e de requisitos de desempenho e tempo de codificação. Eager demanda pouca codificação porque o próprio hibernate já busca todos os dados dos objetos associados. O problema é o desempenho, pois podemos ter várias associações e podem ser buscados dados que não serão usados no processo. Já Lazy demanda um pouco mais de codificação, mesmo em HQL (que será visto em outra sessão) ou na manutenção da sessão, mas é mais eficiente, pois as buscas são feitas sob demanda, explicitamente. Preferencialmente os sistemas usam Lazy Fetch.

O que usar?

- Eager:
 - Pouca codificação;
 - Problemas de desempenho, pois busca todas as associações.
- Lazy – Default e Preferencial:
 - Mais codificação;
 - Mais eficiente, pois só busca os dados da associação explicitamente.



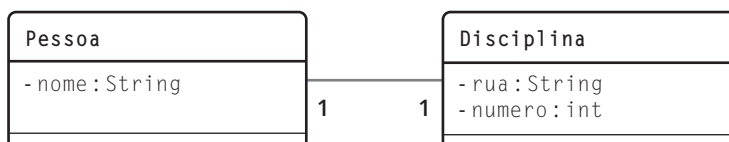
UML

Unified Modeling Language, uma notação usada para descrever sistemas em várias visões. Mais detalhes em: <http://www.uml.org/>

Associação Um-para-Um

Uma associação Um-para-Um indica que um objeto possui relação com somente outro, por exemplo, uma pessoa que possui um e somente um endereço. Nessa relação, um endereço é também de somente uma pessoa. A figura a seguir ilustra como seria a associação, através de uma ligação entre as duas entidades, usando notação **UML**.

Figura 8.1
Associação Um-para-Um.



Nesta notação, as caixas representam as classes (e portanto são modelos para os objetos) e a conexão entre elas representa a associação. O número perto da classe Endereço indica que uma Pessoa pode ter um (1) Endereço. O número perto de Pessoa indica que um Endereço pode ser de uma (1) Pessoa.

Uma associação pode ser bidirecional e, nesse caso, um dos lados deve ser o proprietário da associação. No proprietário, a propriedade da associação é anotada com @OneToMany e é responsável por manter a coluna de associação. É nessa classe que se declara a coluna de ligação, que torna a associação existente no banco de dados. No lado oposto é usada a anotação @ManyToOne, com um atributo chamado mappedBy, para indicar a propriedade do lado inverso, que representa a associação.

Um dos lados deve ser o dono ou proprietário da associação.

- O proprietário:
 - ▢ Na propriedade da associação anota-se com @OneToMany;
 - ▢ É responsável por manter a associação;
 - ▢ Declara a coluna de ligação.
- Do lado oposto ao proprietário, usamos:
 - ▢ Anotação @ManyToOne;
 - ▢ Com atributo mappedBy.
- @JoinColumn é usado para indicar qual é o campo que liga das duas tabelas

Tabela 8.1
Atributos de @OneToMany.

As anotações utilizadas são @OneToMany, que deve ser colocada na classe proprietária da associação, bem como na outra classe. A tabela 8.1 mostra os atributos dessa anotação.

Atributos	Descrição
cascade	Opcional. O tipo de cascadeamento entre as operações.
fetch	Opcional. Se a associação será carregada ou não (lazy) ao se buscar um objeto desse tipo.
mappedBy	Opcional. O campo que é dono do relacionamento. Esse atributo só é configurado do lado inverso da associação (o que não é o dono).
optional	Opcional. Se a associação é opcional (pode ser nula).
orphanRemoval	Opcional. Se a operação de remoção deve ser aplicada na associação também.
targetEntity	Opcional. A classe da entidade alvo da associação

Já a anotação @JoinColumn é usada para indicar o campo que faz a ligação entre as duas tabelas da relação, e seus atributos são apresentados na tabela a seguir. A tabela 8.2 mostra alguns atributos de @JoinColumn.

Atributos	Descrição
name	Opcional. Nome da coluna de ligação entre as tabelas.
nullable	Opcional. Se a coluna pode ser NULL.
table	Opcional. Nome da tabela onde está a coluna de ligação.
unique	Opcional. Se a propriedade é uma chave única.

Tabela 8.2
Atributos de
@JoinColumn.

As classes da figura 8.1 podem ser mapeadas para as seguintes tabelas:

```
create table tb_pessoa (
    id          integer NOT NULL PRIMARY KEY,
    pess_nome character varying(30) NOT NULL);
create table tb_endereco (
    id          integer NOT NULL PRIMARY KEY,
    end_rua     character varying(30) NOT NULL,
    end_numero integer NOT NULL);
alter table tb_pessoa add end_id integer not null REFERENCES tb_endereco(id);
CREATE SEQUENCE seq_pess_id
    INCREMENT 1
    MINVALUE 1
    MAXVALUE 9223372036854775807
    START 11
    CACHE 1;
CREATE SEQUENCE seq_end_id
    INCREMENT 1
    MINVALUE 1
    MAXVALUE 9223372036854775807
    START 11
    CACHE 1;
```

A classe Pessoa deve ser anotada conforme o trecho de código a seguir:

```
package com.cursojava.model;
// imports
@Entity
@Table (name="tb_pessoa")
@SequenceGenerator(name="sequencia", sequenceName="seq_pess_id")
public class Pessoa implements Serializable {
    private Long id;
    private String nome;
    private Endereco endereco;
    @Id
    @GeneratedValue(strategy=GenerationType.SEQUENCE,
        generator="sequencia")
    public Long getId() {
        return id;
    }
}
```

```

    }
    public void setId(Long id) {
        this.id = id;
    }
    @Column(updatable=true, name="pess_nome", nullable=false, length=30)
    public String getNome() {
        return this.nome;
    }
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    }
    @OneToOne(cascade=CascadeType.ALL)
    @JoinColumn(name="end_id", updatable=true)
    public Endereco getEndereco() {
        return this.endereco;
    }
    }
    public void setEndereco(Endereco endereco) {
        this.endereco = endereco;
    }
    }
}

```

A classe Endereco, por sua vez, deve ser anotada conforme o seguinte trecho:

```

package com.cursojava.model;
// imports
@Entity
@Table (name="tb_endereco")
@SequenceGenerator(name="seq_end", sequenceName="seq_end_id")
public class Endereco implements Serializable{
    private Long id;
    private String rua;
    private Integer numero;
    private Pessoa pessoa;
    @Id
    @GeneratedValue(strategy=GenerationType.SEQUENCE, generator="seq_end")
    public Long getId() {
        return id;
    }
    }
    public void setId(Long id) {
        this.id = id;
    }
    }
    @Column(updatable=true, name="end_rua", nullable=false, length=30)
    public String getRua() {
        return this.rua;
    }
    }
    public void setRua(String rua) {
        this.rua = rua;
    }
    }
    @Column(updatable=true, name="end_numero", nullable=false)
    public Integer getNumero() {

```

```

        return this.numero;
    }
    public void setNumero(Integer numero) {
        this.numero = numero;
    }
    @OneToOne(mappedBy = "endereco")
    public Pessoa getPessoa() {
        return this.pessoa;
    }
    public void setPessoa(Pessoa pessoa) {
        this.pessoa = pessoa;
    }
}

```

Por fim, apresentamos um trecho de código usando esta configuração de associação:

```

Pessoa pessoa = new Pessoa();
pessoa.setNome("João");
Endereco endereco = new Endereco();
endereco.setRua("Rua XV de Novembro");
endereco.setNumero(500);
pessoa.setEndereco(endereco);
endereco.setPessoa(pessoa);
Session session = HibernateUtil.getSessionFactory()
    .openSession();
session.beginTransaction();
session.save(pessoa);
session.getTransaction().commit();
session.close();

```

No exemplo anterior o passo de preenchimento do atributo da associação deve ser feito para os dois objetos. Deve-se preencher o endereço na Pessoa e também a pessoa no Endereco.

Associação Um-para-Muitos

Uma associação Um-para-Muitos indica que um objeto possui relação com vários outros. Por exemplo, um professor que ministra várias disciplinas e uma disciplina que é ministrada por somente um professor. A figura a seguir ilustra como seria a associação, através de um ligação entre as duas entidades, usando a notação UML.

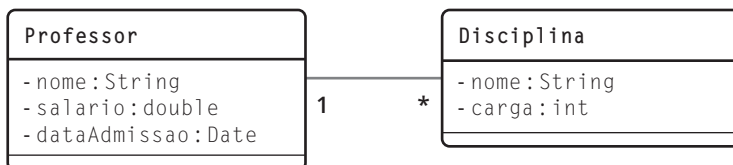


Figura 8.2
Associação
Um-para-Muitos.

Como no primeiro exemplo, as caixas representam as classes (e portanto são modelos para os objetos), e a conexão entre elas representa a associação. O asterisco perto da classe Disciplina indica que um professor pode ministrar nenhuma ou várias (*, ou 0..*) Disciplinas. O número perto de Professor indica que uma Disciplina pode ser ministrada por somente um (1) Professor.

O lado “many” é o proprietário da associação, pois é nele que está a informação de ligação entre as classes ou tabelas. O proprietário é responsável por manter a coluna de associação, e é nessa classe que se declara a coluna de ligação. No lado oposto é usado um atributo chamado mappedBy para indicar a propriedade do lado inverso, que representa a associação.

- O lado “many” é o proprietário da associação.
 - No exemplo: disciplina.
- Em Professor:
 - Na propriedade da associação anota-se com @OneToMany;
 - Usa-se mappedBy para indicar o proprietário da associação.
- Em Disciplina;
 - Anotação @ManyToOne.
 - Como Disciplina é o proprietário da associação;
 - Anota-se @JoinColumn.



As anotações utilizadas são @OneToMany, que deve ser colocada na classe proprietária da associação, @ManyToOne, que deve ser colocada na classe que está sujeita à associação e o @JoinColumn, usado para indicar o campo que faz a ligação entre as duas tabelas.

Como já foi dito, no caso de associações Um-para-Muitos, o lado “many” será o dono da associação, pois manterá o campo de ligação, e portanto será nessa classe que será declarado o @JoinColumn.

Tabela 8.3
Atributos de
@OneToMany.

A tabela 8.3 mostra os atributos de @OneToMany.

Atributos	Descrição
cascade	Opcional. O tipo de cascadeamento entre as operações.
fetch	Opcional. Se a associação será carregada ou não (lazy) ao se buscar um objeto desse tipo.
mappedBy	Opcional. O campo que é dono do relacionamento. Esse atributo só é configurado do lado inverso da associação (o que não é o dono).
orphanRemoval	Opcional. Se a operação de remoção deve ser aplicada na associação também.
targetEntity	Opcional. A classe da entidade alvo da associação

A tabela 8.4 mostra os atributos de @ManyToOne.

Atributos	Descrição
cascade	Opcional. O tipo de cascadeamento entre as operações.
fetch	Opcional. Se a associação será carregada ou não (lazy) ao se buscar um objeto desse tipo.
optional	Opcional. Se a associação é opcional (pode ser nula)

Tabela 8.4
Atributos de
@ManyToOne.

As classes da figura 8.2 podem ser mapeadas para as seguintes tabelas:

```
create table tb_professor (
    id          integer  NOT NULL PRIMARY KEY,
    prof_nome   character varying(30) NOT NULL,
    prof_salario numeric(10, 2) NOT NULL,
    prof_dtadmissao date  NOT NULL);
```



```

create table tb_disciplina (
    id          integer NOT NULL PRIMARY KEY,
    disc_nome    character varying(30) NOT NULL,
    disc_carga integer NOT NULL);
alter table tb_disciplina add prof_id integer not null REFERENCES tb_professor(id);
CREATE SEQUENCE seq_prof_id
    INCREMENT 1
    MINVALUE 1
    MAXVALUE 9223372036854775807
    START 11
    CACHE 1;
CREATE SEQUENCE seq_disc_id
    INCREMENT 1
    MINVALUE 1
    MAXVALUE 9223372036854775807
    START 11
    CACHE 1;

```

Deve-se perceber que, para caracterizar uma associação Um-para-Muitos entre Professor e Disciplina, o campo de ligação (chave estrangeira) está na tabela tb_disciplina, e é por isso que a coluna de ligação será declarada na classe Disciplina.

As classes Professor e Disciplina devem ser anotadas conforme o exemplo a seguir. Primeiramente mostramos a classe Professor.

```

package com.cursojava.model;
// imports
@Entity
@Table (name="tb_professor")
@SequenceGenerator(name="seq_prof", sequenceName="seq_prof_id")
public class Professor implements Serializable {
    private Long id;
    private String nome;
    private Double salario;
    private Date dataAdmissao;
    private List<Disciplina> disciplinas;
    @Id
    @GeneratedValue(strategy=GenerationType.SEQUENCE, generator="seq_prof")
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    @Column(updatable=true, name="prof_nome", nullable=false, length=30)
    public String getNome() {
        return this.nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
}

```

```

    }
    @Column(updatable=true, name="prof_salario", nullable=false)
    public Double getSalario() {
        return this.salario;
    }
    public void setSalario(Double salario) {
        this.salario = salario;
    }
    @Column(updatable=true, name="prof_dtadmissao", nullable=false)
    public Date getDataAdmissao() {
        return this.dataAdmissao;
    }
    public void setDataAdmissao(Date dataAdmissao) {
        this.dataAdmissao = dataAdmissao;
    }
    @OneToMany(mappedBy="professor", cascade=CascadeType.ALL,
        fetch=FetchType.EAGER)
    public List<Disciplina> getDisciplinas() {
        return this.disciplinas;
    }
    public void setDisciplinas(List<Disciplina> disciplinas) {
        this.disciplinas = disciplinas;
    }
}

```

Temos agora as anotações na classe Disciplina.

```

package com.cursojava.model;
// imports
@Entity
@Table (name="tb_disciplina")
@SequenceGenerator(name="seq_disc", sequenceName="seq_disc_id")
public class Disciplina implements Serializable{
    private Long id;
    private String nome;
    private Integer carga;
    private Professor professor;
    @Id
    @GeneratedValue(strategy=GenerationType.SEQUENCE,
        generator="seq_disc")
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    @Column(updatable=true, name="disc_nome", nullable=false, length=30)
    public String getNome() {
        return this.nome;
    }
}

```

```

    public void setNome(String nome) {
        this.nome = nome;
    }
    @Column(updatable=true, name="disc_carga", nullable=false)
    public Integer getCarga() {
        return this.carga;
    }
    public void setCarga(Integer carga) {
        this.carga = carga;
    }
    @ManyToOne
    @JoinColumn(name="prof_id")
    public Professor getProfessor() {
        return this.professor;
    }
    public void setProfessor(Professor professor) {
        this.professor = professor;
    }
}

```

Um exemplo de trecho de código usando esta configuração de associação pode ser visto a seguir.

```

Professor professor = new Professor();
professor.setNome("João");
professor.setDataAdmissao(new Date());
professor.setSalario(15000.00);
ArrayList<Disciplina> lista = new ArrayList<Disciplina>();
Disciplina disciplina = new Disciplina();
disciplina.setNome("Java Básico");
disciplina.setCarga(40);
disciplina.setProfessor(professor);
lista.add(disciplina);
Disciplina disciplina = new Disciplina();
disciplina.setNome("Java Corporativo");
disciplina.setCarga(40);
disciplina.setProfessor(professor);
lista.add(disciplina);
professor.setDisciplinas(lista);
Session session = HibernateUtil.getSessionFactory()
    .openSession();
session.beginTransaction();
session.save(professor);
session.getTransaction().commit();
session.close();

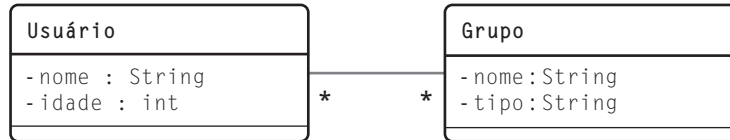
```

As associações do tipo Muitos-para-Um são feitas da mesma forma que as Um-para-Muitos, basicamente alterando a ordem da associação. Mesmo nesse tipo de associação, no lado "many", isto é, na anotação @ManyToOne, é que deve ser declarado o campo de ligação.

Associação Muitos-para-Muitos

Uma associação Muitos-para-Muitos indica que um objeto possui relação com vários outros. Por exemplo, um usuário que participa de vários grupos, e um grupo que possui vários usuários cadastrados. A figura a seguir ilustra como seria a associação, através de uma ligação entre as duas entidades, usando a notação UML.

Figura 8.3
Associação Muitos-para-Muitos.



Como nos exemplos anteriores, as caixas representam as classes (e portanto são modelos para os objetos), e a conexão entre elas representa a associação. O asterisco perto da classe Grupo indica que um Usuário pode participar de nenhum ou de vários (*, ou 0..*) Grupos. O asterisco perto de Usuário indica que um Grupo pode conter nenhum ou vários (*, ou 0..*) Usuários.

Como nos outros tipos de associações, um dos lados deve ser o proprietário da associação. O proprietário é responsável por manter uma tabela de associação, visto que somente um campo em um dos lados não é suficiente para denotar uma associação Muitos-para-Muitos. No lado oposto, é usado um atributo chamado `mappedBy` para indicar a propriedade do lado inverso, que representa a associação.

- Um dos lados deve ser o proprietário da associação.
 - ▣ No exemplo: usuário;
 - ▣ Usa-se (cria-se) uma tabela de ligação.
- Em Usuário:
 - ▣ Na propriedade da associação anota-se com `@ManyToMany`;
 - ▣ Como Usuário é o proprietário da associação anota-se com `@JoinTable`.
- Em Disciplina:
 - ▣ Anotação `@ManyToMany`;
 - ▣ Usa-se `mappedBy` para indicar o proprietário da associação.
- `@JoinColumn`: usado para indicar qual é o campo que liga as duas tabelas.



As anotações utilizadas são `@ManyToMany`, que deve ser colocada tanto na classe proprietária como na outra classe da associação; o `@JoinTable` que é usado para indicar a tabela de ligação entre as duas tabelas, e o `@JoinColumn`, que é usado para descrever a coluna na tabela de ligação.

Tabela 8.5
Atributos de
`@ManyToMany`.

A tabela 8.5 mostra os atributos de `@ManyToMany`:

Atributos	Descrição
<code>cascade</code>	Opcional. O tipo de cascadeamento entre as operações.
<code>fetch</code>	Opcional. Se a associação será carregada ou não (lazy) ao se buscar um objeto desse tipo.
<code>mappedBy</code>	Opcional. O campo que é dono do relacionamento. Esse atributo só é configurado do lado inverso da associação (o que não é o dono).
<code>targetEntity</code>	Opcional. A classe da entidade alvo da associação



A tabela 8.6 mostra os atributos de @JoinTable.

Atributos	Descrição
joinColumns	Opcional. Lista de @JoinColumn indicando as colunas que referenciam a tabela que é dona da associação.
inverseJoinColumns	Opcional. Lista de @JoinColumn indicando as colunas que referenciam a tabela que não é dona da associação.
name	Opcional. Nome da tabela de ligação.

As classes da figura 8.3 podem ser mapeadas para as seguintes tabelas:

Tabela 8.6
Atributos de
@JoinTable.

```
create table tb_usuario (
    id            integer NOT NULL PRIMARY KEY,
    usuario_nome  character varying(50) NOT NULL,
    usuario_idade integer NOT NULL);

create table tb_grupo (
    id            integer NOT NULL PRIMARY KEY,
    grupo_nome    character varying(50) NOT NULL,
    grupo_tipo    character varying(50) NOT NULL);
create table tb_usuario_grupo (
    usuario_id integer NOT NULL REFERENCES tb_usuario(id),
    grupo_id   integer NOT NULL REFERENCES tb_grupo(id));
CREATE SEQUENCE seq_usuario_id
    INCREMENT 1
    MINVALUE 1
    MAXVALUE 9223372036854775807
    START 11
    CACHE 1;
CREATE SEQUENCE seq_grupo_id
    INCREMENT 1
    MINVALUE 1
    MAXVALUE 9223372036854775807
    START 11
    CACHE 1;
```

Deve-se perceber que, para caracterizar uma associação Muitos-para-Muitos entre Usuário e Grupo, foi criada uma nova tabela, chamada tb_usuario_grupo. Essa nova tabela é mapeada no lado dono da associação, o qual, nesse exemplo, foi escolhido como sendo o Usuário.

As classes Usuário e Grupo devem ser anotadas conforme o exemplo a seguir. Primeiro temos a classe Usuário.

```
package com.cursojava.model;
// imports
@Entity
@Table (name="tb_usuario")
@SequenceGenerator(name="seq_usuario", sequenceName="seq_usuario_id")
```

```

public class Usuario implements Serializable {
    private Long id;
    private String nome;
    private Integer idade;
    private Collection<Grupo> grupos;
    public Usuario() {
    }
    @Id
    @GeneratedValue(strategy=GenerationType.SEQUENCE,
                    generator="seq_usuario")

    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    @Column(updatable=true, name="usuario_nome", nullable=false,
            length=50)
    public String getNome() {
        return this.nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    @Column(updatable=true, name="usuario_idade", nullable=false)
    public Integer getIdade() {
        return this.idade;
    }
    public void setIdade(Integer idade) {
        this.idade = idade;
    }
    @ManyToMany(
        targetEntity=com.cursojava.model.Grupo.class,
        cascade={CascadeType.PERSIST, CascadeType.MERGE},
        fetch=FetchType.EAGER
    )
    @JoinTable(
        name="tb_usuario_grupo",
        joinColumns={@JoinColumn(name="usuario_id")},
        inverseJoinColumns={@JoinColumn(name="grupo_id")}
    )
    public Collection<Grupo> getGrupos() {
        return this.grupos;
    }
    public void setGrupos(Collection<Grupo> grupos) {
        this.grupos = grupos;
    }
}

```

Aqui apresentamos as anotações da classe Grupo.

```
package com.cursojava.model;
// imports
@Entity
@Table (name="tb_grupo")
@SequenceGenerator(name="seq_grupo", sequenceName="seq_grupo_id")
public class Grupo implements Serializable{
    private Long id;
    private String nome;
    private String tipo;
    private Collection<Usuario> usuarios;
    public Grupo() {
    }
    @Id
    @GeneratedValue(strategy=GenerationType.SEQUENCE,

        generator="seq_grupo")
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    @Column(uptdatable=true, name="grupo_nome", nullable=false, length=50)
    public String getNome() {
        return this.nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    @Column(uptdatable=true, name="grupo_tipo", nullable=false, length=50)
    public String getTipo() {
        return this.tipo;
    }
    public void setTipo(String tipo) {
        this.tipo = tipo;
    }
    @ManyToMany(
        cascade={CascadeType.PERSIST, CascadeType.MERGE},
        mappedBy="grupos",
        fetch=FetchType.EAGER)
    public Collection<Usuario> getUsuarios() {
        return this.usuarios;
    }
    public void setUsuarios(Collection<Usuario> usuarios) {
        this.usuarios = usuarios;
    }
}
```


Um exemplo de trecho de código usando esta configuração de associação pode ser visto no código a seguir.

```
Session session = HibernateUtil.getSessionFactory()
    .openSession();
session.beginTransaction();
Usuario usuario = (Usuario)session.get(Usuario.class, 10);
Grupo grupo = (Grupo)session.get(Grupo.class, 5);
usuario.getGrupos().add(grupo);
grupo.getUsuarios().add(usuario);
session.save(usuario);
session.getTransaction().commit();
session.close();
```


9

Hibernate – HQL e Criteria

objetivos

Conhecer a linguagem HQL para construção de consultas similares às feitas com SQL; Aprender sobre a biblioteca Criteria, usada para gerar consultas de forma programática, com validação de tipos e queries de forma segura.

Consultas; HQL; JPQL; Criteria.

conceitos

HQL

- Hibernate Query Language.
- Nível alto de abstração.
- Consultas a objetos e classes (não em tabelas e registros).
- Filtros são feitos em atributos (não em campos).
- Baseada em SQL.
- Orientada a objetos (herança, polimorfismo e associações).

JPQL.

- Linguagem do Java Persistence API.
- Baseada em HQL.



Mais informações em:
http://www.hibernate.org/hib_docs/reference/en/html/queryhql.html

O Hibernate traz um nível de abstração muito alto para a aplicação, fazendo com que todas as manipulações sejam em classes e objetos, e não mais em tabelas e registros. Assim, faz-se necessária uma linguagem de consulta, similar ao SQL, mas que manipule elementos de orientação a objetos (e não elementos relacionais) e que implemente herança de forma transparente, bem como polimorfismo e associações.

Essa linguagem é o HQL (Hibernate Query Language). Com ela é possível fazer consultas a objetos de classes (e não registros de tabelas), filtrando os elementos por valores de atributos (e não por valores de campos).

Toda a sintaxe HQL é baseada no SQL, mas levando em conta os elementos de orientação a objetos. A linguagem JPQL (JPA Query Language) é a linguagem oficial do JPA (Java Persistence API: Java EE), baseada no HQL.





O entendimento desta sessão pressupõem que o aluno já esteja familiarizado com o SQL e que o tenha utilizado anteriormente.

Para a execução dos exemplos que serão apresentados nesta sessão, vamos utilizar as classes apresentadas na figura 9.1.

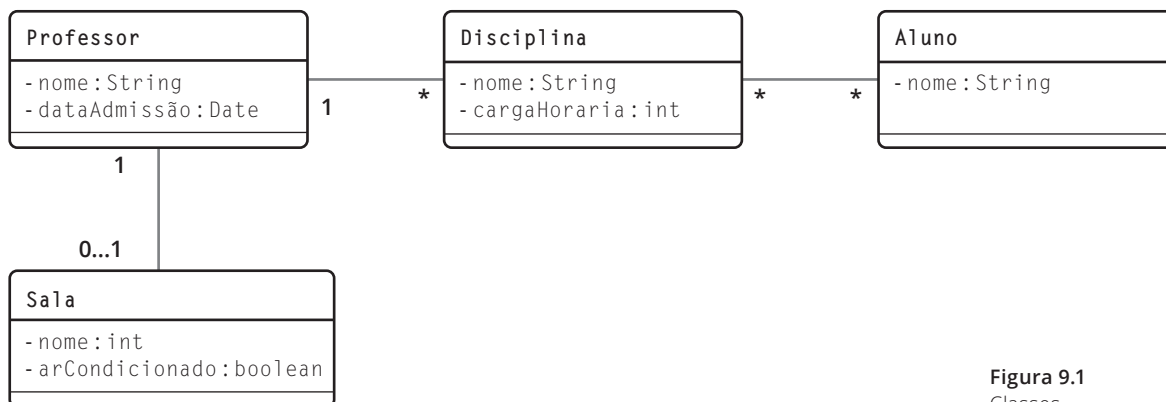


Figura 9.1
Classes.

O banco de dados representado por essa modelagem pode ser criado com o seguinte código:

```
create table tb_professor (
    prof_id serial primary key,
    prof_nm character varying(50),
    prof_dt date
);
create table tb_sala (
    sala_id serial primary key,
    sala_nr integer,
    sala_ar boolean
);
create table tb_disciplina (
    disc_id serial primary key,
    disc_nm character varying(50),
    disc_carga integer
);
create table tb_aluno (
    aluno_id serial primary key,
    aluno_nm character varying(50)
);
create table tb_disciplina_aluno (
    disc_id integer references tb_disciplina(disc_id),
    aluno_id integer references tb_aluno(aluno_id)
);

alter table tb_professor add sala_id integer null REFERENCES tb_sala(sala_id);
alter table tb_disciplina add prof_id integer not null REFERENCES tb_professor(prof_id);
```

A API do Hibernate que cria uma query do HQL para execução é o método `createQuery()` de `session`, conforme o exemplo:

```
Query query = session.createQuery("FROM Aluno");
List<Aluno> resultado = query.list();
```

Esse trecho de código executa a query "FROM Aluno", que retorna todos os objetos de Aluno. O método `list()` de `query`, executa a query e retorna a lista de objetos.

Um laço de exemplo para mostrar os resultados pode ser visto a seguir.

```
for (Aluno o: resultado) {
    System.out.println(o.getNome());
}
```

Para filtrar uma quantidade de resultados, normalmente usados em paginação de resultados, usam-se os métodos `setFirstResult()` e `setMaxResults()`:

Filtrando Resultados:

- `setFirstResult()`: a partir de qual resultado retornar;
- `setMaxResults()`: a quantidade máxima de resultados retornados.

```
Query query = session.createQuery("FROM Aluno");
query.setFirstResult(20);
query.setMaxResults(10);
List<Aluno> resultado = query.list();
```

No exemplo anterior, o filtro aplicado retorna dez alunos a partir do vigésimo elemento.

Para filtrar os resultados, usa-se a cláusula `WHERE` com parâmetros, conforme os exemplos a seguir.

```
Query query = session.createQuery("FROM Disciplina WHERE nome = ?");
query.setString(0, "Java I");
Disciplina resultado = (Disciplina) query.uniqueResult();

Query query = session.createQuery("FROM Disciplina WHERE nome = :nome");
query.setString("nome", "Java II");
Disciplina resultado = (Disciplina) query.uniqueResult();
```

Nos dois exemplos, espera-se que haja somente um resultado da consulta, portanto, usa-se o método `uniqueResult()` em vez de `list()`. No primeiro exemplo, o parâmetro da cláusula `WHERE` é feito com interrogação (?) e, para setar o valor, usa-se o método `setString()`, passando como parâmetro a posição do parâmetro, iniciando em zero. No segundo exemplo, usa-se também `setString()`, mas o parâmetro é nomeado (:nome) e é assim que é identificado no método de atribuição de valor.

Além do `setString()`, existem outro métodos, como `setInt()` e `setDate()`, dependendo do tipo do parâmetro que está sendo passado.



Parâmetros:

- Com?: posição começa em 0;
- Nomeado: usa-se o nome do parâmetro;
- Para preenchê-los:
 - `setString()`
 - `setInt()`
 - `setDate()`

Para retornar somente um elemento:

- `uniqueResult()`: retorna um `Object`, e deve ser feito `type cast` para o tipo desejado

No `FROM` podemos usar `alias` para nomear as classes, como no exemplo:

```
FROM Disciplina as d
WHERE d.nome = 'Java III'
```

Onde `d` é usado para referenciar um objeto do tipo `Disciplina`. A palavra reservada `AS` é opcional e a query poderia ser escrita assim:

```
FROM Disciplina d
WHERE d.nome = 'Java III'
```

Como não foi utilizada a cláusula `SELECT`, o retorno padrão das funções `list()` e `uniqueResult()` é `Object`. Se for usado o `SELECT`, consegue-se mais controle sobre o que será retornado, inclusive com a tipagem correta. Um exemplo mais complexo de `FROM` e `WHERE` pode ser visto a seguir.

Cláusulas `FROM` e `WHERE`

```
SELECT DISTINCT d
FROM Disciplina d
WHERE d.carga > :carga AND d.nome = :nome
```

- Retorna todas as disciplinas conforme critérios dados pelos parâmetros `:carga` e `:nome`
- Não retorna elementos duplicados.
 - Uso do `DISTINCT` no `SELECT`.

Essa consulta retorna todas as disciplinas cuja carga horária seja maior que o parâmetro `:carga` e cujo nome seja igual ao parâmetro `:nome`. Por conta do `DISTINCT` na cláusula `SELECT`, não são retornados elementos duplicados.

O `IN`, na cláusula `WHERE`, verifica se o elemento está dentro de um conjunto de outros elementos. Por exemplo:

```
SELECT DISTINCT p
FROM Professor p, IN(p.disciplinas) d
```

Retorna todos os professores que possuem disciplinas. Equivale a:

```
SELECT DISTINCT p
FROM Professor p JOIN p.disciplinas d
SELECT DISTINCT p
FROM Professor p
WHERE p.disciplinas IS NOT EMPTY
```

No WHERE, além do AND como visto anteriormente, também podemos usar o OR para combinar restrições, como no exemplo:

```
SELECT DISTINCT p
FROM Professor p JOIN p.disciplinas d
WHERE d.carga = 60 OR d.nome='Java III'
```

Essa consulta retorna todas os professores que ministram disciplinas com carga de 60 horas ou a disciplina de Java III.

Cláusula LIKE

```
SELECT DISTINCT a
FROM Aluno a
WHERE a.nome LIKE 'A%'
```

- Retorna todos os alunos cujo nome começam com A.
- Os caracteres coringa que podem ser usados são:
 - ▣ %: qualquer quantidade de caracteres;
 - ▣ _: um caractere qualquer.

As verificações IS NULL e IS NOT NULL podem ser usadas para verificar a existência de valor em algum atributo, como no exemplo a seguir.

```
SELECT DISTINCT p
FROM Professor p
WHERE p.sala IS NULL
```

Essa consulta retorna todos os professores que não possuem sala (que no nosso caso é nenhum). Nesse caso, se for uma associação, está verificando se há o relacionamento entre as duas entidades. Deve-se tomar cuidado, pois não é equivalente a comparar `p.sala = NULL`, visto que pelo padrão SQL ANSI, NULL é um valor desconhecido e, portanto NULL não é igual a NULL, sendo sempre falso.

Em caso de coleções, para verificar se há valores ou não usa-se EMPTY (NOT EMPTY), como no exemplo a seguir.

```
SELECT DISTINCT p
FROM Professor p
WHERE p.disciplinas IS NOT EMPTY
```

Essa consulta retorna todos os professores que possuem disciplinas (uma ou mais). O atributo disciplinas é uma coleção.

As comparações de datas são análogas às feitas em SQL, usando BETWEEN e NOT BETWEEN.

```
SELECT DISTINCT p
FROM Professor p
WHERE p.dataAdmissao BETWEEN :data1 AND :data2
```

Retorna todos professores cuja data de admissão estão entre data1 e data2. É equivalente a comparar:

```
p.dataAdmissao >= data1 AND p.dataAdmissao <= data2
```

SQL ANSI - ISO/IEC 9075:2011: http://www.iso.org/iso/iso_catalogue/catalogue_tc/catalogue_detail.htm?csnumber=53681



Para executar esta query, podemos usar o trecho de código a seguir.

```
Query query = session.createQuery("SELECT DISTINCT p FROM Professor p WHERE  
p.dataAdmissao BETWEEN :data1 AND :data2");  
query.setDate("data1",  
new SimpleDateFormat("dd/MM/yyyy").parse("03/03/2009"));  
query.setDate("data2",  
new SimpleDateFormat("dd/MM/yyyy").parse("06/06/2009"));  
List<Professor> resultado = query.list();
```

Atualizações podem ser feitas usando-se UPDATE e DELETE, como nos exemplos a seguir.

```
UPDATE Disciplina d  
SET d.carga = 100  
WHERE d.carga = 60
```

Esse comando altera a carga horária de todas as disciplinas que possuem 60 horas para 100 horas. Para executar esta query, usa-se o seguinte trecho de código:

```
Query query = session.createQuery("UPDATE Disciplina d SET d.carga = 100 WHERE  
d.carga = 60");  
int rows = query.executeUpdate();
```

Onde o retorno do método query.executeUpdate() é a quantidade de linhas afetadas pelo comando. A seguinte query executa uma deleção.

```
DELETE  
FROM Sala s  
WHERE s.numero = 200
```

Esse comando remove a sala 200.

Joins

Os Joins são usados para se obter, selecionar, restringir etc. elementos que estão associados a um objeto. Por exemplo, a classe Disciplina possui um Professor que a ministra e a classe Professor possui uma coleção de Disciplinas ministradas por ele. Para retornar todas as disciplinas de um determinado professor, precisa-se fazer uma consulta na classe Disciplina, restringindo-a de modo a retornar somente as que são do Professor. Isso é feito unindo a Disciplina com seu Professor, através da palavra reservada JOIN, como no exemplo a seguir:

```
SELECT d  
FROM Disciplina d JOIN d.professor p  
WHERE p.nome = 'Pedro'
```

Esse exemplo retorna todas as disciplinas ministradas pelo professor Pedro. O comando INNER é opcional, e a consulta acima equivale a:

```
SELECT d  
FROM Disciplina d INNER JOIN d.professor p  
WHERE p.nome = 'Pedro'
```


Ambas equivalem a:

```
SELECT d
FROM Disciplina d, IN(d.professor) p
WHERE p.nome = 'Pedro'
```

Os Joins podem ser de forma explícita e implícita também. As explícitas identificando via palavra JOIN, como no exemplo a seguir, que retorna todas as disciplinas do professor Pedro:

```
SELECT d
FROM Disciplina d JOIN d.professor p
WHERE p.nome = 'Pedro'
```

E implícitas deixando o Hibernate resolver o JOIN:

```
SELECT d
FROM Disciplina d
WHERE d.professor.nome = 'Pedro'
```

Os LEFT JOINS são usados para retornar elementos mesmo que não haja relacionamento entre eles. Usa-se as palavras LEFT JOIN e LEFT OUTER JOIN (outer é opcional).

```
SELECT p.nome, d.nome
FROM Professor p LEFT JOIN p.disciplinas d
```

Retorna todos os pares Professor e Disciplina, mesmo que não haja disciplina atrelada ao Professor. Para executar esta query, podemos usar o trecho a seguir.

```
Query query = session.createQuery("SELECT p.nome, d.nome FROM Professor p LEFT JOIN
p.disciplinas d");
List<Object[]> resultado = query.list();
for (Object[] o: resultado) {
    System.out.println(o[0] + ": " + o[1]);
}
```

O retorno de dois dados (p.nome, d.nome) será discutido adiante, na subseção sobre Cláusula SELECT.

Quando em uma associação, por exemplo: Professor x Disciplina, ao se buscar pela disciplina, seu professor não é retornado automaticamente. Somente é consultado quando sua propriedade for acessada (Lazy).

Lazy Fetch:

- Associação Professor x Disciplina;
- Ao buscar Disciplina, NÃO retorna Professor;
- Somente quando uma propriedade do professor é acessada;
- Para evitar isso, ié, sempre buscar os dados da associação (evitar Lazy), faz-se:

```
SELECT d
FROM Disciplina d JOIN FETCH d.professor professor
```

Efeito colateral:

- Traz todos os dados das entidades relacionadas;
- Mesmo que não estejam explicitamente especificados no SELECT.



Cláusula SELECT

Quando não se usa SELECT, buscamos o objeto como um todo. Ao usar SELECT, podemos indicar o objeto que se quer recuperar ou podemos buscar somente algumas propriedades desse objeto. Nos exemplos anteriores, sempre buscava-se um objeto de algum tipo, por exemplo:

```
SELECT d
FROM Disciplina d
WHERE d.professor.nome = 'Pedro'
```

A cláusula SELECT indica que o resultado da query será um ou mais objetos do tipo Disciplina. Se a query não possuir a cláusula SELECT, como no exemplo a seguir.

```
FROM Disciplina d
WHERE d.professor.nome = 'Pedro'
```

O retorno será um objeto ou coleção do tipo Object. Os exemplos a seguir são usados para retornar somente uma parte dos dados dos objetos ou, como no caso do MAX(), uma função.

```
SELECT p.nome
FROM Professor p
SELECT max(d.carga)
FROM Disciplina d
SELECT d.nome, d.carga
FROM Disciplina d
```

No primeiro caso é retornada uma Lista de nomes (somente o nome) de todos os professores. Já no segundo caso é retornado um número, o valor máximo das cargas horárias das disciplinas e no terceiro caso é retornado uma Lista de Object[] (uma lista de arrays de objetos), onde:

- O primeiro elemento do array é o nome da disciplina;
- O segundo elemento do array é a carga horária da disciplina.

Outra maneira de restringir o retorno é usando o DISTINCT, que já foi usado em outras consultas vistas anteriormente. Ele é usado para não retornar resultados duplicados. Por exemplo, a consulta a seguir retorna as disciplinas cursadas por alunos, mas retorna elementos duplicados.

```
SELECT d
FROM Disciplina d JOIN d.alunos a
```

Para retornar elementos únicos, usa-se:

```
SELECT DISTINCT d
FROM Disciplina d JOIN d.alunos a
```

Ordenação com Order By

Para ordenar os elementos retornados, usa-se a cláusula ORDER BY, como no exemplo a seguir.

```
SELECT a
FROM Aluno a
ORDER BY a.nome
```

Esse exemplo retorna todos os alunos, ordenando-os por nome. Podemos ordenar os resultados de forma ascendente ou decendente:

- **Ascendente:** usa-se ORDER BY atributo ASC, é a ordenação default;
- **Descendente:** usa-se ORDER BY atributo DESC, é a ordenação default.

No exemplo a seguir, a ordenação dos alunos será de forma descendente.

```
SELECT a
FROM Aluno a
ORDER BY a.nome DESC
```

Cláusulas Group By e Having

A cláusula Group By agrupa os elementos retornados. Por exemplo, a consulta:

```
SELECT p.nome, COUNT(d)
FROM Professor p JOIN p.disciplinas d
GROUP BY p.nome
```

apresenta a lista de professores e a quantidade de disciplinas. Se for usado LEFT JOIN apresenta também a lista de professores com nenhuma disciplina. HAVING serve para restringir a ação do GROUP BY.

```
SELECT p.nome, AVG(d.carga)
FROM Professor p JOIN p.disciplinas d
GROUP BY p.nome
HAVING AVG(d.carga) >= 40
```

No exemplo, a consulta agrupa os professores retornando a média da carga horária de cada um, considerando somente os professores com carga horária maior ou igual a 40 horas.

Criteria

Criteria é uma API para criação de queries que podem ser verificadas em tempo de compilação. Ao contrário do HQL, cujas queries são verificadas só em tempo de execução.

Outro benefício importante é quando se necessita construir queries de forma programática. em vez de construir uma String em HQL, cria-se um Criteria.

- API para criação de queries que podem ser verificadas em tempo de compilação
 - HQL só em tempo de execução
 - Construção de queries de forma programática
- Objeto Criteria
 - Interface da API



Um exemplo de Criteria pode ser visto a seguir.

```
Criteria criteria = session.createCriteria(Professor.class);
List<Professor> lista = criteria.list();
```

Esse exemplo cria um Criteria para a classe Professor e retorna a lista completa de professores. Um laço de exemplo para mostrar os resultados seria:

```
for (Professor o: lista) {
    System.out.println( o.getNome() );
}
```

A tabela a seguir apresenta alguns métodos de Criteria.

Método	Descrição
add()	Adiciona restrições à consulta
addOrder()	Adicionar uma ordenação (ascendente ou decrescente à consulta)
createCriteria()	Cria um novo criteria para se fazer associações
list()	Retorna uma lista de resultados
setFirstResult()	Limita os dados retornados, indicando qual é o primeiro dado a ser retornado. Usado em paginações
setMaxResult()	Limita os dados retornados, indicando quando devem ser retornados. Usado em paginações
setProjection()	Adiciona uma projeção à consulta
uniqueResult()	Retorna somente um resultado

Para paginação de resultados e limitar a quantidade de elementos retornados, usa-se `setMaxResult()` e `setFirstResult()`, como no exemplo a seguir:

```
Criteria criteria = session.createCriteria(Aluno.class);
criteria.setMaxResults(5);
criteria.setFirstResult(10);
List<Aluno> lista = criteria.list();
```

Tabela 9.1
Alguns métodos
de Criteria.

Ordenação

Para se fazer ordenação, usa-se o método `addOrder()` do objeto `criteria`. Podemos ter uma ordenação crescente (`Order.asc()`) e decrescente (`Order.desc()`). O parâmetro é o nome do atributo da classe pelo qual se quer ordenar os resultados. No exemplo a seguir, temos a ordenação crescente:

```
Criteria criteria = session.createCriteria(Aluno.class);
criteria.addOrder( Order.asc("nome") );
List<Aluno> lista = criteria.list();
```

E para decrescente:

```
Criteria criteria = session.createCriteria(Aluno.class);
criteria.addOrder( Order.desc("nome") );
List<Aluno> lista = criteria.list();
```

Restrições

As restrições feitas na cláusula `WHERE` são criadas com `Restrictions`. As de comparação simples de valores são:

- ▣ **Restrictions.eq:** verifica se um campo é igual a um valor;
- ▣ **Restrictions.lt:** verifica se um campo é menor que um valor;
- ▣ **Restrictions.le:** verifica se um campo é menor ou igual a um valor;
- ▣ **Restrictions.gt:** verifica se um campo é maior a um valor;
- ▣ **Restrictions.ge:** verifica se um campo é maior ou igual a um valor;
- ▣ **Restrictions.like:** verifica se um campo é parecido com um valor (caracteres coringa).



Como nos exemplos:

```
Criteria criteria = session.createCriteria(Disciplina.class);
criteria.add( Restrictions.gt("carga", 40) );
List<Disciplina> lista = criteria.list();

Criteria criteria = session.createCriteria(Disciplina.class);
criteria.add( Restrictions.like("nome", "Java%") );
List<Disciplina> lista = criteria.list();
```

A primeira consulta retorna todas as disciplinas com carga maior que 40. A segunda retorna todas as disciplinas cujo nome inicia com a string "Java".

Para retornar somente um resultado, usa-se `uniqueResult()`, como no exemplo a seguir:

```
Criteria criteria = session.createCriteria(Disciplina.class);
criteria.add( Restrictions.eq("nome", "Java III") );
Disciplina d = (Disciplina)criteria.uniqueResult();
```

Nesse exemplo, é criado um `Criteria` para a classe `Disciplina` e retornada uma disciplina cujo nome é Java III.

Outras restrições (WHERE) também podem ser criadas, como:

- **Restrictions.between:** restrição Between;
- **Restrictions.isNull:** verifica se é nulo;
- **Restrictions.isNotNull:** verifica se não é nulo;
- **Restrictions.isEmpty:** verifica se o valor é vazio;
- **Restrictions.isNotEmpty:** verifica se o valor não é vazio.



Os exemplos a seguir mostram seu uso.

```
Criteria criteria = session.createCriteria(Professor.class);
criteria.add( Restrictions.between("dataAdmissao",
    new SimpleDateFormat("dd/MM/yyyy").parse("03/03/2009"),
    new SimpleDateFormat("dd/MM/yyyy").parse("04/04/2009") ) );
List<Professor> lista = criteria.list();

Criteria criteria = session.createCriteria(Professor.class);
criteria.add( Restrictions.isNull("dataAdmissao") );
List<Professor> lista = criteria.list();

Criteria criteria = session.createCriteria(Professor.class);
criteria.add( Restrictions.isNotNull("dataAdmissao") );
List<Professor> lista = criteria.list();
```

No caso da primeira consulta, deve-se colocar os comandos dentro de um `try...catch` por causa da conversão de datas.

Restringir a consulta por um valor em uma classe associada:

- Cria-se um criteria a partir do criteria principal.

Exemplo:

- Quando se quer retornar as Disciplinas, cria-se um criteria para `Disciplina.class`;
- Para restringir por Professor da disciplina, cria-se um criteria a partir do atributo "professor";
- Faz-se a restrição nesse último criteria criado.

O exemplo a seguir retorna todas as disciplinas do professor "Pedro". Para tal, cria-se um Criteria principal para a classe `Disciplina`. Logo após, um critério é criado a partir deste, para o atributo "professor". Sobre esse último é feita a adição da restrição.

```
Criteria criteria = session.createCriteria(Disciplina.class);
Criteria critProfessor = criteria.createCriteria("professor");
critProfessor.add( Restrictions.eq("nome", "Pedro") );
List<Disciplina> lista = criteria.list();
```

Combinação de Restrições: AND e OR

As restrições podem ser combinadas em cláusulas do tipo AND (E) e OR (OU). Nas do tipo AND, para que um dado seja retornado, todas as restrições devem ser verdadeiras. Nas do tipo OR, um dado é retornado se pelo menos uma das restrições é verdadeira.

A combinação de restrições é feita através de objetos do tipo `LogicalExpression`, que são criados através dos métodos `Restrictions.or()` e `Restrictions.and()`. Para compor as restrições, cria-se objetos do tipo `Criterion` usando-se as restrições vistas na subseção anterior.

Os elementos são:

- **LogicalExpression:** é a combinação de restrições do tipo `Criterion`;
- **Criterion:** é uma restrição específica;
- **Restrictions.or():** retorna um `LogicalExpression` do tipo AND;
- **Restrictions.and():** retorna um `LogicalExpression` do tipo AND.

No exemplo a seguir, serão retornadas todas as disciplinas cujo nome iniciam com "Java" e possuem carga igual a 60 horas.

```
Criteria criteria = session.createCriteria(Disciplina.class);
Criterion restrNome = Restrictions.like("nome", "Java%");
Criterion restrCarga = Restrictions.eq("carga", 60);
LogicalExpression restr = Restrictions.and(restrNome, restrCarga);
criteria.add(restr);
List<Disciplina> resultado = criteria.list();
```

No exemplo a seguir, serão retornadas todas as salas de número maior que 202 ou que não possuam ar-condicionado.

```
Criteria criteria = session.createCriteria(Sala.class);
Criterion restrNr = Restrictions.gt("numero", 202);
Criterion restrAr = Restrictions.eq("arCondicionado", false);
LogicalExpression restr = Restrictions.or(restrNr, restrAr);
criteria.add(restr);
List<Sala> resultado = criteria.list();
```

Projeções

Projeções são restrições ou agregações feitas na cláusula SELECT. É usado para retornar somente alguns atributos ou fazer funções agregadoras. Usa-se `setProjection()` e elementos de `Projections` para isso. No exemplo a seguir, retorna-se a quantidade de elementos de `Aluno` (`rowCount()`). O resultado nesse caso deve ser obtido com `uniqueResult()`, que retorna um `Long`.

```
Criteria criteria = session.createCriteria(Aluno.class);
criteria.setProjection( Projections.rowCount() );
Long r = (Long)criteria.uniqueResult();
```

Para se adicionar mais de uma projeção, usa-se um `ProjectionList`, criado a partir da chamada do método `Projections.projectionList()`. Nesse objeto, usa-se o método `add()` para adicionar todas as projeções necessárias. Ao final, configuramos o `ProjectionList` no objeto `Criteria`, que retornará os dados.

No exemplo a seguir, é montada uma lista de projeções, contendo a menor carga horária (`min("carga")`) e a maior carga horária (`max("carga")`) das Disciplinas.

```
Criteria criteria = session.createCriteria(Disciplina.class);
ProjectionList proj = Projections.projectionList();
proj.add(Projections.min("carga"));
proj.add(Projections.max("carga"));
criteria.setProjection( proj );
Object[] dados = (Object[])criteria.uniqueResult();
```

Como esta consulta retorna somente duas informações, usam-se as posições do vetor para mostrar os resultados, como a seguir.

```
System.out.println( dados[0] + " - " + dados[1] );
```

JSF e Hibernate

As maneiras de se integrar o Hibernate em aplicações JSF são muitas e dependem muito da arquitetura da aplicação que se deseja, dos requisitos do cliente etc.

Levando em conta que uma aplicação JSF é composta, basicamente, por duas partes:

- **XHTML**: páginas a serem apresentadas ao usuário;
- **Managed Beans**: componentes que tratam as requisições das páginas.

Deve-se adicionar um terceiro elemento, que são as classes persistentes para o Hibernate. No nosso caso, as classes `Sala`, `Professor`, `Disciplina` e `Aluno`, que farão parte do modelo de acesso ao banco de dados e vão ser responsáveis por manter os dados persistidos.

Uma forma simples de integração (sem pensar em conceitos como reuso, manutenibilidade e modularidade) é inserir o código de acesso ao banco, no nosso caso do Hibernate, diretamente em métodos dos Managed Beans. Assim, quando o cliente interage com a aplicação, o Managed Bean já pode fazer os acessos ao banco de dados de forma pontual.

Integração JSF e Hibernate

Aplicação JSF é composta por:

- XHTML.
- Managed Beans.

Adicionar mais um element.

- Classes Persistentes.
- Exemplo: sala, Professor, Disciplina e Aluno.

Várias formas de integração.

- Integração simples: código de acesso ao banco (Hibernate) diretamente em métodos dos MBs.

Para que um objeto persistente esteja preenchido para ser usado com o Hibernate, usa-se uma instância desse no Managed Bean e faz-se o XHTML ligar seus campos de formulário a atributos desse objeto. No exemplo a seguir, é apresentado o XHTML e o Managed Bean usado em um cadastro de Aluno.

XHTML – Busca de Aluno:

```
<h:form>
  <h:inputText value="#{exemploMB.aluno.nome}" />
  <h:commandButton value="Salvar"
                    action="#{exemploMB.salvarAluno()}" />
</h:form>
```

Managed Bean: busca de Aluno:

```
@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
    private Aluno aluno = new Aluno();
    public ExemploMB() {
    }
    public Aluno getAluno() {
        return aluno;
    }
    public void setAluno(Aluno aluno) {
        this.aluno = aluno;
    }
    public void salvarAluno() {
        Session session = HibernateUtil.getSessionFactory()

        .openSession();
        session.beginTransaction();
        session.save(aluno);
        session.getTransaction().commit();
    }
}
```


Percebe-se nesse exemplo que o próprio JSF faz o preenchimento do bean Aluno, pela ligação do XHTML com o Managed Bean. Assim, o método salvarAluno() do MB precisa somente chamar o método save(), do Hibernate.

Para se pesquisar um aluno, criam-se dois arquivos XHTML, um para entrada do texto de pesquisa, outro para listagem dos dados retornados. Além disso, criam-se dois atributos no Managed Bean:

- **texto:** (String) para conter o texto da pesquisa;
- **lista:** (List<Aluno>) para conter o resultado da pesquisa.

Os XHTML podem ficar como a seguir. O primeiro é um formulário para entrada do critério de pesquisa. Contém um texto que será usado no HQL de busca.

```
<h:form>
    Texto: <h:inputText value="#{exemploMB.texto}" /> <br/>
    <h:commandButton value="Pesquisar"
        action="#{exemploMB.pesquisarAluno()}" />
</h:form>
```

O segundo arquivo contém um dataTable para mostrar os dados retornados, que estão armazenados no atributo lista do MB.

```
<h:dataTable var="a" value="#{exemploMB.lista}">
    <h:column>
        <f:facet name="header"> Nome </f:facet>
        <h:outputText value="#{a.nome}" />
    </h:column>
</h:dataTable>
```

A nova versão do Managed Bean pode ser vista a seguir, com os novos atributos e o método pesquisarAluno().

```
@Named(value = "exemploMB")
@RequestScoped
public class ExemploMB {
    private Aluno aluno = new Aluno();
    private String texto = "";
    private List<Aluno> lista = new ArrayList<Aluno>();
    public ExemploMB() {
    }
    public Aluno getAluno() {
        return aluno;
    }
    public void setAluno(Aluno aluno) {
        this.aluno = aluno;
    }
    public String getTexto() {
        return texto;
    }
    public void setTexto(String texto) {
        this.texto = texto;
    }
}
```

```

public List<Aluno> getLista() {
    return lista;
}
public void setLista(List<Aluno> lista) {
    this.lista = lista;
}
public void salvarAluno() {
    Session session = HibernateUtil.getSessionFactory().
                                openSession();

    session.beginTransaction();
    session.save(aluno);
    session.getTransaction().commit();
    FacesMessage fm = new FacesMessage("Aluno salvo com sucesso.");
    FacesContext.getCurrentInstance().addMessage(null, fm);
}
public String pesquisarAluno() {
    Session session = HibernateUtil.getSessionFactory().
                                openSession();

    Query query = session.createQuery("SELECT a FROM Aluno a WHERE a.nome like
:criterio ");
    query.setParameter("criterio", "%" + this.texto + "%");
    lista = query.list();

    return "list_aluno";
}
}

```

10

Java EE

objetivos

Conhecer conceitos avançados em Java EE e algumas tecnologias disponíveis para o desenvolvimento corporativo.

Objetos distribuídos; EJB; Singleton; MDB; JMS; Transações; JTA; CMT; BMT; web Services; SOAP; REST; SOA; WSDL; UDDI; JSON; JAXB e ESB.

conceitos

Java EE

O desenvolvimento de sistemas computacionais é baseado em requisitos. Esses requisitos podem ser de dois tipos:

Requisitos funcionais:

- Entradas, seu processamento (regras de negócio) e suas saídas.

Requisitos não funcionais:

- Dizem respeito à qualidade do software, orçamento, segurança, usabilidade, disponibilidade, tecnologias etc.;
- Persistência em BD, Transações, Segurança, Threads, Paralelismo etc.;
- Infraestrutura.

Implementar tudo, do zero, é complicado, demorado, custoso etc.

O desenvolvimento dos requisitos funcionais demanda muito esforço e é a parte do software que realmente vai aderir ao processo de negócio do cliente, sendo extremamente com foco aos seus objetivos.

Já os requisitos não funcionais dizem respeito a como o sistema desempenhará suas funcionalidades. Para alcançar esses objetivos, é preciso pensar em uma arquitetura para o sistema. Muitos desses requisitos podem ser modelados como soluções (ou infraestruturas), podendo ser facilmente reutilizados ou replicados em vários projetos. Por exemplo: persistência. Podemos implementar uma infraestrutura de persistência que pode ser reutilizada de forma configurável em outros projetos.

O desafio está, para o desenvolvimento do projeto, em a equipe ter como tarefas não só o desenvolvimento dos requisitos funcionais, mas também toda a infraestrutura que atende aos requisitos não funcionais. Assim, surge o Java EE, como uma plataforma para facilitar a implementação de vários requisitos não funcionais.





O Java EE é:

- Um conjunto de especificações.
- Caminho sobre como implementar um software.
- Uma descrição do uso de serviços de infraestrutura já implementados.

Com Java EE, é possível alterar a tecnologia de infraestrutura, sem maiores modificações no software.

- Baixo acoplamento.

O Java EE é um conjunto de especificações de várias tecnologias que dão suporte ao desenvolvimento de sistemas corporativos com requisitos não funcionais complexos. Também é uma descrição de uso de serviços de infraestrutura já implementados, dos quais o programador pode se beneficiar ao utilizá-los em seus projetos.

Com Java EE, é possível alterar a tecnologia e, muitas vezes, o comportamento da infraestrutura sem muitas modificações nas partes de negócio do software, levando a um baixo acoplamento.

O Java EE possui especificações para:

- Desenvolvimento web (Servlets, JSP, JSF);
- Objetos Distribuídos (EJB);
- Objetos Persistentes (**JPA**);
- Serviços na web (web Services);
- Segurança (JAAS);
- Transações (JTA);
- Etc.



JPA

É uma tecnologia do Java EE que foi baseada no Hibernate. Hoje, JPA é uma especificação e Hibernate é uma implementação do JPA.

Objetos distribuídos

Objetos distribuídos em Java são implementados através de EJBs (Enterprise Java Beans).

Usando EJB, é possível usar objetos distribuídos para o desenvolvimento de aplicações que possuem esse requisito não funcional.

Benefícios de EJB (Enterprise Java Beans):

- Uso de transações: integrado com JTA (Java Transaction API), inclusive com suporte a transações remotas;
- Segurança: autenticação e autorização de fora transparente com JAAS (Java Authentication and Authorization Service);
- Objetos remotos: a possibilidade de distribuir objetos em vários servidores e fazê-los cooperar;
- Concorrência: aplicações podem ser acessadas por múltiplos usuários, de forma controlada;
- Persistência: integração com **JPA** (Java Persistence API);
- Integração: é integrada com outras tecnologias Java, como, por exemplo, JSF.

São duas classes de EJBs divididas em quatro tipos:



Beans de Sessão (Session Beans)

- @Stateless, que não guardam estado entre as requisições.
- @Stateful, que guardam estado entre as requisições.
- @Singleton para as quais há somente uma instância no sistema todo.
- Beans de Mensagem.
- @MessageDriven, que são baseadas em mensagens assíncronas.

Os Session Beans são componentes EJB criados a partir de classes POJO, com anotações específicas para marcar seu comportamento. São usadas basicamente para representar regras de negócio, e possuem métodos públicos, que são considerados métodos de negócio. São administradas pelo EJB Container do Servidor Java EE.

Stateless Session Beans

- Usados para implementar regras de negócio.
- Podem ser acessados de forma local ou remota.
- Não armazenam informações entre uma requisição e outra.
- A operação tem de ser terminada em uma só chamada.
- Usamos a anotação @Stateless.

Exemplo:

- Consultar dados de um usuário.
- Efetuar uma inserção.

Os Stateless Session Beans são usados para representar regras de negócio, e podem ser acessados de forma local ou remota. Sua grande característica é que não armazenam informações entre uma requisição e outra, portanto, a tarefa designada precisa terminar em uma só chamada. Para criá-los, basta uma classe POJO anotada com @Stateless.

Local é seu comportamento padrão. Para anotá-lo, explicitamente podemos utilizar:

```
@Local
```

Ou podemos criar uma interface para indicar seus métodos de negócio e anotá-la como:

```
@Local(NomeDaInterface.class)
```

Para ser remoto, é obrigatória a criação de uma interface para usar a anotação:

```
@Remote(NomeDaInterface.class)
```

Quando o Bean é Local:

- Não precisa criar uma interface com os métodos;
- Se criar, deve-se anotar com:
 - @Local(NomeDaInterface.class)

Quando o Bean é Remoto:

- Acessível localmente e de outras máquinas
- É necessário criar uma interface com os métodos
- Deve-se anotar com:
 - @Remote(NomeDaInterface.class)

A anotação `@Stateless` é obrigatória, tanto com interface local como remote. Por exemplo:

```
@Stateless
public class BibliotecaBean {
    public int somar(int a, int b) {
        return a + b;
    }
    public int subtrair(int a, int b) {
        return a - b;
    }
}
```

- Usa-se a anotação `@Inject` para injetar o Bean;
- O container injeta a dependência correta;
- Depois, basta usar o objeto injetado;
- Podemos usar em:
 - ▣ Servlet;
 - ▣ `ManagedBean`;
 - ▣ Bean;
 - ▣ Pojo.

Para usar um Stateless Session Bean, no componente que for utilizá-lo (Servlet, `Managed Bean`, Bean etc.), usa-se a anotação `@Inject` para que o Container injete uma instância do EJB automaticamente. Depois, basta usar a instância injetada. Um exemplo de cliente local pode ser visto a seguir:

```
@Named
public class CalculosMB {
    @Inject
    private Biblioteca biblioteca;
    private int resultado;
    public void calcular() {
        this.resultado = this.biblioteca.somar(10, 20);
    }
}
```

Como todos os Session Beans, o Stateless também possui um ciclo de vida gerenciado pelo container.

Os Stateless Session Beans possuem só dois estados:

- Não existente: não pode atender requisições, pois ainda não foi criado;
- PRONTO: já foi criado e pode atender requisições.

Dependendo da quantidade de requisições, o container pode criar novas instâncias para respondê-las. Quando uma chamada é realizada, uma das instâncias que se encontram em estado PRONTO é selecionada para atender à requisição. Quando um Bean está atendendo uma requisição, não pode atender a outras. Ao terminar de atender uma requisição, volta ao estado pronto e pode atender outras requisições e, conforme critérios do servidor, ele pode destruir instâncias dos beans.

Stateful Session Beans

- Usado para implementar regras de negócio.
- Mantém o estado conversacional.
- Não podem atender a qualquer cliente, pois devem manter os dados (seu estado) por cliente.
- Usa-se a anotação `@Stateful`.
- Podem ser acessados de forma local e remota.

Exemplo:

- Carrinho de compras.

Os Stateful Session Beans também são usados para implementar regras de negócio, mas possuem uma grande diferença: eles mantêm dados entre uma requisição do cliente e outra. Assim, não podem atender a qualquer requisição, e o container deve controlar de qual cliente o EJB mantém os dados.

Para criá-los, usamos a anotação `@Stateful` e também podem ser locais ou remotos.

```
@Stateful
public class CarrinhoBean {
    private Set<Produto> produtos =
        new HashSet<Produto>();
    public void adicionar(Produto p) {
        this.produtos.add(p);
    }
    public void remover(Produto p) {
        this.produtos.remove(p);
    }
}
```

Um Stateful Session Bean pode estar em um dos três estados:

- Não existente: não pode responder requisições, pois ainda não foi criado;
- PRONTO: já instanciado e pode atender requisições;
- PASSIVADO: está atendendo requisições de um cliente, mas ficou muito tempo sem receber requisições; assim, pode ser armazenado em disco.

O cliente é responsável por indicar quando aquela instância não é mais útil, e faz isso chamando algum método do Bean que esteja marcado com `@Remove`. Por exemplo, no código a seguir, uma chamada ao método `finalizar()`, independente do seu conteúdo, fará com que, no final de sua execução, a instância do bean seja descartada.

```
@Stateful
public class CarrinhoBean {
    private Set<Produto> produtos =
        new HashSet<Produto>();
    public void adicionar(Produto p) {
        this.produtos.add(p);
    }
    public void remover(Produto p) {
```

```

        this.produtos.remove(p);
    }
    @Remove
    public void finalizar() {
    }
}

```

Singleton Session Beans

- EJB 3.1+.
- O Container EJB só cria uma instância do Bean.
- Compartilhar dados transientes entre todos os usuários da aplicação.
- Usa-se a anotação @Singleton.
- Pode ser local e remota.

Exemplo:

- Contagem de usuários.
- Cache.
- Dados compartilhados.

Um Singleton Session Bean é um EJB que é instanciado somente uma vez e, portanto, compartilha seus dados entre todos os usuários da aplicação. Surgiu com a especificação EJB 3.1. Para criá-lo, anota-se uma classe com @Singleton, e sua interface, também, pode ser local ou remota.

```

@Singleton
public class ContadorBean {
    private int contador = 0;
    public void adicionar() {
        this.contador++;
    }
    public int getContador() {
        return this.contador;
    }
}

```

O Singleton pode estar em um dos dois estados:

- Não existente: não possui instância criada, portanto não pode atender requisições;
- PRONTO: quando já foi criado e pode atender requisições.

Singletons são componentes usados para acesso concorrente entre vários elementos da aplicação. Assim, a maneira de sincronizar a chamada aos seus métodos pode ser indicada pelo desenvolvedor. Se nada for especificado, conforme o exemplo anterior, a concorrência é controlada pelo Container e o acesso aos métodos é bloqueado, caso algum outro elemento já esteja executando algum de seus métodos.

Message-Driven Beans

- Bean com a capacidade de processar mensagens de forma assíncrona.
- Classe anotada com @MessageDriven.
- Atua como um JMS Listener:
 - JMS – Java Messaging System.
 - Recebe e consome mensagens JMS.
 - Processa as mensagens.

Características:

- Assincronia na sua invocação: um cliente manda uma mensagem e continua seu processamento.
- Segurança, pois há a garantia de entrega da mensagem.
- Baixo acoplamento entre componentes da aplicação.

Os Message-Driven Beans (MDB) são componentes capazes de lidar com mensagens e de serem invocados de forma assíncrona. Para criá-los, deve-se anotar uma classe com @MessageDriven. JMS (Java Message System) é a API do Java voltado para mensagens e, portanto, os MDBs atuam como listeners JMS, recebendo, consumindo e processando mensagens.

A arquitetura JMS é composta pelas seguintes partes:

- JMS Provider: uma implementação do Java EE que suporte JMS;
- JMS Clients: programas ou componentes que produzem e consomem mensagens;
- Mensagens: objetos de comunicação entre dois JMS Clients;
- Objetos Administrados: objetos JMS pré-configurados para uso dos clientes – destinos e fábricas de conexões (connection factories).

Temos dois estilos de envio de mensagens.

Ponto-a-Ponto (Point-to-Point):

- Emissores, Receptores e Filas;
- Cada mensagem tem somente um consumidor;
- Cada mensagem é endereçada a uma fila específica;
- Receptores retiram suas mensagens de filas específicas;
- Filas guardam todas as mensagens até que sejam consumidas ou que expirem.

Publicação/Inscrição (Publish/Subscribe):

- Emissores enviam mensagens para um tópico;
- Um tópico pode ter vários assinantes;
- Beans podem se inscrever (assinar) tópicos;
- Tópicos guardam as mensagens até que sejam consumidas;
- O Bean precisa estar ativo para consumir as mensagens do tópico.



Mensagens podem ser consumidas de duas formas:

Sincronamente:

- Consumidor explicitamente obtém uma mensagem invocando `receive()`;
- `receive()` bloqueia até que a mensagem seja obtida ou dá time-out se a mensagem não chegar em um determinado espaço de tempo.

Assincronamente:

- Uma aplicação pode registrar um `Message Listener` como um consumidor de mensagens;
- Sempre que uma mensagem chega, o método `onMessage()` do listener é invocado;
- No JavaEE, os MDBs funcionam como `Message Listeners`.

Os MDBs podem estar em dois estados: NÃO EXISTENTE e PRONTO, quando podem processar mensagens e, assim que uma mensagem chega, o método `onMessage()` é invocado.

Ciclo de Vida – dois Estados:

- NÃO EXISTENTE:
 - MDB ainda não foi instanciado e não pode responder requisições ou mensagens.
- PRONTO:
 - Pode responder requisições ou mensagens.
 - Quando uma mensagem chega, executa o método `onMessage()`

A seguir, exemplo de um MDB.

```
@MessageDriven
public class MensagemBean
    implements MessageListener {
    public void onMessage(Message msg) {
    }
}
```

Transações

- Execução de várias tarefas diferentes.
- Separar uma tarefa em passos menores.
 - Devem ser executados em sequência.
 - Se com sucesso, a tarefa como um todo é realizada.
 - Se algum passo falhar, a tarefa como um todo é abortada.
- JTA (Java Transaction API) habilita transações distribuídas.
 - Transações distribuídas: manipulam informações em vários recursos através da rede.
- Gerenciador de Transações (Transaction Manager).
 - Elemento crucial no JTA.
 - Decide se operações serão validadas ou abortadas, em um contexto distribuído.

Uma transação é a execução de várias tarefas diferentes que devem ser consolidadas ou abortadas como uma só. Em geral, uma tarefa complicada é separada em passos menores, e a execução desses passos configura uma transação.

JTA (Java Transaction API) habilita transações distribuídas, isto é, que manipulam informações em vários recursos através da rede. Basicamente, o gerenciador de transações (Transaction Manager), elemento crucial no JTA, tem como tarefa decidir se operações serão validadas ou abortadas, isso em um contexto distribuído.

! Os passos de uma transação devem ser executados em sequência. Se com sucesso, a tarefa como um todo é realizada, se algum passo falhar, a tarefa como um todo é abortada.

Java EE possui dois tipos disponíveis:

- CMT – Container Managed Transactions (default): controlada pelo servidor;
- BMT – Bean Managed Transactions: controlada pela aplicação.

CMT

- Servidor Abre, Confirma ou Aborta a transação.
- Para cada método, podemos definir qual seu comportamento em relação à transação corrente.

Exemplos:

- Sempre abrir uma nova transação, mesmo que já haja outra aberta.
- Usar a transação corrente ou abrir uma nova.
- Não usar transação

A seguir, um exemplo de EJB com transação:

```
@Stateful
public class TesteBean {
    @TransactionAttribute(TransactionAttributeType.REQUIRED)
    public void tarefa(String parâmetro){
    }
}
```

BMT

- A aplicação deve controlar a transação.
- Deve-se anotar o Bean indicando o tipo de transação usado.
- Usa-se métodos específicos para iniciar, confirmar ou abortar a transação.

O exemplo a seguir mostra um EJB com um método que, explicitamente, controla uma transação.

```
@Stateful
@TransactionManagement(TransactionManagementType.BEAN)
public class TesteBean {
    @Resource
    private UserTransaction ut;
    public void tarefa(String parâmetro) {
        try {
            ut.begin();
            // Tarefas
        }
    }
}
```

```

        ut.commit();
    }
    catch (TarefaException e) {
        ut.rollback();
    }
    catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

web Services

Um serviço que pode ser acessado diretamente por outro sistema de qualquer plataforma, que esteja na rede ou internet.

- Arquitetura:
- Cada plataforma tem suas ferramentas.
- W3C oferece alguns padrões (exemplo: SOAP).
- Java API for XML-Based web Services – JAX-WS.
 - ▣ Padrão W3C (<http://www.w3.org/TR/ws-arch/>).
 - ▣ Baseado em SOAP.
 - ▣ Depende de Java Architecture for XML Binding – JAXB.
- Java API for RESTful web Services – JAX-RS.
 - ▣ Representational State Transfer (REST).
 - ▣ Em geral mais fáceis de implementar e de evoluir.
 - ▣ Usa JAXB para produzir XML ou JSON.

Nos web Services baseados em SOAP, o tráfego dos dados é padronizado (XML), e várias especificações estão disponíveis para garantir qualidade, segurança, transações etc. É um padrão maduro no mercado, portanto, qualquer plataforma possui interfaces e facilidades para uso de SOAP.

Nos web Services baseados em REST, é usado o protocolo HTTP, portanto, não há necessidade de especificações adicionais, apresentando, geralmente, melhor desempenho. Para o tráfego de informações, podemos escolher a representação, sendo comumente utilizados XML ou JSON.

web Services baseados em SOAP:

- Tráfego de dados padronizado: xML;
- Várias especificações para garantir qualidade, segurança, transações;
- Padrão maduro no Mercado.

web Services baseados em REST:

- Tráfego sobre protocolo HTTP;
- Geralmente melhor desempenho;
- Tráfego de informações via XML ou JSON.

SOAP

Em web Services do tipo SOAP (Simple Object Access Protocol), toda a interação é feita via SOAP, que é um protocolo de troca de mensagens baseado em XML.

A figura 10.1 mostra os serviços de uma arquitetura SOAP e suas interações.

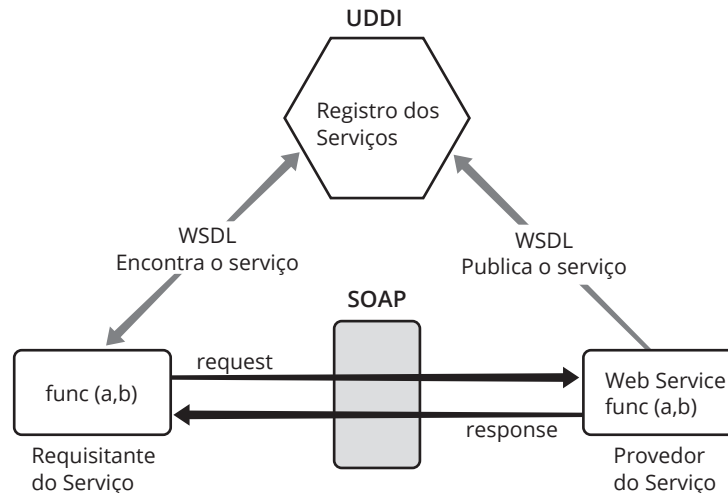


Figura 10.1
Serviços web
Service tipo SOAP.

Para saber mais,
acesse: <http://www.soa-manifesto.org/>

Os passos para uso são:

- Fornecedor de serviço contacta o UDDI, registrando e informando os serviços, e onde encontrá-los;
- Consumidor de um serviço utiliza métodos de busca oferecidos pelo UDDI para encontrar as informações que deseja sobre entidades e serviços;
- Consumidor de um serviço se comunica diretamente com o fornecedor do serviço.

Basicamente, SOAP usa XML sobre HTTP, sendo que todas as mensagens trocadas são encapsuladas em XML, com uma padronização. Assim, ao ser enviada uma mensagem SOAP, ela deve ser encapsulada em um XML específico. Ao ser recebida uma mensagem SOA, ela deve ser decodificada e interpretada, para que a ação seja efetivamente executada.

- SOAP usa XML sobre HTTP:
 - Todas as mensagens trocadas são encapsuladas em XML, com uma padronização.
- Podem ser descobertos (UDDI);
- Elementos.
 - SOAP – Simple Object Access Protocol;
 - UDDI – Universal Description, Discovery and Integration;
 - WSDL – web Service Description Language.

SOAP é um protocolo de comunicação que define um formato para envio de mensagens. Toda a comunicação sobre a internet via XML, portanto, independente de plataforma e independente de linguagem. É simples e extensível e, por ser um padrão W3C, é bem documentado.

A seguir um exemplo de solicitação SOAP.

```
POST /ServidorWS/TesteWS/endpoint HTTP/1.1
Host: localhost
Content-Type: text/xml; charset="utf-8"
Content-Length: 322
SOAPAction: ""
<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Header/>
  <S:Body>
    <ns2:mostrar xmlns:ns2="http://ws.com/">
      <str>oi</str>
    </ns2:mostrar>
  </S:Body>
</S:Envelope>
```

A seguir, um exemplo de resposta SOAP.

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset="utf-8"
Content-Length: 367
<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:mostrarResponse xmlns:ns2="http://ws.com/">
      <return>Teste: oi</return>
    </ns2:mostrarResponse>
  </S:Body>
</S:Envelope>
```

WSDL (web Services Description Language) é uma linguagem baseada em XML usada para descrever e localizar webServices.

WSDL

- web Services Description Language;
- Baseado em XML;
- Usado para descrever webServices;
- Usado para localizar webServices;
- É um Padrão W3C.

A seguir, um exemplo de WSDL.

```
<definitions xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd" xmlns:wsp="http://www.w3.org/ns/ws-policy"
xmlns:wsp_2="http://schemas.xmlsoap.org/ws/2004/09/policy" xmlns:wsam="http://www.w3.org/2007/05/addressing/metadata" xmlns:soap="http://schemas.xmlsoap.org/
wsdl/soap/" xmlns:tns="http://ws.com/" xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns="http://schemas.xmlsoap.org/wsdl/" targetNamespace="http://ws.com/"
name="TesteWS">
<types>
```



```

<xsd:schema>
  <xsd:import namespace="http://ws.com/" schemaLocation="http://localhost:8080/
ServidorWS/TesteWS?xsd=1"/>
</xsd:schema>
</types>
<message name="hello">
  <part name="parameters" element="tns:hello"/>
</message>
<message name="helloResponse">
  <part name="parameters" element="tns:helloResponse"/>
</message>
<message name="mostrar">
  <part name="parameters" element="tns:mostrar"/>
</message>
<message name="mostrarResponse">
  <part name="parameters" element="tns:mostrarResponse"/>
</message>
<portType name="TesteWS">
  <operation name="hello">
    <input wsam:Action="http://ws.com/TesteWS/helloRequest" message="tns:hello"/>
    <output wsam:Action="http://ws.com/TesteWS/helloResponse"
message="tns:helloResponse"/>
  </operation>
  <operation name="mostrar">
    <input wsam:Action="http://ws.com/TesteWS/mostrarRequest"
message="tns:mostrar"/>
    <output wsam:Action="http://ws.com/TesteWS/mostrarResponse"
message="tns:mostrarResponse"/>
  </operation>
</portType>
<binding name="TesteWSPortBinding" type="tns:TesteWS">
  <soap:binding transport="http://schemas.xmlsoap.org/soap/http"
style="document"/>
  <operation name="hello">
    <soap:operation soapAction=""/>
    <input>
      <soap:body use="literal"/>
    </input>
    <output>
      <soap:body use="literal"/>
    </output>
  </operation>
  <operation name="mostrar">
    <soap:operation soapAction=""/>
    <input>
      <soap:body use="literal"/>
    </input>
    <output>

```

```

        <soap:body use="literal"/>
    </output>
</operation>
</binding>
<service name="TesteWS">
    <port name="TesteWSPort" binding="tns:TesteWSPortBinding">
        <soap:address location="http://localhost:8080/ServidorWS/TesteWS"/>
    </port>
</service>
</definitions>

```

UDDI (Universal Description, Discovery and Integration) define mecanismos para publicar e descobrir web Services.

UDDI

- Universal Description, Discovery and Integration;
- Entidades que disponibilizam WS (organizações, empresas);
- Um local para armazenar informações sobre web Services;
- Usado com web Services baseados em WSDL;
- Comunicação via SOAP;
- Define mecanismos para publicar e descobrir web Services;
- Camada sobre o SOAP, requests e responses são objetos UDDI enviados via SOAP.



REST

REST: Representational State Transfer.

- Tese de Doutorado de Roy Fielding.
- Meados de 2000.
- Interface web simples, sem abstrações de protocolos ou padrões de trocas de mensagens.
- Biblioteca JAX-RS: java API for RESTful web Services.
 - Define como implementar os web Services REST.



O outro tipo de web é o REST (Representational State Transfer), baseado na tese de doutorado de Roy Fielding, de meados de 2000.

REST define interface web simples, sem abstrações de protocolos ou padrões de trocas de mensagens, usa os mecanismos do HTTP e identificação através de URIs.

Os métodos de acesso a recursos são feitos com métodos do HTTP, por exemplo:

- GET, PUT, DELETE idempotentes;
- POST não idempotente.

Os retornos são feitos através dos códigos de retorno HTML, exemplo: 404, 201, 500 etc.

Por ser basicamente requisições HTML, os serviços são acessíveis através de qualquer linguagem de programação. Recursos como autenticação, criptografia e autorização podem ser usados diretamente do HTML.

Ao contrário do SOAP, não há necessidade de se codificar as mensagens em XML (e posteriormente decodificá-las), visto que o próprio método HTTP de acesso ao recurso já indica qual é a operação a ser efetuada.

Não há necessidade de codificação em XML.

- O método HTTP indica a operação.

Exemplo de acesso a um recurso cliente:

- GET /cliente/{id} – Recupera o cliente id;
- DELETE /cliente/{id} – Deleta o cliente id;
- POST /cliente – Cria o cliente;
- GET /cliente – Retorna todos.

Os códigos de retorno são os do HTTP, por exemplo:

- 404 – não encontrado;
- 500 – Erro desconhecido do servidor;
- 201 – criado.

A figura a seguir representa o funcionamento de um web Service RESTful.

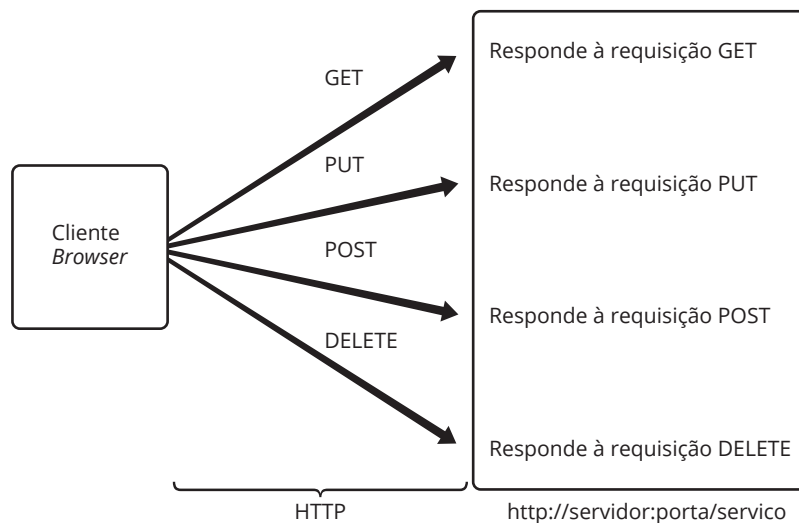


Figura 10.2
web Service
RESTful.

Um web Services RESTful implementado com JAX-RS usa anotações para definir que método é invocado em cada chamada do HTTP

- **@GET**: chamada via GET;
- **@POST**: chamada via POST;
- **@PUT**: chamada via PUT;
- **@POST**: chamada via POST;
- **@DELETE**: chamada via DELETE.

Outras anotações também são utilizadas, como @Path e @PathParam. O código a seguir mostra um trecho de um web Service implementado com JAX-RS.

```
@Path("/cliente")
public class ClienteResource {
    @GET
    public String getClientes (){
        // retorna todos os clientes
    }
    @GET
    @Path("/{id}")
    public String getClientes (@PathParam("id") String id){
        // retorna cliente com id
    }
}
```

Os formatos de consumo de web Services pode ser de duas formas:

- XML – Extensible Markup Language:
 - Dados representados com tags configuráveis.
- JSON – JavaScript Object Notation:
 - Dados representados com objetos em JavaScript;
 - Uma notação do objeto em Strings.

Os exemplos a seguir apresentam o dado Pessoa contendo Nome e Descrição nas duas notações:

XML

```
< Pessoa >
  < nome >Razer Montano</ nome >
  < descricao >Professor</ descricao >
</ Pessoa >
```

JSON – JavaScript Object Notation

```
{ "nome": "Razer Montano", "descricao": "Professor" }
```

JAXB (Java Architecture for XML Binding) é uma arquitetura de mapeamento entre objetos Java e XML. Basicamente, transforma objetos Java em texto XML e vice-versa.

Java Architecture for XML Binding

- Mapeamento Java < > XML;
- Transforma objetos Java em texto XML e vice-versa;
- Netbeans já cria códigos específicos;
- Exemplo de uso:

```
@XmlRootElement
public class Conta {
    private double saldo;
    private double limite;
    private Cliente cliente;
}
```

SOA – Arquitetura Orientada a Serviços

SOA

- Service Oriented Architecture: arquitetura baseada no paradigma de orientação a serviços.
- Sistemas são construídos por funcionalidades, de forma desacoplada.
- Favorece Reuso e Governança (alinhamento de estratégias de negócio x TI).
- Implementado de várias formas, por exemplo, web Services, SOAP, REST.

O conceito de SOAP e web Services não deve ser confundido com o conceito de SOA (Service Oriented Architecture). SOA é uma arquitetura que se baseia no paradigma de orientação a serviços, onde as funcionalidades dos sistemas são desenvolvidos de forma desacoplada, favorecendo o reuso e alinhamento dos objetivos de negócio e as estratégias de TI.

Segundo o Gartner Group: “SOA é uma abordagem arquitetural corporativa que permite a criação de serviços de negócio interoperáveis que podem facilmente ser reutilizados e compartilhados entre aplicações e empresas.”

SOA não é:

- Uma tecnologia;
- web Services.

SOA é:

- Filosofia de TI;
- Fácil integração entre sistemas atuais e legados;
- Baseado no conceito de serviços.

Dessa forma, SOAP, web Services e REST são formas ou padrões de implementação de Arquitetura SOA.

Um modelo de Arquitetura de Software tipicamente usado em aplicações baseadas em SOA é o Enterprise Service Bus (ESB). O ESB é uma arquitetura modular, baseada em componentes que têm como objetivo ser um canal de comunicação transparente e padronizado entre consumidores de serviços e seus provedores. As figuras a seguir ilustram esse conceito.

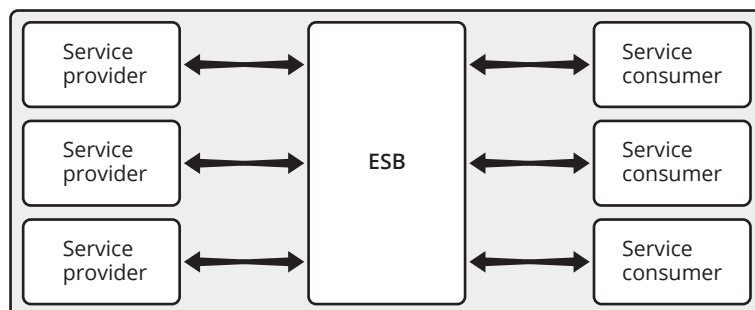


Figura 10.3
Ilustração do ESB.

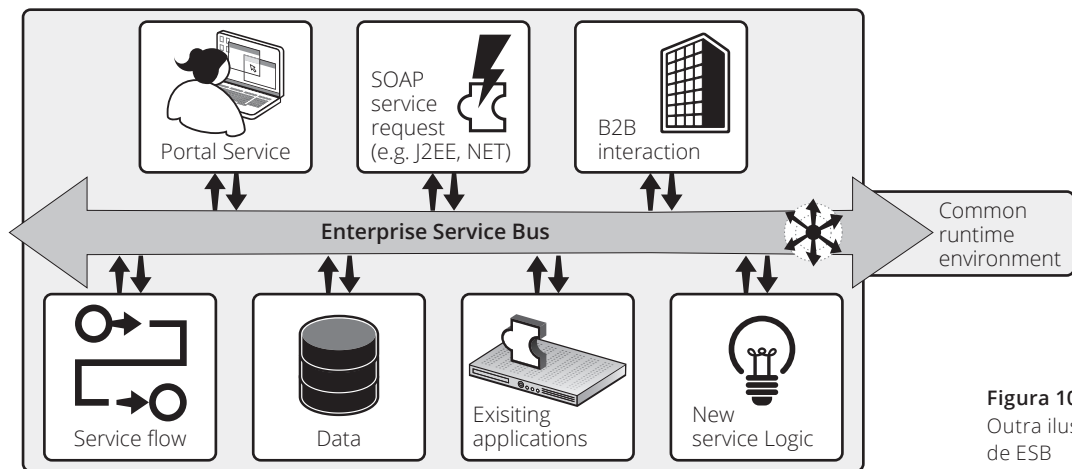


Figura 10.4
Outra ilustração
de ESB



Razer Anthom Nizer Rojas Montaña

é Doutorando em Informática (ênfase em Inteligência Artificial) pela UFPR, Mestre e Bacharel em Informática pela UFPR. Atualmente é professor da UFPR ministrando disciplinas de desenvolvimento em Java Web e de Aplicações

Corporativas. Possui certificação SCJP, COBIT, ITIL. Acumula mais de 15 anos de experiência em docência e mais de 20 anos de experiência no mercado de desenvolvimento, análise e arquitetura de aplicações.

O curso aborda o uso de frameworks e tecnologias para desenvolvimento de aplicações corporativas em Java. Este tipo de aplicação exige um grau de confiabilidade e performance mais elevado, fazendo uso de recursos específicos do Java Enterprise Edition - Java EE, tais como JSF (Java Server Faces), AJAX, Primefaces e Hibernate.

O livro inicia com uma visão geral da Arquitetura do Java EE e características dos servidores de aplicação capazes de suportar as tecnologias/aplicações corporativas, para em seguida se aprofundar no JSF e Hibernate.

ISBN 978-85-63630-56-8

